MACSE502 Programming for Data Science

Question bank with answers, Modules 1 to 6

55 multiple-choice questions, 51 short-answer questions and 26 long questions, each with a model answer. Questions follow your syllabus (ETH, version 1.0), the high-frequency list in your FAT notes and the 20 lab experiments. Code answers were run, and the real output is printed beside them.

What is inside

Module	Title	Hours	MCQ	Short	Long	Marks
1	Python data structures, OOP, APIs and NLP	8 hours	10	9	6	104
2	Python libraries for data science	9 hours	10	9	6	103
3	Scala data structures and object-oriented programming	12 hours	11	10	5	93
4	Scala libraries and Spark basics	8 hours	11	11	5	101
5	Programming language selection criteria	6 hours	8	8	2	53
6	Contemporary issues	2 hours	5	4	2	41

Marks: 1 per multiple-choice question, the stated marks for short and long questions. The marks given are for self-scoring; they are not the official weights of your paper.

How to use it

1. Pick a module. Do **Part 1 (Test yourself)** on paper without looking at the answers. Time yourself: about 1 minute per MCQ, 4 minutes per 3-mark answer, 8 minutes per 5-mark answer, 20 to 25 minutes per 10-mark answer.
2. Open **Part 2 (Answers)**. Tick each point of the model answer that you covered and score yourself with the suggested marks split under each long answer.
3. Write your scores in the scorecard at the back. Anything below 60 percent goes on the revision list; redo it after one day and again after one week.
4. For code questions, type the code yourself, run it, and compare your output with the printed output.
5. Finish with the mock paper at the back (it samples every module).

What was verified

Every program in this book was executed: Python 3.10 for modules 1, 2 and 5; Scala 2.13.12 on Java 17 (through scala-cli) for modules 3 and 4, including Breeze, Spire, Saddle and Spark 3.5.1 in local mode. The REST API examples use a mock server that returns real `requests.Response` objects, so no live service was called. ChatterBot 1.2.15 was run for the chatbot question. Details are in the appendix at the back.

Watch out. Module 6 has no topic list in the syllabus, and I could not find a published VIT marking scheme for this course. Treat the writing advice as sound general practice and check it against two or three past papers (links at the back).

How to write answers that score

What follows comes from three sources: the way your FAT notes phrase the likely questions (Explain, Compare, Detail, Define, Evaluate, State, Contrast, Write code), general exam-writing practice, and what a marker needs in order to give a mark quickly. It is not an official VIT rubric.

1. Read the command word first

Command word	What to produce	Example from your FAT notes
Define, State, What is	one or two precise sentences, then the key properties	What is an RDD? Define its four properties.
Explain, Describe, Detail	what it is, how it works, one example or diagram	Detail four built-in capabilities of case classes.
Compare, Differentiate, Contrast	a table with at least four rows, then one line saying when to use each	Compare BeautifulSoup and Scrapy.
Write code, Implement, Develop	clean code with comments, sample input, printed output, two lines of explanation	Write code demonstrating `groupby` with multiple aggregations.
Trace, Show the output	a step-by-step table of values	Stack contents after each undo/redo step.
Design, Propose	components (classes with attributes and methods), flow diagram, how parts interact	Propose an object-oriented design for the chatbot.
Evaluate, Justify, Discuss	criteria, evidence taken from the scenario, trade-off, conclusion	Evaluate Python vs Scala across the selection criteria.
List	short numbered points, no essay	List 3 transformations and 3 actions.

2. Match the length to the marks

Marks	Typical length	Contents
1	one line or one option	the fact
2 to 3	3 to 6 lines	definition + 2 to 3 points (or a 3-row table) + a one-line example
5	half a page	definition, 4 to 5 points, small diagram or code, example
10	two pages	introduction, headed sections, diagram or table, code with output, worked example, one-line conclusion

These are rules of thumb. If the paper has a different pattern, follow the question's marks and the space given.

3. Structure of a 10-mark answer

1. Definition in two lines (use the textbook's term).
2. Diagram or comparison table (label every part; examiners often award marks just for the diagram).
3. Numbered explanation, one idea per point, each with the reason "because ...".
4. Code or worked example, with output.
5. Link to the scenario in the question (use its numbers and names).
6. One-line conclusion that answers the question asked.

4. Code answers

- Indent consistently, use meaningful names, and put a comment on each step that carries marks.
- Show the output (copy it from a real run). A correct output is strong evidence that the code works.
- Handle the error case if the question mentions failure (try/except in Python, `Try` or `Option` in Scala).
- State the complexity or the reason for a choice in one line, for example "`deque.popleft()` is O(1), `list.pop(0)` is O(n)".

5. Words that tell the marker you know the topic

Module	Use these terms where they apply
1	hashable, O(1) average, LIFO, FIFO, min-heap, idempotent, status code, retry with back-off, fail fast, `with` closes the file, tokenization, lemmatization
2	vectorised, broadcasting, split-apply-combine, `loc` is label based, `iloc` is positional, inner/left/outer, non-blocking, event loop, standardise before PCA, inertia

Module	Use these terms where they apply
3	immutable, `val` vs `var`, structural equality, companion object, `apply`, `unapply`, trait mixin, linearization, `override`, `final`, pattern matching
4	driver, executor, cluster manager, partition, DAG, lineage, lazy evaluation, narrow vs wide, shuffle, action, Catalyst, `Option`/`Try`/`Either`
5	criteria, trade-off, weighted decision matrix, time to production, elasticity, static vs dynamic typing, justify with the scenario
6	consent, data minimisation, pseudonymisation, bias, hallucination, drift, accountability

6. Common slips that cost marks

- No output shown after the code.
- Mixing Python and Scala syntax: `val x = 5` vs `x = 5`; `a[0]` (Python) vs `a(0)` (Scala array); forgetting `new` in Scala when there is no companion `apply`.
- Python: calling a list "faster than a tuple" as the only difference; using `loc` when you mean `iloc`; a bare `except:`.
- Spark: saying a transformation "runs" before an action; forgetting that `collect` brings all data to the driver; saying Spark "stores" data.
- Scala: `==` vs `eq`; calling an immutable `List` mutable because the `var` changed.
- A comparison answer written as paragraphs instead of a table.

Tip. In the lab viva you may be asked to explain your own code line by line. Be able to run each lab file and say what each combinator, constructor or DataFrame call returns.

Contents

How to write answers that score 2

Module 1: Python data structures, OOP, APIs and NLP 5

Module 1: Test yourself (questions only) . . . 5

Module 1: Answers . . . 9

M1-C1. Campus placement shortlisting [10 marks] . . . 11

M1-C2. Text editor with undo and redo [10 marks] . . . 13

M1-C3. Trending keywords from news headlines [10 marks] . . . 15

M1-C4. Hostel complaint chatbot [10 marks] . . . 18

M1-C5. Asset price alert system [10 marks] . . . 21

M1-C6. Student record manager [10 marks] . . . 24

Module 2: Python libraries for data science 27

Module 2: Test yourself (questions only) . . . 27

Module 2: Answers . . . 30

M2-C1. Data manipulation and cleaning with pandas [10 marks] . . . 31

M2-C2. Split-apply-combine and merging DataFrames [10 marks] . . . 33

M2-C3. NumPy arrays and broadcasting [10 marks] . . . 34

M2-C4. Beautiful Soup versus Scrapy [10 marks] . . . 35

M2-C5. Python visualisation libraries [10 marks] . . . 37

M2-C6. EDA, K-means, PCA and the modelling libraries [10 marks] . . . 38

Module 3: Scala data structures and object-oriented programming 41

Module 3: Test yourself (questions only) . . . 41

Module 3: Answers . . . 44

M3-C1. Scala collections and functional combinators [10 marks] . . . 45
M3-C2. Classes, constructors, inheritance and companion objects [10 marks] . . . 46
M3-C3. Case classes and pattern matching [10 marks] . . . 48
M3-C4. Traits, mixins and linearization [10 marks] . . . 49
M3-C5. Scala fundamentals [10 marks] . . . 50

Module 4: Scala libraries and Spark basics **52**

Module 4: Test yourself (questions only) . . . 52
Module 4: Answers . . . 55
M4-C1. Exception handling in Scala [10 marks] . . . 57
M4-C2. Scala libraries: Breeze, Saddle and Spire [10 marks] . . . 58
M4-C3. Apache Spark architecture [10 marks] . . . 59
M4-C4. RDDs, lineage, transformations and actions [10 marks] . . . 60
M4-C5. Spark components and the Catalyst optimiser [10 marks] . . . 61

Module 5: Programming language selection criteria **63**

Module 5: Test yourself (questions only) . . . 63
Module 5: Answers . . . 65
M5-C1. Python versus Scala versus Java versus R [10 marks] . . . 66
M5-C2. Language selection case studies with a decision matrix [10 marks] . . . 67

Module 6: Contemporary issues **69**

Module 6: Test yourself (questions only) . . . 69
Module 6: Answers . . . 71
M6-C1. Responsible data science: a hospital pipeline [10 marks] . . . 71
M6-C2. AI coding assistants in data-science work [10 marks] . . . 72

Self-scorecard **73**

Mock paper (100 marks, 3 hours) **73**

Study material **75**

Appendix: how the code was verified **77**

Module 1: Python data structures, OOP, APIs and NLP

Syllabus (8 hours): conditions and branching; list, tuple, dictionary, set; user-defined stack, queue, priority queue; string handling; exception handling; object-oriented concepts; APIs and data collection (simple and REST APIs, HTTP requests); file handling; frameworks and libraries: NLTK, ChatterBot.

Revision table: built-in and user-defined structures

Structure	Mutable	Ordered	Duplicates	Typical cost	Use
list	yes	yes	yes	index O(1); `in` O(n); insert/pop(0) O(n); append O(1) amortised	sequences, stack
tuple	no	yes	yes	index O(1)	fixed records, dict keys
dict	yes	insertion order	keys unique	lookup, insert O(1) average	key -> value lookup
set	yes	no	no	membership O(1) average	uniqueness, set algebra
stack (list)	yes	LIFO	yes	push/pop at the end O(1)	undo, recursion, parsing
queue (deque)	yes	FIFO	yes	append / popleft O(1)	scheduling, BFS
priority queue (heapq)	yes	by priority	yes	push/pop O(log n); peek O(1)	top-k, scheduling by importance

Tip. Module 1 questions in this bank are written as **mini case studies**, because that is how applied questions are usually framed: you must pick structures, justify them, then show code with output. Practise explaining *why* a structure fits.

Watch out. Do not write "tuple is faster than list" as your only difference. State immutability, hashability and use.

Module 1: Test yourself (questions only)

A. Multiple choice (10 questions, 1 mark each)

M1-A1. Which built-in structure is the best choice to test whether a roll number is among 10 million stored roll numbers?

(a) list
(b) tuple
(c) set
(d) str

M1-A2. `heapq.heappop(h)` **on a list** `h` **managed by heapq returns:**

(a) the largest item
(b) the smallest item
(c) the most recently pushed item
(d) the oldest item

M1-A3. `d = {'a': 1}; d.get('z', 0)` **evaluates to:**

(a) KeyError
(b) None
(c) 0
(d) 'z'

M1-A4. Which statement about `finally` **is correct?**

(a) It runs only if an exception occurs
(b) It runs only if no exception occurs
(c) It runs whether or not an exception occurs
(d) It replaces `except`

M1-A5. Which HTTP status code means "Too Many Requests"?

(a) 401
(b) 404
(c) 429
(d) 503

M1-A6. File mode `"a"` does what?

(a) Reads the file
(b) Truncates the file then writes
(c) Appends to the end and creates the file if missing
(d) Opens for read and write without creating

M1-A7. Inside `class A`, the attribute `self.__x` is stored under the name:

(a) `__x`
(b) `_A__x`
(c) `A.__x`
(d) `x`

M1-A8. Which operation turns "geese" into "goose"?

(a) Tokenization
(b) Stemming (Porter)
(c) Lemmatization
(d) Stopword removal

M1-A9. Which can be used as a dictionary key?

(a) `[1, 2]`
(b) `{1, 2}`
(c) `(1, 2)`
(d) `{'a': 1}`

M1-A10. Removing the first element repeatedly from a very long Python `list` (`pop(0)`) is slow because:

(a) lists are immutable
(b) all remaining elements are shifted, O(n) per pop
(c) pop(0) copies the whole list to disk
(d) Python locks the list

B. Short answers

M1-B1. Differentiate between a list and a tuple. When would you prefer a tuple? [3 marks]

M1-B2. Explain `try`, `except`, `else` and `finally` with a small example. [5 marks]

M1-B3. Implement a stack and a queue using Python built-ins and state the cost of each operation. [5 marks]

M1-B4. How is a priority queue built in Python? How do you get highest-priority-first behaviour? [3 marks]

M1-B5. Compare HTTP GET and POST. [3 marks]

M1-B6. Explain public, protected and private members in Python. What do `@classmethod` and `@staticmethod` do? [5 marks]

M1-B7. List the common file modes and explain why `with open(...)` is preferred. [3 marks]

M1-B8. Define tokenization, stopwords, stemming and lemmatization (NLTK). [5 marks]

M1-B9. State the five classes of HTTP status codes and give one example for each. [2 marks]

C. Long questions

M1-C1. Campus placement shortlisting [10 marks]

A placement cell shortlists students for companies. A student is eligible for a company if the CGPA is at least the company's minimum and the student has no backlogs.

Roll No	Name	CGPA	Backlogs
26AIM1001	Ananya Rao	8.6	0
26AIM1002	Rahul Verma	7.8	1
26CSE1003	Sneha Iyer	9.1	0
26DSC1004	Kabir Shah	7.2	0
26ECE1005	Meera Nair	8.2	0

Company	Minimum CGPA	Openings
TechNova	8.0	2
DataWorks	7.5	1
CloudSys	7.0	3

1. Store the students and companies using suitable built-in data structures.
2. Use conditional statements to find the eligible students for each company and keep them as sets.
3. Using set operations, find the students eligible for all three companies, for none, and for CloudSys only.
4. Use a priority queue (highest CGPA first) to shortlist students up to each company's openings.
5. Using string handling, validate each roll number (2 digits, 3 letters, 4 digits) and extract the branch code.
6. Handle the exception when a user asks about a company that is not in the dataset.
7. Write the shortlists to a file, then read the file back and display it.
8. Suggest where lists, tuples, dictionaries, sets, stacks, queues and priority queues can be applied in the system.

M1-C2. Text editor with undo and redo [10 marks]

A lightweight text editor must support: typing text (appended to the document), deleting the last n characters, undo and redo of the last operations, and saving to and loading from a file.

Propose an object-oriented design by identifying suitable classes, attributes and methods. Explain how two stacks implement undo and redo, how string handling performs the edits, and how exception handling (deleting more characters than exist, a missing file) and file handling make the editor reliable.

Trace this sequence and show the document text and both stacks after each step: `type("Hello")`, `type(" World")`, `delete(5)`, `undo()`, `undo()`, `redo()`.

M1-C3. Trending keywords from news headlines [10 marks]

A media company collects news headlines through a REST API (for example, `GET /headlines?category=tech`) and wants to find trending keywords. The application should send HTTP requests for several categories; handle network failures and invalid or empty JSON; use NLTK to tokenize each headline, remove stopwords and lemmatize; count keyword frequency per category and find keywords unique to one category; and save the top five keywords per category to a file.

Explain the complete workflow. Discuss how the API, NLTK, exception handling, dictionaries, sets and file handling work together to produce reliable results.

M1-C4. Hostel complaint chatbot [10 marks]

A hostel plans a chatbot using NLTK and ChatterBot to handle student complaints and queries. The chatbot should identify the complaint type from keywords (water, Wi-Fi, electricity, mess); treat urgent complaints (no water, power failure) before routine ones; answer common questions such as mess timings and the warden's contact; handle unclear or invalid messages; and log every complaint to a file.

Propose an object-oriented design with classes, attributes and methods. Explain how NLTK, ChatterBot, a priority queue, exception handling and file handling can be combined to improve accuracy and user experience.

M1-C5. Asset price alert system [10 marks]

An application tracks the prices of several assets through a REST API. It should send HTTP GET requests at regular intervals; keep the last five readings of each asset as `(timestamp, price)` pairs; raise an alert when a price goes above an upper limit or below a lower limit; handle timeouts, HTTP errors (such as 404 and 429) and invalid JSON, retrying where sensible; and append every alert to a log file.

Explain the workflow. Discuss GET versus POST, the meaning of common status codes, JSON parsing, `try / except / else / finally`, and how lists, tuples, dictionaries, conditionals and file modes (r, w, a) work together to keep the system robust.

M1-C6. Student record manager [10 marks]

A department stores student records in a CSV file and processes them in Python.

```
reg_no,name,email,marks
VL2026001,  asha nair ,asha@vit.ac.in,86
VL2026002,Rohan Das,rohan.vit.ac.in,74
VL2026003,Meena Rao,meena@vit.ac.in,abc
VL2026004,Kabir Shah,kabir@vit.ac.in,58
```

1. Design a `Person` class and a `Student` class that inherits from it, with attributes and methods for the record.
2. Read the file using the `csv` module and create `Student` objects.
3. Using string methods, strip and title-case each name, and validate the email (it must contain `@` and end in `.ac.in`).
4. Grade with conditionals: 85 and above is A, 70 to 84 is B, 50 to 69 is C, below 50 is Fail.
5. Handle malformed rows with exception handling (an invalid email or non-numeric marks). Skip them and log the reason to a file.
6. Store the valid students in a list of objects and a dictionary keyed by register number.
7. Save the final report as a JSON file using the `json` module.
8. Explain how OOP, strings, exceptions and file handling work together here.

Module 1: Answers

A. Multiple-choice key

No.	Ans.	Why
M1-A1	c	A set is a hash table, so `in` is O(1) on average. A list or tuple scans every element, O(n).
M1-A2	b	heapq implements a min-heap. For a max-heap push negated priorities.
M1-A3	c	`get` returns the default (0) when the key is missing; `d['z']` would raise KeyError.
M1-A4	c	`finally` always runs (even after `return`), so it is the place for clean-up such as closing a connection.
M1-A5	c	429 is rate limiting. 401 = unauthorised, 404 = not found, 503 = service unavailable.
M1-A6	c	`"w"` truncates, `"a"` appends, `"r"` reads (error if missing), `"r+"` reads and writes an existing file.
M1-A7	b	Name mangling rewrites `__x` to `_ClassName__x` to avoid accidental clashes in subclasses. It is not true privacy.
M1-A8	c	A lemmatizer uses a dictionary (WordNet) and returns a real word. A stemmer only chops suffixes.
M1-A9	c	Keys must be hashable. Tuples of hashable items are; lists, sets and dicts are mutable and unhashable.
M1-A10	b	Use `collections.deque` and `popleft()` (O(1)) for queues.

B. Short answers

M1-B1. Differentiate between a list and a tuple. When would you prefer a tuple? [3 marks]

Point	List	Tuple
Mutability	Mutable: append, insert, remove, item assignment	Immutable: fixed after creation
Syntax	`[1, 2, 3]`	`(1, 2, 3)`; a one-item tuple needs a comma: `(1,)`
Hashable	No, cannot be a dict key or set member	Yes, if all its items are hashable
Speed / memory	Over-allocates for growth	Slightly smaller and faster to create

Prefer a tuple for a fixed record such as `(name, cgpa, backlogs)`, for dictionary keys such as `(row, col)`, and for returning several values from a function.

M1-B2. Explain `try`, `except`, `else` and `finally` with a small example. [5 marks]

`try` holds the risky code. `except` handles a named exception. `else` runs only when the `try` block raised nothing. `finally` always runs, so it holds clean-up code.

```
def read_int(text):
    try:
        n = int(text)             # may raise ValueError
    except ValueError:
        print("not a number"); return None
    else:
        print("parsed OK"); return n     # only when no exception
    finally:
        print("done")                # always, even after return
```

Good practice: catch the narrowest exception, keep the `try` block small, never use a bare `except:`, and raise custom exceptions (`class AppError(Exception)`) for domain errors.

M1-B3. Implement a stack and a queue using Python built-ins and state the cost of each operation. [5 marks]

```
from collections import deque

stack = []                  # LIFO
stack.append(1); stack.append(2)
top = stack.pop()           # 2, O(1) at the end

queue = deque()             # FIFO
queue.append(1); queue.append(2)
first = queue.popleft()    # 1, O(1)
```

A list is a good stack. For a queue use `deque` because `list.pop(0)` shifts every element (O(n)). Stack uses: undo/redo, bracket matching, recursion. Queue uses: task scheduling, BFS, print spooling.

M1-B4. How is a priority queue built in Python? How do you get highest-priority-first behaviour? [3 marks]

`heapq` keeps a list arranged as a binary min-heap: `heappush` and `heappop` cost O(log n), looking at `h[0]` is O(1).

```
import heapq
h = []
heapq.heappush(h, (-9.1, 'Sneha'))   # negate for max-first
heapq.heappush(h, (-8.6, 'Ananya'))
print(heapq.heappop(h))        # (-9.1, 'Sneha')
```

Tuples compare element by element, so ties on priority fall back to the next item; add a running counter `(priority, counter, item)` to keep first-come-first-served order and to avoid comparing non-comparable objects.

M1-B5. Compare HTTP GET and POST. [3 marks]

	GET	POST
Purpose	Read a resource	Create or submit data
Data location	URL query string (`?category=tech`)	Request body (JSON, form)
Idempotent / safe	Yes, repeating it changes nothing	No, repeating can create duplicates
Caching / bookmarking	Can be cached and bookmarked	Normally not cached
Size / secrecy	URL length limits; data visible in logs	Large bodies; not shown in the URL

M1-B6. Explain public, protected and private members in Python. What do `@classmethod` and `@staticmethod` do? [5 marks]

- `name`: public. `_name`: protected by convention (a hint, not enforced). `__name`: private by name mangling (stored as `_Class__name`).
- `@classmethod` receives the class (`cls`) and is used for alternative constructors, e.g. `from_dict(cls, d)`.
- `@staticmethod` receives neither `self` nor `cls`; it is a plain utility function placed in the class namespace, e.g. `validate_name(name)`.
- Instance methods receive `self` and work on one object's data.

```
class Pipeline:
    def __init__(self, name):
        self.name = name
        self.__key = 'token'              # mangled to _Pipeline__key
    @classmethod
    def from_dict(cls, d):
        return cls(d['name'])
    @staticmethod
    def validate_name(n):
        return n.isalnum()
```

M1-B7. List the common file modes and explain why `with open(...)` is preferred. [3 marks]

- `r` read (error if missing), `w` write (creates or truncates), `a` append, `x` create (error if exists), `r+` read and write; add `b` for binary and `encoding='utf-8'` for text.
- `with` closes the file automatically, even when an exception occurs, so data is flushed and the handle is released.

M1-B8. Define tokenization, stopwords, stemming and lemmatization (NLTK). [5 marks]

Term	Meaning	Example
Tokenization	Split text into words or sentences (`word_tokenize`)	"AI is fun" -> ['AI','is','fun']
Stopwords	Very common words with little meaning (`stopwords.words('english')`)	the, is, of
Stemming	Rule-based suffix chopping (`PorterStemmer`); output may not be a word	studies -> studi
Lemmatization	Dictionary-based reduction to a valid base form (`WordNetLemmatizer`); better with a POS tag	geese -> goose; `lemmatize('running', pos='v')` -> run

M1-B9. State the five classes of HTTP status codes and give one example for each. [2 marks]

1xx informational (100 Continue), 2xx success (200 OK, 201 Created), 3xx redirection (301 Moved Permanently), 4xx client error (400, 401, 403, 404, 429), 5xx server error (500, 503).

C. Long answers

M1-C1. Campus placement shortlisting [10 marks]

1. Choosing the structures

Data	Structure	Reason
Students	`dict`: roll no -> `tuple (name, cgpa, backlogs)`	O(1) lookup by roll number; a tuple is a fixed, read-only record
Companies	`dict`: name -> `tuple (min_cgpa, openings)`	O(1) lookup by company; fixed record
Eligible students	`set` of roll numbers per company	unique members and built-in set algebra
Shortlist order	`heapq` priority queue of `(-cgpa, roll)`	pops the best CGPA first in O(log n)

2. Eligibility and set operations

A loop with an `if cgpa >= min_cgpa and backlogs == 0` test adds the roll number to the company's set. Then, with `E` the dictionary of eligible sets:

- all three: `set.intersection(*E.values())`
- none: `all_students - set.union(*E.values())`
- CloudSys only: `E['CloudSys'] - E['TechNova'] - E['DataWorks']`

3. Priority queue

`heapq` is a min-heap, so each entry is pushed as `(-cgpa, roll)`. Popping repeatedly returns the highest CGPA first; stop after `openings` pops. The roll number acts as the tie-breaker.

4. Strings, exception, file

- Validation uses `len(roll) == 9`, `roll[:2].isdigit()`, `roll[2:5].isalpha()`, `roll[5:].isdigit()`; the branch is the slice `roll[2:5]`. A regular expression `^\d{2}[A-Za-z]{3}\d{4}$` does the same in one line.
- An unknown company raises a custom `CompanyNotFoundError`; `try / except / else / finally` reports it without crashing.
- `with open('shortlists.txt', 'w')` writes; reopening with `'r'` reads the content back.

5. Where each structure applies

Structure	Use in this system
List	Ordered shortlist for display; list of companies in application order
Tuple	Immutable student record; heap entries `(-cgpa, roll)`; `(company, openings)` pairs
Dictionary	Roll number -> student, company -> rules, company -> eligible set
Set	Eligibility, removing duplicate applications, set algebra across companies
Stack	Undo the last allocation; "back" navigation in the placement portal; checking nested rule expressions
Queue (FIFO)	Interview slots or application processing in arrival order
Priority queue	Ranking by CGPA; later, urgent or premium applications first

Code: code/m1_q1_placement.py

```
"""M1-Q1  Campus Placement Shortlisting
Topics: conditionals, dict/tuple/set, set operations, priority queue (heapq),
string handling, custom exception, file handling.
"""
import heapq
import re

# --- 1. Data kept in built-in structures -------------------------------
# dict keyed by roll no (fast lookup); each record is a tuple (fixed, read-only)
students = {
    "26AIM1001": ("Ananya Rao", 8.6, 0),
    "26AIM1002": ("Rahul Verma", 7.8, 1),
    "26CSE1003": ("Sneha Iyer", 9.1, 0),
    "26DSC1004": ("Kabir Shah", 7.2, 0),
    "26ECE1005": ("Meera Nair", 8.2, 0),
}
# company -> (minimum CGPA, openings)
companies = {
    "TechNova": (8.0, 2),
    "DataWorks": (7.5, 1),
    "CloudSys": (7.0, 3),
}
```

```
class CompanyNotFoundError(Exception):
    """Raised when a company is not in the dataset."""


# --- 2. Eligibility with conditionals -> sets ---------------------------
def eligible_set(company):
    if company not in companies:
        raise CompanyNotFoundError(f"'{company}' is not in the dataset")
    min_cgpa, _ = companies[company]
    result = set()
    for roll, (name, cgpa, backlogs) in students.items():
        if cgpa >= min_cgpa and backlogs == 0:   # eligibility rule
            result.add(roll)
    return result


eligible = {c: eligible_set(c) for c in companies}
print("Eligible sets")
for c, s in eligible.items():
    print(f"  {c:<10}: {sorted(s)}")

# --- 3. Set operations --------------------------------------------------
all_students = set(students)
in_all = set.intersection(*eligible.values())
in_any = set.union(*eligible.values())
in_none = all_students - in_any
cloud_only = eligible["CloudSys"] - eligible["TechNova"] - eligible["DataWorks"]
print("\nEligible for all three :", sorted(in_all))
print("Eligible for none      :", sorted(in_none))
print("CloudSys only          :", sorted(cloud_only))


# --- 4. Priority queue (highest CGPA first) -----------------------------
def shortlist(company):
    _, openings = companies[company]
    heap = []
    for roll in eligible_set(company):
        cgpa = students[roll][1]
        heapq.heappush(heap, (-cgpa, roll))      # heapq is a min-heap -> negate
    chosen = []
    while heap and len(chosen) < openings:
        neg_cgpa, roll = heapq.heappop(heap)
        chosen.append((roll, students[roll][0], -neg_cgpa))
    return chosen


shortlists = {c: shortlist(c) for c in companies}
print("\nShortlists (top CGPA, limited to openings)")
for c, rows in shortlists.items():
    print(f"  {c} ({companies[c][1]} openings):")
    for roll, name, cgpa in rows:
        print(f"     {roll}  {name:<12} {cgpa}")


# --- 5. String handling: validate roll no and extract branch ------------
def validate_roll(roll):
    """2 digits + 3 letters + 4 digits. Returns branch code or None."""
    roll = roll.strip().upper()
    if len(roll) == 9 and roll[:2].isdigit() and roll[2:5].isalpha() and roll[5:].isdigit():
        return roll[2:5]
    return None


print("\nRoll number validation")
for r in list(students) + ["2AIM1001", "26AI1001X", "26aim1006"]:
    branch = validate_roll(r)
    print(f"  {r:<10} -> {'valid, branch ' + branch if branch else 'INVALID'}")

# the same check with a regular expression
pattern = re.compile(r"^\d{2}[A-Za-z]{3}\d{4}$")
assert all(pattern.match(r) for r in students)

# --- 6. Exception handling ----------------------------------------------
print("\nQuery handling")
for query in ["TechNova", "Zoho"]:
    try:
        names = [students[r][0] for r in sorted(eligible_set(query))]
    except CompanyNotFoundError as err:
        print(f"  Error: {err}")
    else:
        print(f"  {query}: eligible = {names}")
    finally:
        print("  (query finished)")

# --- 7. File handling: write, then read back ----------------------------
with open("shortlists.txt", "w", encoding="utf-8") as f:
    for c, rows in shortlists.items():
        f.write(f"{c}\n")
        for roll, name, cgpa in rows:
```

```
            f.write(f"  {roll},{name},{cgpa}\n")

print("\nContents of shortlists.txt")
with open("shortlists.txt", "r", encoding="utf-8") as f:
    print(f.read(), end="")
```

Output

```
Eligible sets
  TechNova  : ['26AIM1001', '26CSE1003', '26ECE1005']
  DataWorks : ['26AIM1001', '26CSE1003', '26ECE1005']
  CloudSys  : ['26AIM1001', '26CSE1003', '26DSC1004', '26ECE1005']

Eligible for all three : ['26AIM1001', '26CSE1003', '26ECE1005']
Eligible for none      : ['26AIM1002']
CloudSys only          : ['26DSC1004']

Shortlists (top CGPA, limited to openings)
  TechNova (2 openings):
     26CSE1003  Sneha Iyer   9.1
     26AIM1001  Ananya Rao   8.6
  DataWorks (1 openings):
     26CSE1003  Sneha Iyer   9.1
  CloudSys (3 openings):
     26CSE1003  Sneha Iyer   9.1
     26AIM1001  Ananya Rao   8.6
     26ECE1005  Meera Nair   8.2

Roll number validation
  26AIM1001  -> valid, branch AIM
  26AIM1002  -> valid, branch AIM
  26CSE1003  -> valid, branch CSE
  26DSC1004  -> valid, branch DSC
  26ECE1005  -> valid, branch ECE
  2AIM1001   -> INVALID
  26AI1001X  -> INVALID
  26aim1006  -> valid, branch AIM

Query handling
  TechNova: eligible = ['Ananya Rao', 'Sneha Iyer', 'Meera Nair']
  (query finished)
  Error: 'Zoho' is not in the dataset
  (query finished)

Contents of shortlists.txt
TechNova
  26CSE1003,Sneha Iyer,9.1
  26AIM1001,Ananya Rao,8.6
DataWorks
  26CSE1003,Sneha Iyer,9.1
CloudSys
  26CSE1003,Sneha Iyer,9.1
  26AIM1001,Ananya Rao,8.6
  26ECE1005,Meera Nair,8.2
```

Suggested marks split. structures 2, eligibility and set operations 2, priority queue 2, strings 1, exception 1, file 1, applications of structures 1.

M1-C2. Text editor with undo and redo [10 marks]

1. Class design

Class	Attributes	Methods
`TextEditor`	`_text: str`, `_undo: list` (stack), `_redo: list` (stack)	`type_text(s)`, `delete(n)`, `undo()`, `redo()`, `save(path)`, `load(path)`, `text` (read-only property)
`EditorError(Exception)`	message	raised for a bad delete length, empty undo or redo, and file problems

Each stack entry is a tuple `(kind, data)`: `('T', 'Hello')` for typing and `('D', 'World')` for a deletion. A deletion stores the removed text, because that is the only way to reverse it.

2. How two stacks work

1. A new edit applies the change, pushes its record on the **undo** stack and clears the **redo** stack (a fresh edit makes the old redo history invalid).
2. **Undo** pops the top of the undo stack, applies the reverse action (remove typed text, or re-append deleted text) and pushes the record on the redo stack.
3. **Redo** pops the redo stack, re-applies the action and pushes it back on the undo stack.
4. Both stacks are Python lists used LIFO: `append` to push, `pop` to pop, each O(1).

3. Strings, exceptions, files

- Strings are immutable, so every edit builds a new string: typing is `text + s`, deleting n characters is the slice `text[:len(text) - n]`, and the deleted part is `text[len(text) - n:]`. (`text[:-n]` is wrong when n is 0.)
- `delete(n)` raises `EditorError` when `n < 0` or `n > len(text)`; `undo` and `redo` raise `EditorError` on an empty stack.
- `save` uses `with open(path, 'w', encoding='utf-8')`. `load` catches `FileNotFoundError` and re-raises it as `EditorError('file not found')`; a successful load clears both stacks because the old history no longer matches the text.

4. Trace

Step	Document text	Undo stack (bottom to top)	Redo stack
type("Hello")	Hello	T:Hello	empty
type(" World")	Hello World	T:Hello, T: World	empty
delete(5)	Hello (with trailing space)	T:Hello, T: World, D:World	empty
undo()	Hello World	T:Hello, T: World	D:World
undo()	Hello	T:Hello	D:World, T: World
redo()	Hello World	T:Hello, T: World	D:World

Code: code/m1_q2_editor.py

```
"""M1-Q2  Text editor with undo / redo using two stacks.
Topics: OOP, stack (list), string slicing, exceptions, file handling.
"""


class EditorError(Exception):
    """Any editor-level problem (bad delete length, nothing to undo, file issue)."""


class TextEditor:
    def __init__(self):
        self._text = ""
        self._undo = []     # stack of operations already done
        self._redo = []     # stack of operations that were undone

    @property
    def text(self):
        return self._text

    # ---- primitive actions (also used by undo / redo) ----
    def _apply(self, op, data):
        if op == "T":                       # type
            self._text += data
        else:                               # "D": delete -> data is the removed text
            self._text = self._text[: len(self._text) - len(data)]

    def _reverse(self, op, data):
        if op == "T":
            self._text = self._text[: len(self._text) - len(data)]
        else:
            self._text += data

    # ---- public operations ----
    def type_text(self, s):
        self._apply("T", s)
        self._undo.append(("T", s))
        self._redo.clear()                  # a new edit invalidates the redo history

    def delete(self, n):
        if n < 0 or n > len(self._text):
            raise EditorError(f"cannot delete {n} characters from a text of length {len(self._text)}")
        removed = self._text[len(self._text) - n:] if n else ""
        self._apply("D", removed)
        self._undo.append(("D", removed))
        self._redo.clear()

    def undo(self):
        if not self._undo:
            raise EditorError("nothing to undo")
        op, data = self._undo.pop()
        self._reverse(op, data)
        self._redo.append((op, data))

    def redo(self):
        if not self._redo:
            raise EditorError("nothing to redo")
        op, data = self._redo.pop()
        self._apply(op, data)
        self._undo.append((op, data))
```

```
    # ---- file handling ----
    def save(self, path):
        try:
            with open(path, "w", encoding="utf-8") as f:
                f.write(self._text)
        except OSError as err:
            raise EditorError(f"could not save '{path}': {err.strerror}") from err

    def load(self, path):
        try:
            with open(path, "r", encoding="utf-8") as f:
                self._text = f.read()
        except FileNotFoundError as err:
            raise EditorError(f"file '{path}' not found") from err
        self._undo.clear()
        self._redo.clear()

    def snapshot(self):
        return repr(self._text), self._undo[:], self._redo[:]


if __name__ == "__main__":
    ed = TextEditor()
    steps = [
        ('type("Hello")', lambda: ed.type_text("Hello")),
        ('type(" World")', lambda: ed.type_text(" World")),
        ("delete(5)", lambda: ed.delete(5)),
        ("undo()", ed.undo),
        ("undo()", ed.undo),
        ("redo()", ed.redo),
    ]
    print(f"{'step':<16}{'text':<16}undo stack / redo stack")
    for label, action in steps:
        action()
        text, undo, redo = ed.snapshot()
        print(f"{label:<16}{text:<16}{undo}  /  {redo}")

    print("\nException handling")
    for label, action in [("delete(50)", lambda: ed.delete(50)),
                          ("load('missing.txt')", lambda: ed.load("missing.txt"))]:
        try:
            action()
        except EditorError as err:
            print(f"  {label}: EditorError - {err}")

    ed.save("note.txt")
    ed2 = TextEditor()
    ed2.load("note.txt")
    print("\nsaved and reloaded:", repr(ed2.text))
```

Output

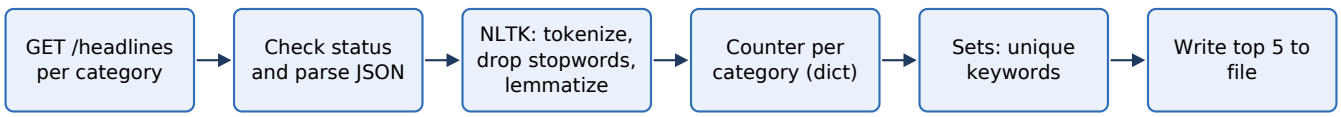
```
step            text            undo stack / redo stack
type("Hello")   'Hello'         [('T', 'Hello')]  /  []
type(" World")  'Hello World'   [('T', 'Hello'), ('T', ' World')]  /  []
delete(5)       'Hello '        [('T', 'Hello'), ('T', ' World'), ('D', 'World')]  /  []
undo()          'Hello World'   [('T', 'Hello'), ('T', ' World')]  /  [('D', 'World')]
undo()          'Hello'         [('T', 'Hello')]  /  [('D', 'World'), ('T', ' World')]
redo()          'Hello World'   [('T', 'Hello'), ('T', ' World')]  /  [('D', 'World')]

Exception handling
  delete(50): EditorError - cannot delete 50 characters from a text of length 11
  load('missing.txt'): EditorError - file 'missing.txt' not found

saved and reloaded: 'Hello World'
```

Suggested marks split. classes and methods 3, two-stack logic 3, trace 2, string handling 1, exceptions and files 1.

M1-C3. Trending keywords from news headlines [10 marks]



Workflow of the keyword application.

1. Fetching

- `requests.get(url, params={'category': cat}, timeout=5)`. The timeout is mandatory: without it a dead server hangs the program.

- `resp.raise_for_status()` turns 4xx and 5xx into `HTTPError`. `resp.json()` raises `ValueError` on invalid JSON. The code then checks that the key `headlines` exists and is a non-empty list.
- Network errors (`Timeout`, `ConnectionError`) are transient: retry a few times with a delay. A 404 or invalid JSON will not improve on retry, so skip that category and record the reason.

Failure	Exception raised	Action
Server unreachable or slow	`ConnectionError`, `Timeout`	retry (2 attempts here), then skip the category
404 Not Found	`HTTPError`	skip, do not retry
Body is not JSON	`ValueError` (`JSONDecodeError`)	skip category
JSON without `headlines`	`KeyError`	skip category
Empty list	own `ApiError`	skip category (nothing to count)

2. NLTK pipeline

For every headline: lower-case, `word_tokenize`, keep alphabetic tokens that are not in `stopwords.words('english')` (and are longer than two letters), then `WordNetLemmatizer().lemmatize`. This makes "startups" and "startup" count as one keyword.

3. Dictionaries and sets

- `counts = {category: Counter(...)}`. `Counter.most_common(5)` gives the top five.
- Keywords unique to a category: `set(counts[c]) - set().union(*other_counters)`. Keywords in every category are an intersection.
- Writing the file with mode `'w'` regenerates the report each run; `with` guarantees it is closed.

Watch out. `lemmatize()` assumes a noun unless you pass a POS tag, so the verb "discuss" becomes "discus" in the output below. Mention it in the viva: tag with `nltk.pos_tag` and map to WordNet POS for better lemmas.

Code: code/m1_q3_headlines.py

```
"""M1-Q3  Trending keywords from news headlines (REST API + NLTK).
The HTTP layer is a MockSession that returns genuine requests.Response objects,
so every error path below is the same code that would run against a live API.
Topics: REST/GET, status codes, JSON, exceptions, NLTK, dict/set/Counter, files.
"""
import json
import time
from collections import Counter

import nltk
import requests
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

for pkg, path in [("punkt_tab", "tokenizers/punkt_tab"), ("stopwords", "corpora/stopwords"),
                  ("wordnet", "corpora/wordnet")]:
    try:
        nltk.data.find(path)
    except LookupError:
        nltk.download(pkg, quiet=True)

BASE_URL = "https://api.example-news.test/headlines"      # placeholder host


# ---------------------------------------------------------------- mock API
def _response(status, body):
    r = requests.Response()
    r.status_code = status
    r._content = body.encode() if isinstance(body, str) else body
    r.url = BASE_URL
    return r


def _ok(category, headlines):
    return _response(200, json.dumps({"category": category, "headlines": headlines}))


class MockSession:
    """Scripted behaviour per category; a list means 'one outcome per call'."""

    def __init__(self):
        self.calls = {}
        self.script = {
            "tech": [_ok("tech", [
                "AI chips drive record profits for chipmakers",
                "New AI model beats humans at coding tests",
```

```
                "Cybersecurity firms warn of rising ransomware attacks",
                "Startups adopt AI assistants for customer support",
                "Record funding for AI startups in the cloud"])],
            "sports": [_ok("sports", [
                "Local team wins championship after dramatic final",
                "Star striker signs record transfer deal",
                "Coach praises young players after the final",
                "Champions celebrate record crowd at the stadium"])],
            "business": [_ok("business", [
                "Markets rally as record profits lift banks",
                "Startups raise funding despite rising interest rates",
                "Airline profits climb on record travel demand"])],
            "politics": [requests.exceptions.Timeout("read timed out"),     # 1st call
                         _ok("politics", ["Parliament debates new data protection law",
                                          "Leaders meet to discuss trade deal",
                                          "Voters react to new law on privacy"])],
            "health": [_ok("health", [])],                                  # empty list
            "science": [_response(200, "<html>Service busy</html>")],       # not JSON
            "travel": [_response(404, "not found")],
            "weather": [requests.exceptions.ConnectionError("network down")] * 3,
        }

    def get(self, url, params=None, timeout=None):
        cat = params["category"]
        n = self.calls.get(cat, 0)
        self.calls[cat] = n + 1
        scripted = self.script[cat]
        outcome = scripted[min(n, len(scripted) - 1)]
        if isinstance(outcome, Exception):
            raise outcome
        return outcome


# ---------------------------------------------------------------- fetching
class ApiError(Exception):
    pass


def fetch_headlines(category, session, retries=2):
    """GET /headlines?category=... ; returns list[str] or raises ApiError."""
    last_problem = None
    for attempt in range(1, retries + 1):
        try:
            resp = session.get(BASE_URL, params={"category": category}, timeout=5)
            resp.raise_for_status()                  # 4xx / 5xx -> HTTPError
            data = resp.json()                       # bad JSON -> ValueError
            headlines = data["headlines"]            # missing key -> KeyError
            if not isinstance(headlines, list) or not headlines:
                raise ApiError("empty headline list")
            return [h for h in headlines if isinstance(h, str) and h.strip()]
        except (requests.exceptions.Timeout, requests.exceptions.ConnectionError) as err:
            last_problem = f"network problem ({type(err).__name__})"      # worth retrying
            time.sleep(0)                            # real code: back off (e.g. 2**attempt)
        except requests.exceptions.HTTPError as err:
            raise ApiError(f"HTTP {err.response.status_code}") from err   # do not retry 404
        except ValueError as err:
            raise ApiError("invalid JSON") from err
        except KeyError as err:
            raise ApiError(f"missing field {err}") from err
    raise ApiError(f"{last_problem} after {retries} attempts")


# ---------------------------------------------------------------- NLTK
STOP = set(stopwords.words("english"))
LEMMA = WordNetLemmatizer()


def keywords(headline):
    tokens = word_tokenize(headline.lower())
    return [LEMMA.lemmatize(t) for t in tokens if t.isalpha() and t not in STOP and len(t) > 2]


# ---------------------------------------------------------------- workflow
session = MockSession()
counts = {}                        # category -> Counter
failures = {}
for cat in ["tech", "sports", "business", "politics", "health", "science", "travel", "weather"]:
    try:
        heads = fetch_headlines(cat, session)
    except ApiError as err:
        failures[cat] = str(err)
        print(f"[skip] {cat:<9}: {err}")
        continue
    c = Counter()
    for h in heads:
        c.update(keywords(h))
    counts[cat] = c
    print(f"[ok]   {cat:<9}: {len(heads)} headlines, {len(c)} distinct keywords")
```

```
print("\nExample: ", keywords("Startups adopt AI assistants for customer support"))

print("\nTop 5 keywords per category")
for cat, c in counts.items():
    print(f"  {cat:<9}", c.most_common(5))

print("\nKeywords unique to one category")
for cat, c in counts.items():
    others = set().union(*(set(o) for k, o in counts.items() if k != cat))
    unique = set(c) - others
    print(f"  {cat:<9} {len(unique)} unique, e.g. {sorted(unique)[:6]}")

with open("top_keywords.txt", "w", encoding="utf-8") as f:
    for cat, c in counts.items():
        f.write(f"{cat}: " + ", ".join(f"{w}({n})" for w, n in c.most_common(5)) + "\n")

print("\ntop_keywords.txt")
print(open("top_keywords.txt", encoding="utf-8").read(), end="")
print("\nFailed categories:", failures)
```

Output

```
[ok]   tech     : 5 headlines, 24 distinct keywords
[ok]   sports   : 4 headlines, 20 distinct keywords
[ok]   business : 3 headlines, 17 distinct keywords
[ok]   politics : 3 headlines, 14 distinct keywords
[skip] health   : empty headline list
[skip] science  : invalid JSON
[skip] travel   : HTTP 404
[skip] weather  : network problem (ConnectionError) after 2 attempts

Example:  ['startup', 'adopt', 'assistant', 'customer', 'support']

Top 5 keywords per category
  tech      [('record', 2), ('startup', 2), ('chip', 1), ('drive', 1), ('profit', 1)]
  sports    [('final', 2), ('record', 2), ('local', 1), ('team', 1), ('win', 1)]
  business  [('record', 2), ('profit', 2), ('market', 1), ('rally', 1), ('lift', 1)]
  politics  [('new', 2), ('law', 2), ('parliament', 1), ('debate', 1), ('data', 1)]

Keywords unique to one category
  tech      18 unique, e.g. ['adopt', 'assistant', 'attack', 'beat', 'chip', 'chipmakers']
  sports    18 unique, e.g. ['celebrate', 'champion', 'championship', 'coach', 'crowd', 'dramatic']
  business  12 unique, e.g. ['airline', 'bank', 'climb', 'demand', 'despite', 'interest']
  politics  12 unique, e.g. ['data', 'debate', 'discus', 'law', 'leader', 'meet']

top_keywords.txt
tech: record(2), startup(2), chip(1), drive(1), profit(1)
sports: final(2), record(2), local(1), team(1), win(1)
business: record(2), profit(2), market(1), rally(1), lift(1)
politics: new(2), law(2), parliament(1), debate(1), data(1)

Failed categories: {'health': 'empty headline list', 'science': 'invalid JSON', 'travel': 'HTTP 404', 'weather':
    'network problem (ConnectionError) after 2 attempts'}
```

Note. The API is simulated by a mock session that returns genuine `requests.Response` objects, so the error handling shown is the same code that runs against a live service. The host name is a placeholder.

Suggested marks split. workflow and API handling 3, NLTK pipeline 3, dict/set logic 2, exception handling 1, file output 1.

M1-C4. Hostel complaint chatbot [10 marks]

1. Class design

Class	Attributes	Methods / role
`Complaint`	`cid`, `text`, `category`, `priority`	`__str__`; one object per logged complaint
`Classifier`	category keyword sets, urgent tokens, lemmatizer	`tokens(text)`, `category(tokens)`, `is_urgent(tokens)`
`FaqBook`	list of (required keywords, answer)	`answer(tokens)` for mess timings, warden contact
`ComplaintLog`	file path, clock	`write(complaint)` appends one line
`HostelChatbot`	classifier, FAQ book, heap, counter, optional ChatterBot	`handle(message)`, `process_all()`

2. How the technologies combine

Component	Role	Why it helps
NLTK	lower-case, `word_tokenize`, lemmatize ("timings" -> "timing", "leaks" -> "leak")	keyword matching works on base forms, so spelling variants still match
Rules + dictionary	keyword sets decide category; "no" + water, or "power" + failure, marks urgency	predictable and explainable for complaints that must be routed correctly
Priority queue	`heapq` entries `(priority, arrival_no, complaint)`; urgent = 0, routine = 1	urgent complaints are served first; the counter keeps routine ones in arrival order
ChatterBot	`ListTrainer` on greeting pairs; reply used only when `confidence >= 0.6`	friendly small talk without letting it answer complaints; low confidence falls back to a clarification
Exceptions	`ValueError` for empty or non-text input; guarded optional import of ChatterBot	the bot never crashes on bad input and still works if ChatterBot is missing
File handling	`ComplaintLog` appends `time | id | category | priority | text` (mode `'a'`)	complete audit trail; the log can be re-read after a restart

Accuracy improves because lemmatization and tokenization remove surface differences; user experience improves because FAQs get an instant answer, unclear messages get a helpful prompt instead of a wrong guess, and urgent complaints are not stuck behind routine ones.

Code: code/m1_q4_chatbot.py

```
"""M1-Q4  Hostel complaint chatbot.
NLTK does the language work, a heapq priority queue orders complaints, a
dictionary answers FAQs, and ChatterBot (optional) handles small talk.
Topics: OOP, NLTK, priority queue, exceptions, file handling.
"""
import heapq
import itertools
from datetime import datetime

import nltk
from nltk.stem import WordNetLemmatizer
from nltk.tokenize import word_tokenize

for pkg, path in [("punkt_tab", "tokenizers/punkt_tab"), ("wordnet", "corpora/wordnet")]:
    try:
        nltk.data.find(path)
    except LookupError:
        nltk.download(pkg, quiet=True)

URGENT, ROUTINE = 0, 1            # smaller number = served first by heapq


class Complaint:
    def __init__(self, cid, text, category, priority):
        self.cid, self.text, self.category, self.priority = cid, text, category, priority

    def __str__(self):
        level = "URGENT " if self.priority == URGENT else "routine"
        return f"#{self.cid} [{level}] {self.category:<11} {self.text}"


class Classifier:
    CATEGORIES = {
        "water": {"water", "tap", "geyser", "bathroom", "leak", "drinking"},
        "wifi": {"wifi", "internet", "network", "router", "signal"},
        "electricity": {"electricity", "power", "light", "fan", "socket", "switch", "fuse"},
        "mess": {"mess", "food", "breakfast", "lunch", "dinner", "stale", "menu"},
    }
    URGENT_TOKENS = {"emergency", "urgent", "fire", "spark", "shock", "flood"}

    def __init__(self):
        self.lemma = WordNetLemmatizer()

    def tokens(self, text):
        text = text.lower().replace("wi-fi", "wifi")
        return [self.lemma.lemmatize(t) for t in word_tokenize(text) if t.isalnum()]

    def category(self, tokens):
        scores = {c: len(words & set(tokens)) for c, words in self.CATEGORIES.items()}
        best = max(scores, key=scores.get)
        return best if scores[best] > 0 else None

    def is_urgent(self, tokens):
        t = set(tokens)
        no_service = "no" in t and ("water" in t or "electricity" in t or "power" in t)
        power_loss = "power" in t and bool({"failure", "cut", "outage", "off"} & t)
        return bool(self.URGENT_TOKENS & t) or no_service or power_loss


class FaqBook:
```

```
    ENTRIES = [   # (keywords that must all appear, answer)  -- sample data, not real hostel data
        ({"mess", "timing"}, "Mess: breakfast 7:30-9:00, lunch 12:30-2:00, dinner 7:30-9:00."),
        ({"warden", "contact"}, "Warden office: ext. 2145 (9am-6pm). Emergencies: security desk, ext. 2100."),
        ({"warden", "number"}, "Warden office: ext. 2145 (9am-6pm). Emergencies: security desk, ext. 2100."),
    ]

    def answer(self, tokens):
        t = set(tokens)
        for needed, reply in self.ENTRIES:
            if needed <= t:
                return reply
        return None


class ComplaintLog:
    def __init__(self, path, clock=datetime.now):
        self.path, self.clock = path, clock

    def write(self, c):
        with open(self.path, "a", encoding="utf-8") as f:          # append, never overwrite
            f.write(f"{self.clock()} | #{c.cid} | {c.category} | "
                    f"{'URGENT' if c.priority == URGENT else 'routine'} | {c.text}\n")


class HostelChatbot:
    def __init__(self, log_path, small_talk=None):
        self.clf, self.faq = Classifier(), FaqBook()
        self.log = ComplaintLog(log_path, clock=lambda: "2026-07-06 21:00:00")
        self.heap, self.counter = [], itertools.count(1)            # counter breaks ties -> FIFO
        self.small_talk = small_talk                                # optional ChatterBot instance

    def handle(self, message):
        if not isinstance(message, str) or not message.strip():
            raise ValueError("empty or non-text message")
        tokens = self.clf.tokens(message)
        reply = self.faq.answer(tokens)
        if reply:
            return f"FAQ  -> {reply}"
        category = self.clf.category(tokens)
        if category is None:
            if self.small_talk is not None:
                answer = self.small_talk.get_response(message)
                if answer.confidence >= 0.6:            # trust ChatterBot only when it is sure
                    return f"BOT  -> {answer}"
            return "BOT  -> Sorry, I did not understand. Is it about water, Wi-Fi, electricity or the mess?"
        priority = URGENT if self.clf.is_urgent(tokens) else ROUTINE
        cid = next(self.counter)
        c = Complaint(cid, message.strip(), category, priority)
        heapq.heappush(self.heap, (priority, cid, c))               # (priority, arrival order)
        self.log.write(c)
        return f"LOGGED -> {c}"

    def process_all(self):
        while self.heap:
            _, _, c = heapq.heappop(self.heap)
            yield c


def build_chatterbot():
    """Optional: only used when the 'chatterbot' package is installed."""
    try:
        from chatterbot import ChatBot
        from chatterbot.trainers import ListTrainer
    except ImportError:
        return None
    bot = ChatBot("HostelBot",
                  storage_adapter="chatterbot.storage.SQLStorageAdapter",
                  database_uri="sqlite:///hostelbot.sqlite3",
                  read_only=True)                       # do not learn from live users
    ListTrainer(bot).train(["Hi", "Hello! How can I help you today?",
                            "Thanks", "You are welcome!"])
    return bot


if __name__ == "__main__":
    open("complaints.log", "w").close()                             # fresh log for the demo run
    bot = HostelChatbot("complaints.log", small_talk=build_chatterbot())
    messages = [
        "The mess food was stale at dinner",
        "There is no water in the bathroom of block B",
        "Wi-Fi is very slow in my room",
        "What are the mess timings?",
        "Hi",
        "Power failure in block C since evening",
        "asdf qwerty",
        "",
        "Who is the warden and what is the contact number?",
        "The fan in room 204 is not working",
    ]
```

```
    print("Incoming messages")
    for m in messages:
        try:
            print(f"  {m!r:<55} {bot.handle(m)}")
        except ValueError as err:
            print(f"  {m!r:<55} INVALID -> {err}")

    print("\nProcessing order from the priority queue")
    for c in bot.process_all():
        print(" ", c)

    print("\ncomplaints.log")
    print(open("complaints.log", encoding="utf-8").read(), end="")
```

Output

```
Incoming messages
  'The mess food was stale at dinner'                    LOGGED -> #1 [routine] mess        The mess food was stale
       at dinner
  'There is no water in the bathroom of block B'         LOGGED -> #2 [URGENT ] water       There is no water in
      the bathroom of block B
  'Wi-Fi is very slow in my room'                        LOGGED -> #3 [routine] wifi        Wi-Fi is very slow in
      my room
  'What are the mess timings?'                           FAQ  -> Mess: breakfast 7:30-9:00, lunch 12:30-2:00,
      dinner 7:30-9:00.
  'Hi'                                                   BOT  -> Hello! How can I help you today?
  'Power failure in block C since evening'               LOGGED -> #4 [URGENT ] electricity Power failure in block
      C since evening
  'asdf qwerty'                                          BOT  -> Sorry, I did not understand. Is it about water,
      Wi-Fi, electricity or the mess?
  ''                                                     INVALID -> empty or non-text message
  'Who is the warden and what is the contact number?'    FAQ  -> Warden office: ext. 2145 (9am-6pm). Emergencies:
      security desk, ext. 2100.
  'The fan in room 204 is not working'                   LOGGED -> #5 [routine] electricity The fan in room 204 is
      not working

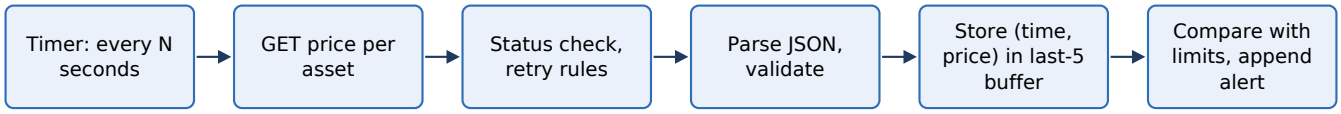
Processing order from the priority queue
  #2 [URGENT ] water       There is no water in the bathroom of block B
  #4 [URGENT ] electricity Power failure in block C since evening
  #1 [routine] mess        The mess food was stale at dinner
  #3 [routine] wifi        Wi-Fi is very slow in my room
  #5 [routine] electricity The fan in room 204 is not working

complaints.log
2026-07-06 21:00:00 | #1 | mess | routine | The mess food was stale at dinner
2026-07-06 21:00:00 | #2 | water | URGENT | There is no water in the bathroom of block B
2026-07-06 21:00:00 | #3 | wifi | routine | Wi-Fi is very slow in my room
2026-07-06 21:00:00 | #4 | electricity | URGENT | Power failure in block C since evening
2026-07-06 21:00:00 | #5 | electricity | routine | The fan in room 204 is not working
```

Note. Tested with ChatterBot 1.2.15. That version needs the spaCy model: `pip install chatterbot`, then `python -m spacy download en_core_web_sm`. FAQ answers and phone extensions in the code are made-up sample data.

Suggested marks split. class design 3, NLTK and rules 2, priority queue 2, ChatterBot 1, exceptions 1, file logging 1.

M1-C5. Asset price alert system [10 marks]



Workflow of the price alert application.

1. GET versus POST

Prices are read-only data, so the app uses **GET** with the asset in the query string (`?asset=BTC`). GET is safe and idempotent, so retrying cannot duplicate anything. POST would be used to create something, for example to register an alert subscription, and sends its data in the body; retrying a POST blindly can create duplicates.

2. Status codes and retry policy

Code	Meaning	Action in this app
200 OK	request succeeded	parse JSON
400 / 404	bad request / unknown asset	skip, do not retry (it will never succeed)
401 / 403	not authenticated / not allowed	stop and fix the API key

Code	Meaning	Action in this app
429	too many requests	wait for the `Retry-After` seconds, retry
500 / 503	server error / unavailable	retry a limited number of times, with back-off
Timeout (no code)	no answer in time	retry, then skip this cycle

3. JSON parsing

`resp.json()` converts the text into dict/list/float objects (JSON object -> dict, array -> list, number -> int or float, null -> None). Invalid text raises `ValueError`; a missing field raises `KeyError`; a non-numeric price raises `ValueError` or `TypeError` on `float(...)`. All three mean "skip this reading".

4. try / except / else / finally in `fetch_price`

- **try**: GET, `raise_for_status`, parse.
- **except**: separate handlers for `Timeout`, `HTTPError` (branch on the code) and `ValueError/KeyError/TypeError`.
- **else**: returns the price only when nothing failed, so success logic is not mixed with error handling.
- **finally**: closes the response and releases the connection on every path.

5. Data structures, conditionals, file modes

- `dict`: asset -> `deque(maxlen=5)`. A deque with `maxlen` drops the oldest reading automatically (a list with slicing `[-5:]` also works).
- `tuple (timestamp, price)`: an immutable reading.
- `if price > upper ... elif price < lower`: alert conditions. Limits are kept in a dict `asset -> (lower, upper)`.
- `open('alerts.log', 'a')`: append mode keeps earlier alerts. `'w'` would erase the history, and `'r'` is for reading it back.

Code: code/m1_q5_price_alerts.py

```
"""M1-Q5  Asset price alert system (REST polling + retries).
Uses a MockSession that returns real requests.Response objects so the error
handling is the same code that would run against a live API.
Topics: GET, status codes (200/404/429/5xx), JSON, try/except/else/finally,
list/tuple/dict, conditionals, file modes (a for the alert log).
"""
import io
import json
from collections import deque

import requests

BASE_URL = "https://api.example-prices.test/price"          # placeholder host
LIMITS = {"BTC": (60000, 70000), "GOLD": (1900, 2100), "ETH": (1500, 2500), "OIL": (70, 90)}


def _resp(status, body, headers=None):
    r = requests.Response()
    r.status_code = status
    r._content = body.encode()
    r.raw = io.BytesIO(b"")          # a real response owns a socket-backed 'raw' stream
    r.headers.update(headers or {})
    return r


def _price(asset, p):
    return _resp(200, json.dumps({"asset": asset, "price": p}))


class MockSession:
    """One scripted outcome per call, per asset."""

    def __init__(self):
        self.n = {}
        self.script = {
            "BTC": [_price("BTC", p) for p in (64000, 65500, 70500, 66000, 59000, 61000)],
            "GOLD": [requests.exceptions.Timeout("slow"), _price("GOLD", 2000),
                     _resp(429, "slow down", {"Retry-After": "1"}), _price("GOLD", 2050),
                     _price("GOLD", 2110), _price("GOLD", 2080), _price("GOLD", 2060)],
            "ETH": [_resp(404, "unknown asset")],
            "OIL": [_price("OIL", 80), _resp(200, "{oops"), _price("OIL", 91),
                    _price("OIL", 85), _price("OIL", 69), _price("OIL", 75), _price("OIL", 78)],
        }

    def get(self, url, params=None, timeout=None):
        a = params["asset"]
        i = self.n.get(a, 0)
        self.n[a] = i + 1
        out = self.script[a][min(i, len(self.script[a]) - 1)]
```

```
        if isinstance(out, Exception):
            raise out
        return out


def fetch_price(asset, session, max_tries=3):
    """Return a float, or None when the asset must be skipped this cycle."""
    for attempt in range(1, max_tries + 1):
        resp = None
        try:
            resp = session.get(BASE_URL, params={"asset": asset}, timeout=3)
            resp.raise_for_status()
            price = float(resp.json()["price"])
        except requests.exceptions.Timeout:
            print(f"    {asset}: timeout (try {attempt}/{max_tries}) -> retry")
        except requests.exceptions.HTTPError:
            code = resp.status_code
            if code == 429:
                wait = int(resp.headers.get("Retry-After", 1))
                print(f"    {asset}: 429 Too Many Requests -> wait {wait}s, retry")
            elif code >= 500:
                print(f"    {asset}: {code} server error -> retry")
            else:                                   # 404 etc. -> client error, retrying is useless
                print(f"    {asset}: HTTP {code} -> skip, no retry")
                return None
        except (ValueError, KeyError, TypeError):
            print(f"    {asset}: invalid JSON / missing price -> skip this cycle")
            return None
        else:
            return price                            # only runs when no exception happened
        finally:
            if resp is not None:                    # always release the connection
                resp.close()
    print(f"    {asset}: gave up after {max_tries} tries")
    return None


session = MockSession()
history = {a: deque(maxlen=5) for a in LIMITS}      # last 5 (timestamp, price) tuples per asset
alerts = []
clock = ["10:00", "10:01", "10:02", "10:03", "10:04", "10:05"]

for stamp in clock:                                 # a real app would loop with time.sleep(interval)
    print(f"Cycle {stamp}")
    for asset, (low, high) in LIMITS.items():
        price = fetch_price(asset, session)
        if price is None:
            continue
        history[asset].append((stamp, price))
        if price > high:
            alerts.append(f"{stamp} {asset} ABOVE upper limit {high}: {price}")
        elif price < low:
            alerts.append(f"{stamp} {asset} BELOW lower limit {low}: {price}")

with open("alerts.log", "a", encoding="utf-8") as log:      # 'a' keeps old alerts
    for line in alerts:
        log.write(line + "\n")

print("\nLast five readings per asset")
for a, h in history.items():
    print(f"  {a:<5}", list(h))
print("\nAlerts raised")
for line in alerts:
    print(" ", line)
```

Output

```
Cycle 10:00
    GOLD: timeout (try 1/3) -> retry
    ETH: HTTP 404 -> skip, no retry
Cycle 10:01
    GOLD: 429 Too Many Requests -> wait 1s, retry
    ETH: HTTP 404 -> skip, no retry
    OIL: invalid JSON / missing price -> skip this cycle
Cycle 10:02
    ETH: HTTP 404 -> skip, no retry
Cycle 10:03
    ETH: HTTP 404 -> skip, no retry
Cycle 10:04
    ETH: HTTP 404 -> skip, no retry
Cycle 10:05
    ETH: HTTP 404 -> skip, no retry

Last five readings per asset
  BTC   [('10:01', 65500.0), ('10:02', 70500.0), ('10:03', 66000.0), ('10:04', 59000.0), ('10:05', 61000.0)]
  GOLD  [('10:01', 2050.0), ('10:02', 2110.0), ('10:03', 2080.0), ('10:04', 2060.0), ('10:05', 2060.0)]
  ETH   []
  OIL   [('10:00', 80.0), ('10:02', 91.0), ('10:03', 85.0), ('10:04', 69.0), ('10:05', 75.0)]
```

```
Alerts raised
  10:02 BTC ABOVE upper limit 70000: 70500.0
  10:02 GOLD ABOVE upper limit 2100: 2110.0
  10:02 OIL ABOVE upper limit 90: 91.0
  10:04 BTC BELOW lower limit 60000: 59000.0
  10:04 OIL BELOW lower limit 70: 69.0
```

Note. Prices come from a mock session that returns real `requests.Response` objects (timeouts, 429 with `Retry-After`, 404 and invalid JSON are scripted). A real app would wrap the loop in `time.sleep(interval)`.

Suggested marks split. workflow 2, GET/POST and status codes 3, JSON and exception blocks 3, data structures and file modes 2.

M1-C6. Student record manager [10 marks]

1. Class design (inheritance)

Class	Attributes	Methods
`Person`	`name`, `email`	`__init__` (cleans and validates), `__str__`
`Student(Person)`	inherits `name`, `email`; adds `reg_no`, `marks`	`__init__` calls `super().__init__`, `grade()`, `to_dict()`

Validation lives in the constructors, so an invalid object can never exist (fail fast). `Student` reuses the name/email checks of `Person` through `super().__init__(...)`.

2. Strings, grading, errors

- `name.strip().title()` turns `" asha nair "` into `"Asha Nair"`.
- Email rule: `"@" in email and email.endswith(".ac.in")`. `rohan.vit.ac.in` has no `@`, so it is rejected.
- `float(marks)` raises `ValueError` for `"abc"`; the constructor re-raises it with a clear message.
- Grade ladder: `if m >= 85: 'A' elif m >= 70: 'B' elif m >= 50: 'C' else 'Fail'` (order matters: test the highest bound first).

3. File flow

1. `csv.DictReader` reads each row as a dict keyed by the header (open with `newline=''`).
2. `try: Student(...)`, `except ValueError as err:` records `(line, reg_no, reason)`; `else:` appends to the list and the dict.
3. `errors.log` is written with `'w'` (one fresh log per run); use `'a'` to accumulate across runs.
4. `json.dump(report, f, indent=2)` writes the report: counts plus a list of `to_dict()` results.

Code: code/m1_q6_students.py

```python
"""M1-Q6  Student record manager
Topics: inheritance, csv, string methods, conditionals, exceptions, json, dict.
"""
import csv
import json


class Person:
    def __init__(self, name, email):
        self.name = name.strip().title()
        self.email = email.strip().lower()
        if "@" not in self.email or not self.email.endswith(".ac.in"):
            raise ValueError(f"invalid email '{email.strip()}'")

    def __str__(self):
        return f"{self.name} <{self.email}>"


class Student(Person):
    def __init__(self, reg_no, name, email, marks):
        super().__init__(name, email)           # validates name / email
        self.reg_no = reg_no.strip()
        try:
            self.marks = float(marks)
        except ValueError:
            raise ValueError(f"non-numeric marks '{marks}'") from None
        if not 0 <= self.marks <= 100:
            raise ValueError(f"marks out of range '{marks}'")

    def grade(self):
        if self.marks >= 85:
            return "A"
        elif self.marks >= 70:
```

```
            return "B"
        elif self.marks >= 50:
            return "C"
        return "Fail"

    def to_dict(self):
        return {"reg_no": self.reg_no, "name": self.name, "email": self.email,
                "marks": self.marks, "grade": self.grade()}


students = []            # list of Student objects
by_reg = {}              # dict keyed by register number
errors = []              # (line number, reason)

with open("students.csv", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for line_no, row in enumerate(reader, start=2):      # line 1 is the header
        try:
            s = Student(row["reg_no"], row["name"], row["email"], row["marks"])
        except ValueError as err:
            errors.append((line_no, row["reg_no"], str(err)))
        else:
            students.append(s)
            by_reg[s.reg_no] = s

with open("errors.log", "w", encoding="utf-8") as log:
    for line_no, reg, reason in errors:
        log.write(f"line {line_no} ({reg}): skipped - {reason}\n")

report = {
    "valid_count": len(students),
    "skipped_count": len(errors),
    "students": [s.to_dict() for s in students],
}
with open("report.json", "w", encoding="utf-8") as f:
    json.dump(report, f, indent=2)

print("Valid students")
for s in students:
    print(f"  {s.reg_no}  {s.name:<12} {s.marks:>5}  grade {s.grade()}")
print("\nDictionary keys:", list(by_reg))
print("\nerrors.log")
print(open("errors.log", encoding="utf-8").read(), end="")
print("\nreport.json")
print(open("report.json", encoding="utf-8").read())
```

Output

```
Valid students
  VL2026001  Asha Nair     86.0  grade A
  VL2026004  Kabir Shah    58.0  grade C

Dictionary keys: ['VL2026001', 'VL2026004']

errors.log
line 3 (VL2026002): skipped - invalid email 'rohan.vit.ac.in'
line 4 (VL2026003): skipped - non-numeric marks 'abc'

report.json
{
  "valid_count": 2,
  "skipped_count": 2,
  "students": [
    {
      "reg_no": "VL2026001",
      "name": "Asha Nair",
      "email": "asha@vit.ac.in",
      "marks": 86.0,
      "grade": "A"
    },
    {
      "reg_no": "VL2026004",
      "name": "Kabir Shah",
      "email": "kabir@vit.ac.in",
      "marks": 58.0,
      "grade": "C"
    }
  ]
}
```

4. How the pieces work together

OOP packages the data and the rules (constructor validation, `grade()`), inheritance avoids repeating the person checks, string methods clean raw text before it is stored, exceptions separate bad rows from good ones without stopping the run, and file handling provides the input (CSV), the audit trail (log) and the deliverable (JSON). Each layer is small and testable.

Suggested marks split. class design 2, CSV and string handling 2, grading 1, exceptions and log 2, list and dict 1, JSON 1, explanation 1.

Module 2: Python libraries for data science

Syllabus (9 hours): Pandas: Series, DataFrame, missing values, built-in functions, data operations, filtering, extraction, text data, merging; data mining with Scrapy and Beautiful Soup; processing and modelling with NumPy, SciPy, Pandas, Keras, scikit-learn, PyTorch, TensorFlow, XGBoost; visualisation with Matplotlib, Seaborn, Bokeh, Plotly, Folium.

Labs 1 to 12 in your list of experiments belong to this module; the long questions below follow them. Questions marked FAT in the text come from the high-frequency list in your FAT notes.

Tip. Always end a pandas answer with a small output table. Examiners compare it with what the code should produce, and it also proves you know the shape of the result.

Module 2: Test yourself (questions only)

A. Multiple choice (10 questions, 1 mark each)

M2-A1. For a DataFrame with the default integer index, `df.loc[1:3]` and `df.iloc[1:3]` return respectively:

(a) 3 rows and 3 rows
(b) 3 rows and 2 rows
(c) 2 rows and 3 rows
(d) 2 rows and 2 rows

M2-A2. Which array can be broadcast against an array of shape (3, 4) in an addition?

(a) shape (3,)
(b) shape (4,)
(c) shape (2, 4)
(d) shape (3, 2)

M2-A3. `pd.merge(a, b, on='k', how='left')` returns:

(a) only keys present in both
(b) all rows of `a`, with NaN where `b` has no match
(c) all rows of `b`, with NaN where `a` has no match
(d) the Cartesian product

M2-A4. `df.groupby('city')['sales'].sum()` is an example of:

(a) map-reduce on disk
(b) split-apply-combine
(c) broadcasting
(d) one-hot encoding

M2-A5. Which call removes every row that has at least one missing value?

(a) `df.fillna(0)`
(b) `df.dropna()`
(c) `df.isna()`
(d) `df.drop_duplicates()`

M2-A6. Scrapy achieves high crawl throughput mainly because it is built on:

(a) multiprocessing only
(b) the Twisted asynchronous, event-driven engine
(c) Selenium
(d) a single blocking loop

M2-A7. Which library is used to draw interactive maps with markers (Leaflet-based)?

(a) Seaborn
(b) Bokeh
(c) Folium
(d) SciPy

M2-A8. Before running K-means or PCA on features measured in very different units you should:

(a) drop the target
(b) standardise the features
(c) convert them to strings
(d) sort the rows

M2-A9. XGBoost is best described as:

(a) a deep learning framework
(b) an optimised gradient-boosted decision tree library
(c) a plotting library
(d) a web scraper

M2-A10. `pd.concat([a, b], axis=1)` **does what?**

(a) stacks `b` below `a`
(b) places `b` beside `a`, aligning on the index
(c) merges on a key column
(d) removes duplicates

B. Short answers

M2-B1. Differentiate a pandas Series from a DataFrame, and show how to create each. [3 marks]

M2-B2. Describe four ways of handling missing values in pandas and when each is suitable. [5 marks]

M2-B3. Distinguish `.loc`, `.iloc` and boolean filtering with examples. [3 marks]

M2-B4. Compare `merge`, `join` and `concat` in pandas. [5 marks]

M2-B5. Why is NumPy faster than Python lists for numerical work? [3 marks]

M2-B6. How do you work with text data in a DataFrame? Give four `.str` operations. [3 marks]

M2-B7. Compare Matplotlib and Seaborn. [3 marks]

M2-B8. State the role of SciPy, scikit-learn, Keras, TensorFlow, PyTorch and XGBoost in one line each. [5 marks]

M2-B9. What is `robots.txt` and what are the rules of responsible scraping? [3 marks]

C. Long questions

M2-C1. Data manipulation and cleaning with pandas [10 marks]

(Lab experiments 1 and 2) **Part A.** Create a DataFrame with columns Name, Age and City for five individuals. Select only Name and Age; filter rows where Age is greater than 25; add a column Country with a default value; sort by Age in descending order; calculate the average age.

Part B. Create a DataFrame with missing values in Name, Age and City. Fill missing Age values with the mean age, drop rows where Name or City is missing, convert Age to integer, and normalise Age with min-max scaling. Explain each step.

M2-C2. Split-apply-combine and merging DataFrames [10 marks]

(FAT high-frequency question 1) Explain the split-apply-combine strategy in pandas. Write code that uses `groupby` with multiple aggregation functions. Also explain the different kinds of merge on two related tables (customers and orders).

M2-C3. NumPy arrays and broadcasting [10 marks]

(Lab experiment 3, FAT high-frequency question 8) Using NumPy create a 2-D array of shape (3, 4) with random integers between 0 and 10. Calculate the mean and standard deviation, slice the first two rows and last two columns, reshape to (4, 3) and multiply element-wise with another array of the same shape. State the broadcasting rules and show one example that fails.

M2-C4. Beautiful Soup versus Scrapy [10 marks]

(FAT high-frequency question 2, Lab experiment 5) Compare Beautiful Soup and Scrapy. Explain why Scrapy's Twisted event loop gives higher throughput. Scrape a page and store the result as CSV or JSON.

M2-C5. Python visualisation libraries [10 marks]

(Lab experiments 6 to 10) Compare Matplotlib, Seaborn, Bokeh, Plotly and Folium and say when you would use each. Write code for: an interactive Plotly bar chart of city populations; a Plotly scatter plot of house size against price; a Bokeh line plot of a week's temperatures; a Bokeh bar chart of product sales; a Bokeh scatter plot of Iris petal length against petal width.

M2-C6. EDA, K-means, PCA and the modelling libraries [10 marks]

(Lab experiments 4, 11 and 12) For a table with Feature1, Feature2 and Target: create the log of Feature1, bin Feature2 into low, medium and high, and compute the correlation matrix. Then perform K-means (k = 3) on the Iris data and visualise the clusters, and perform PCA to two components. Explain the steps and interpret the results. State the role of the main modelling libraries.

Module 2: Answers

A. Multiple-choice key

No.	Ans.	Why
M2-A1	b	`loc` slices by label and includes the end label (rows 1, 2, 3). `iloc` slices by position and excludes the end (rows 1, 2).
M2-A2	b	Compare trailing dimensions: (4,) matches 4. (3,) would be compared with 4 and fails; (2,4) fails on 2 vs 3; (3,2) fails on 2 vs 4.
M2-A3	b	A left join keeps every left row. Use `inner` for matches only and `outer` for the union of keys.
M2-A4	b	Split the rows by city, apply `sum` to each group, combine the results into one Series.
M2-A5	b	`dropna()` drops rows (axis=0) containing NaN; `subset=[...]` restricts the check to some columns.
M2-A6	b	Non-blocking I/O lets one process keep many requests in flight while others wait for the network.
M2-A7	c	Folium wraps Leaflet.js: `folium.Map(...)`, `folium.Marker(...)`, saved as HTML.
M2-A8	b	Both methods use distances or variance, so large-scale features dominate unless scaled (`StandardScaler`).
M2-A9	b	It builds trees sequentially, each correcting the previous errors; very strong on tabular data.
M2-A10	b	axis=0 stacks rows (union of rows), axis=1 joins columns side by side.

B. Short answers

M2-B1. Differentiate a pandas Series from a DataFrame, and show how to create each. [3 marks]

A **Series** is a one-dimensional labelled array (values + index) with a single dtype. A **DataFrame** is a two-dimensional table of aligned Series sharing one index; each column can have its own dtype.

```
import pandas as pd
s = pd.Series([22, 31, 27], index=['a', 'b', 'c'], name='Age')
df = pd.DataFrame({'Name': ['Asha', 'Ravi', 'Meena'], 'Age': [22, 31, 27]})
type(df['Age'])    # Series: one column of a DataFrame
```

M2-B2. Describe four ways of handling missing values in pandas and when each is suitable. [5 marks]

Method	Code	Use when
Detect	`df.isna().sum()`	always first: count NaN per column
Drop rows	`df.dropna(subset=['Name','City'])`	key columns are missing and cannot be guessed; few rows lost
Fill with a statistic	`df['Age'].fillna(df['Age'].mean())`	numeric column, roughly symmetric (use median if skewed)
Forward / back fill, interpolate	`ffill()`, `bfill()`, `interpolate()`	ordered data such as time series
Fill with a constant or category	`fillna('Unknown')`	categorical column where "missing" is informative

M2-B3. Distinguish `.loc`, `.iloc` and boolean filtering with examples. [3 marks]

- `df.loc[1:2, 'Name']`: label based, end label included.
- `df.iloc[1:2, 0]`: position based, end excluded.
- `df[df['Age'] > 25]`: boolean mask selects rows that satisfy a condition; combine with `&`, `|` and parentheses: `df[(df.Age > 25) & (df.City == 'Pune')]`.

M2-B4. Compare `merge`, `join` and `concat` in pandas. [5 marks]

	merge	join	concat
Matches on	column values (`on=`, `left_on=`, `right_on=`)	the index of the right frame (by default)	the index / position, no key logic
Types	`inner`, `left`, `right`, `outer`	same `how` options	stacks rows (axis=0) or columns (axis=1)
Typical use	SQL-style join of two tables on a key	shortcut for index-based joins	append monthly files, glue columns
Example	`pd.merge(c, o, on='cust_id', how='left')`	`c.join(o.set_index('cust_id'), on='cust_id')`	`pd.concat([a, b], axis=0)`

M2-B5. Why is NumPy faster than Python lists for numerical work? [3 marks]

- An `ndarray` stores one fixed dtype in a contiguous block of memory; a list stores pointers to separate Python objects.
- Operations are **vectorised**: the loop runs in compiled C instead of the Python interpreter, with no per-element type check.
- Broadcasting and slicing create views without copying, saving time and memory.
- Measured here (Python 3.10, one million integers): `a * 2` took 2.5 ms, the list comprehension `[x * 2 for x in l]` took 96 ms, about 38 times slower. Your ratio will differ by machine.

M2-B6. How do you work with text data in a DataFrame? Give four `.str` operations. [3 marks]

```
df['name'] = df['name'].str.strip().str.title()
df[df['name'].str.contains('sh', case=False)]
df['first'] = df['name'].str.split().str[0]
df['email'].str.endswith('.ac.in')
df['name'].str.replace(r'\s+', ' ', regex=True)
```

The `.str` accessor applies string methods to a whole column and propagates NaN instead of raising.

M2-B7. Compare Matplotlib and Seaborn. [3 marks]

Matplotlib is the low-level base library: full control of figures, axes and styles, static output. Seaborn is built on Matplotlib and works directly with DataFrames; it offers statistical plots (`histplot`, `boxplot`, `heatmap`, `pairplot`) and attractive defaults with one line of code. Use Seaborn for quick exploration, Matplotlib to customise details.

M2-B8. State the role of SciPy, scikit-learn, Keras, TensorFlow, PyTorch and XGBoost in one line each. [5 marks]

Library	Role
SciPy	scientific algorithms on arrays: optimisation, statistics, linear algebra, signal processing, sparse matrices
scikit-learn	classical machine learning with a uniform `fit / predict / transform` API: regression, classification, clustering, PCA, pipelines
TensorFlow	Google's deep learning framework with tools for deployment (serving, mobile, browser)
Keras	high-level neural-network API (`Sequential`, `Model`) that runs on TensorFlow and other backends
PyTorch	deep learning with dynamic (define-by-run) graphs; popular for research and prototyping
XGBoost	gradient-boosted decision trees, often the strongest model for tabular data

M2-B9. What is `robots.txt` and what are the rules of responsible scraping? [3 marks]

- `robots.txt` at the site root tells crawlers which paths they may fetch. Scrapy obeys it with `ROBOTSTXT_OBEY = True`.
- Throttle requests (`DOWNLOAD_DELAY`, `AUTOTHROTTLE_ENABLED`), identify your crawler with a User-Agent, respect the site's terms of service and do not collect personal data without a lawful basis.
- Prefer an official API when one exists.

C. Long answers

M2-C1. Data manipulation and cleaning with pandas [10 marks]

Concepts

- A DataFrame is a labelled 2-D table. `df[['Name','Age']]` (double brackets) returns a DataFrame with those columns; a boolean mask `df[df.Age > 25]` filters rows; assigning a scalar to a new column broadcasts it to every row.
- `sort_values('Age', ascending=False)` returns a new sorted frame (original unchanged unless `inplace=True`).
- Missing-value pipeline: `isna().sum()` to inspect, `fillna(mean)` for numeric gaps, `dropna(subset=[...])` for rows missing key fields, `astype(int)` after the NaNs are gone (an int column cannot hold NaN).
- Min-max scaling maps values to [0, 1]: **x' = (x - min) / (max - min)**.

Watch out. The mean in the output below (32.0) is computed *before* the incomplete rows are dropped, as the question's order requires. If you drop first, the mean changes. State the order you used.

Code: code/m2_q1_pandas_basic.py

```python
"""M2-Q1  Pandas basics, cleaning and scaling (Lab experiments 1 and 2)."""
import numpy as np
import pandas as pd

# ---------------------------------------------------------------- Lab 1
df = pd.DataFrame({
    "Name": ["Asha", "Ravi", "Meena", "Kabir", "Sneha"],
    "Age": [22, 31, 27, 35, 24],
    "City": ["Chennai", "Delhi", "Pune", "Mumbai", "Chennai"],
})
print("1. select Name and Age\n", df[["Name", "Age"]], sep="")
print("\n2. filter Age > 25\n", df[df["Age"] > 25], sep="")
df["Country"] = "India"
print("\n3. add Country, sort by Age (descending)\n", df.sort_values("Age", ascending=False), sep="")
print("\n4. average age:", df["Age"].mean())
print("\n.loc (label) vs .iloc (position)")
print("   df.loc[1:2, 'Name'] ->", df.loc[1:2, "Name"].tolist(), "   (end label INCLUDED)")
print("   df.iloc[1:2, 0]     ->", df.iloc[1:2, 0].tolist(), "           (end position EXCLUDED)")

# ---------------------------------------------------------------- Lab 2
raw = pd.DataFrame({
    "Name": ["Asha", None, "Meena", "Kabir", "Sneha"],
    "Age": [22, 31, np.nan, 35, 40],
    "City": ["Chennai", "Delhi", "Pune", None, "Chennai"],
})
print("\n\nRaw data with missing values\n", raw, sep="")
print("\nmissing per column\n", raw.isna().sum(), sep="")

clean = raw.copy()
clean["Age"] = clean["Age"].fillna(clean["Age"].mean())         # mean of 22, 31, 35, 40 = 32.0
clean = clean.dropna(subset=["Name", "City"])                    # drop rows missing Name or City
clean["Age"] = clean["Age"].round().astype(int)                  # float -> int
print("\nAfter fillna / dropna / astype(int)\n", clean, sep="")

a = clean["Age"]
clean["Age_scaled"] = (a - a.min()) / (a.max() - a.min())        # min-max scaling to [0, 1]
print("\nMin-max scaled Age\n", clean, sep="")
```

Output

```
1. select Name and Age
    Name  Age
0   Asha   22
1   Ravi   31
2  Meena   27
3  Kabir   35
4  Sneha   24

2. filter Age > 25
    Name  Age    City
1   Ravi   31   Delhi
2  Meena   27    Pune
3  Kabir   35  Mumbai

3. add Country, sort by Age (descending)
    Name  Age     City Country
3  Kabir   35   Mumbai   India
1   Ravi   31    Delhi   India
2  Meena   27     Pune   India
4  Sneha   24  Chennai   India
0   Asha   22  Chennai   India

4. average age: 27.8

.loc (label) vs .iloc (position)
   df.loc[1:2, 'Name'] -> ['Ravi', 'Meena']    (end label INCLUDED)
   df.iloc[1:2, 0]     -> ['Ravi']            (end position EXCLUDED)


Raw data with missing values
    Name   Age     City
0   Asha  22.0  Chennai
1   None  31.0    Delhi
2  Meena   NaN     Pune
3  Kabir  35.0     None
4  Sneha  40.0  Chennai

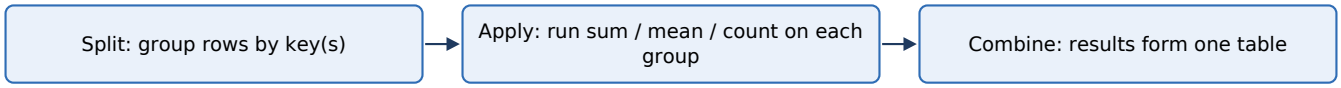
missing per column
Name    1
Age     1
City    1
dtype: int64

After fillna / dropna / astype(int)
    Name  Age     City
0   Asha   22  Chennai
2  Meena   32     Pune
4  Sneha   40  Chennai
```

```
Min-max scaled Age
    Name  Age     City  Age_scaled
0   Asha   22  Chennai    0.000000
2  Meena   32     Pune    0.555556
4  Sneha   40  Chennai    1.000000
```

Suggested marks split. concepts 3, correct operations 4, missing-value handling and scaling 2, output 1.

M2-C2. Split-apply-combine and merging DataFrames [10 marks]



Split-apply-combine.

Concept

`groupby` **splits** a DataFrame into groups that share the same key, **applies** a function to every group independently, and **combines** the per-group results into a new Series or DataFrame. It replaces explicit loops and is the pandas equivalent of SQL `GROUP BY`.

- `agg` reduces each group to one row per aggregation (named aggregation `total=('sales','sum')` gives readable column names).
- `transform` returns a result the same length as the input (e.g. each row's share of its group total).
- `apply` runs any function on each group and is the most flexible but slowest.
- `reset_index()` turns the group keys back into ordinary columns.

Merge types

how=	Rows kept	Rows in the example output
`inner`	keys present in both tables	4 (customer 4 has no orders; order 105's customer 9 does not exist)
`left`	all customers, NaN where no order	5 (customer 4 appears with NaN)
`right`	all orders, NaN where no customer	5
`outer`	union of keys	6

`concat` does not match on keys: `axis=0` stacks rows, `axis=1` places frames side by side by index alignment.

Code: code/m2_q2_pandas_merge.py

```python
"""M2-Q2  Merging, split-apply-combine, text data (FAT top-10 question 1)."""
import pandas as pd

customers = pd.DataFrame({
    "cust_id": [1, 2, 3, 4],
    "name": ["  asha NAIR", "ravi kumar ", "Meena Rao", "kabir shah"],
    "region": ["South", "North", "West", "South"],
})
orders = pd.DataFrame({
    "order_id": [101, 102, 103, 104, 105],
    "cust_id": [1, 1, 2, 3, 9],                 # 9 has no matching customer; 4 has no orders
    "category": ["Books", "Tech", "Tech", "Books", "Tech"],
    "sales": [500, 1200, 800, 300, 400],
    "profit": [50, 240, 120, 30, 40],
})

print("inner merge (only matching keys)")
print(pd.merge(customers, orders, on="cust_id", how="inner")[["cust_id", "order_id", "sales"]])
print("\nleft merge (all customers, NaN where no orders)")
print(pd.merge(customers, orders, on="cust_id", how="left")[["cust_id", "order_id", "sales"]])
print("\nouter merge row count:", len(pd.merge(customers, orders, on="cust_id", how="outer")))

print("\nconcat axis=0 (stack rows) shape:",
      pd.concat([orders.iloc[:2], orders.iloc[2:]], axis=0).shape,
      "| concat axis=1 (side by side) shape:",
      pd.concat([orders[["order_id"]], orders[["sales"]]], axis=1).shape)

# split -> apply -> combine
merged = pd.merge(customers, orders, on="cust_id", how="inner")
summary = (merged.groupby(["region", "category"])
                 .agg(total_sales=("sales", "sum"),
                      avg_profit=("profit", "mean"),
                      orders=("order_id", "count"))
                 .reset_index())
print("\ngroupby(region, category).agg(...)\n", summary, sep="")
```

```
# working with text data via the .str accessor
customers["name"] = customers["name"].str.strip().str.title()
print("\nCleaned names:", customers["name"].tolist())
print("Names containing 'sh' (case-insensitive):",
      customers.loc[customers["name"].str.contains("sh", case=False), "name"].tolist())
print("First names:", customers["name"].str.split().str[0].tolist())
```

Output

```
inner merge (only matching keys)
   cust_id  order_id  sales
0        1       101    500
1        1       102   1200
2        2       103    800
3        3       104    300

left merge (all customers, NaN where no orders)
   cust_id  order_id   sales
0        1     101.0   500.0
1        1     102.0  1200.0
2        2     103.0   800.0
3        3     104.0   300.0
4        4       NaN     NaN

outer merge row count: 6

concat axis=0 (stack rows) shape: (5, 5) | concat axis=1 (side by side) shape: (5, 2)

groupby(region, category).agg(...)
  region category  total_sales  avg_profit  orders
0  North     Tech          800       120.0       1
1  South    Books          500        50.0       1
2  South     Tech         1200       240.0       1
3   West    Books          300        30.0       1

Cleaned names: ['Asha Nair', 'Ravi Kumar', 'Meena Rao', 'Kabir Shah']
Names containing 'sh' (case-insensitive): ['Asha Nair', 'Kabir Shah']
First names: ['Asha', 'Ravi', 'Meena', 'Kabir']
```

Suggested marks split. definition and diagram 2, groupby/agg code 3, merge types 3, concat vs merge 2.

M2-C3. NumPy arrays and broadcasting [10 marks]

Broadcasting rules

1. Write the two shapes aligned on the **right**; a shorter shape is padded with 1s on the left.
2. For each aligned pair of dimensions the sizes must be **equal, or one of them must be 1**.
3. A dimension of size 1 is virtually stretched to match the other (no data is copied).
4. If any pair fails, NumPy raises `ValueError: operands could not be broadcast together`.

Shapes	Aligned	Result
(3,4) + (4,)	(3,4) vs (1,4)	(3,4) valid: the row is added to every row
(3,4) + (3,1)	(3,4) vs (3,1)	(3,4) valid: the column is added to every column
(3,1) + (1,4)	(3,1) vs (1,4)	(3,4) valid: outer-sum pattern
(3,4) + (3,)	(3,4) vs (1,3)	error: 4 vs 3 differ and neither is 1

Why it matters: broadcasting removes Python loops and extra copies; e.g. column standardisation is `(x - x.mean(axis=0)) / x.std(axis=0)`. `axis=0` runs down the rows (one result per column), `axis=1` runs across the columns (one result per row).

Code: code/m2_q3_numpy.py

```
"""M2-Q3  NumPy arrays and broadcasting (Lab experiment 3, FAT top-10 question 8)."""
import numpy as np

rng = np.random.default_rng(seed=7)                # seed -> repeatable output
a = rng.integers(0, 11, size=(3, 4))               # integers 0..10
print("a =\n", a, sep="")
print("shape", a.shape, "| mean", round(a.mean(), 3), "| std", round(a.std(), 3))

print("\nfirst two rows, last two columns:\n", a[:2, -2:], sep="")
r = a.reshape(4, 3)
print("\nreshape(4, 3):\n", r, sep="")
b = rng.integers(1, 5, size=(4, 3))
print("\nelement-wise product with another (4,3) array:\n", r * b, sep="")

print("\n--- broadcasting ---")
m = np.arange(12).reshape(3, 4)                    # (3, 4)
row = np.array([10, 20, 30, 40])                   # (4,)   -> treated as (1, 4)
col = np.array([[1], [2], [3]])                    # (3, 1)
```

```
print("(3,4) + (4,)  ->", (m + row).shape)
print("(3,4) + (3,1) ->", (m + col).shape)
print("(3,1) + (1,4) ->", (col + np.array([[1, 2, 3, 4]])).shape)
try:
    m + np.array([1, 2, 3])                        # (3,4) vs (3,) -> trailing dims 4 and 3 differ
except ValueError as err:
    print("(3,4) + (3,)  -> ValueError:", err)

x = np.array([[1.0, 2.0], [3.0, 4.0], [5.0, 6.0]])
print("\ncolumn-wise standardisation using broadcasting:\n",
      np.round((x - x.mean(axis=0)) / x.std(axis=0), 3), sep="")
```

Output

```
a =
[[10  6  7  9]
 [ 6  8  9  2]
 [ 0  3  3  9]]
shape (3, 4) | mean 6.0 | std 3.136

first two rows, last two columns:
[[7 9]
 [9 2]]

reshape(4, 3):
[[10  6  7]
 [ 9  6  8]
 [ 9  2  0]
 [ 3  3  9]]

element-wise product with another (4,3) array:
[[40  6 14]
 [36  6 32]
 [ 9  4  0]
 [ 6  6 18]]

--- broadcasting ---
(3,4) + (4,)  -> (3, 4)
(3,4) + (3,1) -> (3, 4)
(3,1) + (1,4) -> (3, 4)
(3,4) + (3,)  -> ValueError: operands could not be broadcast together with shapes (3,4) (3,)

column-wise standardisation using broadcasting:
[[-1.225 -1.225]
 [ 0.     0.   ]
 [ 1.225  1.225]]
```

Note. The random numbers depend on the seed and NumPy version, so yours may differ. The shapes and the error message are what to remember.

Suggested marks split. array operations 4, broadcasting rules 3, failing example 2, vectorisation benefit 1.

M2-C4. Beautiful Soup versus Scrapy [10 marks]

Feature	Beautiful Soup (with requests)	Scrapy
What it is	HTML/XML parsing library; you fetch pages yourself	complete crawling framework
Execution model	synchronous: one request at a time unless you add threads	asynchronous, event-driven (Twisted)
Concurrency	single-threaded by default	many concurrent non-blocking requests (`CONCURRENT_REQUESTS`)
Pipelines / export	write your own code	Item Pipelines, Feed Exporters (JSON, CSV, XML)
Politeness	manual `time.sleep()`	`DOWNLOAD_DELAY`, AutoThrottle, `ROBOTSTXT_OBEY`
Best for	small, one-off pages and quick scripts	large, repeatable, multi-page crawls

Why the Twisted event loop is faster

A blocking scraper spends most of its time waiting for the network, with the CPU idle. Twisted runs an event loop: when a request is sent, control returns immediately and the loop services other requests; a callback fires when the response arrives. One thread therefore keeps dozens of downloads in flight, so total time is close to the slowest request rather than the sum of all of them.

Scrapy components

Spider (parsing rules) -> Engine -> Scheduler (queue of requests) -> Downloader (fetch, via middlewares) -> Spider callback yields items -> Item Pipeline (clean, validate, store).

Code: code/m2_q4_scrape_bs4.py

```
"""M2-Q4a  Scrape a page with Beautiful Soup and store CSV + JSON (Lab experiment 5).
The HTML is embedded so the script runs offline; with a live site you would
replace `html` by requests.get(url, timeout=10).text.
"""
import csv
import json

from bs4 import BeautifulSoup

html = """
<html><body>
  <div class="book"><h2>Python Basics</h2><span class="price">Rs. 450</span><a href="/b/1">details</a></div>
  <div class="book"><h2>Data Science 101</h2><span class="price">Rs. 700</span><a href="/b/2">details</a></div>
  <div class="book"><h2>Scala in Depth</h2><span class="price">Rs. 820</span><a href="/b/3">details</a></div>
</body></html>
"""
with open("books.html", "w", encoding="utf-8") as f:       # reused by the Scrapy example
    f.write(html)

soup = BeautifulSoup(html, "html.parser")
rows = []
for div in soup.find_all("div", class_="book"):
    rows.append({
        "title": div.h2.get_text(strip=True),
        "price": int(div.select_one("span.price").get_text().replace("Rs.", "").strip()),
        "link": div.a["href"],
    })

with open("books_bs4.csv", "w", newline="", encoding="utf-8") as f:
    w = csv.DictWriter(f, fieldnames=["title", "price", "link"])
    w.writeheader()
    w.writerows(rows)
with open("books_bs4.json", "w", encoding="utf-8") as f:
    json.dump(rows, f, indent=2)

print("Parsed", len(rows), "books")
print(open("books_bs4.csv", encoding="utf-8").read())
print("Average price:", sum(r["price"] for r in rows) / len(rows))
```

Output

```
Parsed 3 books
title,price,link
Python Basics,450,/b/1
Data Science 101,700,/b/2
Scala in Depth,820,/b/3

Average price: 656.6666666666666
```

Code: code/m2_q4_scrape_scrapy.py

```
"""M2-Q4b  The same extraction as a Scrapy spider (runs against the local books.html
through a file:// URL so no website is contacted).
"""
import json
from pathlib import Path

import scrapy
from scrapy.crawler import CrawlerProcess

page = Path("books.html")
if not page.exists():
    raise SystemExit("run m2_q4_scrape_bs4.py first - it writes books.html")


class BookSpider(scrapy.Spider):
    name = "books"
    start_urls = [page.resolve().as_uri()]                # file:///.../books.html
    custom_settings = {"LOG_LEVEL": "ERROR", "ROBOTSTXT_OBEY": True,
                       "DOWNLOAD_DELAY": 1,               # politeness delay (ignored for file:///)
                       "AUTOTHROTTLE_ENABLED": True,
                       "FEEDS": {"books_scrapy.json": {"format": "json", "overwrite": True}}}

    def parse(self, response):
        for div in response.css("div.book"):
            yield {                                       # each yielded dict becomes an item
                "title": div.css("h2::text").get(),
                "price": int(div.css("span.price::text").get().replace("Rs.", "").strip()),
                "link": div.css("a::attr(href)").get(),
            }


process = CrawlerProcess()
process.crawl(BookSpider)
process.start()                                           # blocks until the crawl finishes

items = json.loads(Path("books_scrapy.json").read_text(encoding="utf-8"))
print("Scrapy yielded", len(items), "items")
```

```
for it in items:
    print(" ", it)
```

Output

```
Scrapy yielded 3 items
  {'title': 'Python Basics', 'price': 450, 'link': '/b/1'}
  {'title': 'Data Science 101', 'price': 700, 'link': '/b/2'}
  {'title': 'Scala in Depth', 'price': 820, 'link': '/b/3'}
```

Note. Both scripts read a local HTML page (the Scrapy one through a `file://` URL), so no website is contacted. For a real site, use its URL and respect `robots.txt`.

Suggested marks split. comparison table 4, event-loop explanation 3, working code 2, ethics 1.

M2-C5. Python visualisation libraries [10 marks]

Library	Output	Strength	Typical call
Matplotlib	static (PNG, PDF, SVG)	full control; base for others	`plt.plot(x, y)`
Seaborn	static	statistical plots on DataFrames, good defaults	`sns.histplot(df, x='age')`
Bokeh	interactive HTML (BokehJS), servers	linked brushing, widgets, streaming	`figure(); p.line(...); HoverTool`
Plotly	interactive HTML (Plotly.js), Dash apps	quickest interactive charts with `plotly.express`	`px.bar(df, x=..., y=...)`
Folium	interactive Leaflet map in HTML	markers, popups, choropleths on real maps	`folium.Map(location=[lat, lon])`

Choice guide: exploring a DataFrame -> Seaborn; report figure -> Matplotlib; interactive dashboard -> Plotly (Dash) or Bokeh server; geographic data -> Folium.

Code: code/m2_q5_viz.py

```
"""M2-Q5  Interactive visualisation: Plotly, Bokeh, Folium (Lab experiments 6-10).
Each chart is written to an HTML file; open the files in a browser to interact.
"""
import numpy as np
import pandas as pd
import plotly.express as px
from bokeh.models import ColumnDataSource, HoverTool
from bokeh.plotting import figure, output_file, save
from sklearn.datasets import load_iris
import folium

# ---- Lab 6: Plotly interactive bar chart ----------------------------------
cities = pd.DataFrame({"city": ["Mumbai", "Delhi", "Bengaluru", "Chennai", "Hyderabad"],
                       "population_m": [12.4, 11.0, 8.4, 7.1, 6.8]})        # illustrative values
fig_bar = px.bar(cities, x="city", y="population_m", title="Population of cities (millions)",
                 hover_data={"population_m": ":.1f"})
fig_bar.write_html("plotly_bar.html", include_plotlyjs="cdn")

# ---- Lab 7: Plotly scatter, house size vs price ---------------------------
rng = np.random.default_rng(1)
size = rng.integers(500, 3000, 40)
house = pd.DataFrame({"size_sqft": size, "price_lakh": size * 0.03 + rng.normal(0, 8, 40)})
fig_sc = px.scatter(house, x="size_sqft", y="price_lakh", trendline="ols",   # 'ols' needs statsmodels
                    title="House size vs price")
fig_sc.write_html("plotly_scatter.html", include_plotlyjs="cdn")

# ---- Lab 8: Bokeh line plot, daily temperature ----------------------------
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
temps = [31, 33, 32, 35, 34, 30, 29]
src = ColumnDataSource(dict(day=days, temp=temps))
p1 = figure(x_range=days, title="Daily temperature", height=300, width=500,
            tools="pan,wheel_zoom,reset,save")
p1.line("day", "temp", source=src, line_width=2)
p1.scatter("day", "temp", source=src, size=8)
p1.add_tools(HoverTool(tooltips=[("day", "@day"), ("temp", "@temp °C")]))
output_file("bokeh_line.html")
save(p1)

# ---- Lab 9: Bokeh bar chart, product sales --------------------------------
products = ["Laptop", "Phone", "Tablet", "Watch"]
sales = [120, 300, 90, 150]
p2 = figure(x_range=products, title="Sales by product", height=300, width=500)
p2.vbar(x=products, top=sales, width=0.6)
output_file("bokeh_bar.html")
save(p2)

# ---- Lab 10: Bokeh scatter, Iris petal length vs width --------------------
```

```
iris = load_iris(as_frame=True).frame
p3 = figure(title="Iris: petal length vs width", height=300, width=500)
p3.scatter(iris["petal length (cm)"], iris["petal width (cm)"], size=6, alpha=0.6)
output_file("bokeh_scatter.html")
save(p3)

# ---- Folium: map with a marker ------------------------------------------------
m = folium.Map(location=[12.9692, 79.1559], zoom_start=12)                # Vellore
folium.Marker([12.9692, 79.1559], popup="VIT Vellore").add_to(m)
m.save("folium_map.html")

for name in ["plotly_bar", "plotly_scatter", "bokeh_line", "bokeh_bar", "bokeh_scatter", "folium_map"]:
    with open(f"{name}.html", encoding="utf-8") as f:
        text = f.read()
    print(f"{name + '.html':<20} written, {len(text):>9,} characters")
print("\nPlotly bar trace type  :", fig_bar.data[0].type, "with", len(fig_bar.data[0].x), "bars")
print("Plotly scatter traces  :", [t.mode for t in fig_sc.data])
print("Iris points plotted    :", len(iris))
```

Output

```
plotly_bar.html      written,     8,190 characters
plotly_scatter.html  written,     9,758 characters
bokeh_line.html      written,     6,570 characters
bokeh_bar.html       written,     6,396 characters
bokeh_scatter.html   written,     7,401 characters
folium_map.html      written,     4,073 characters

Plotly bar trace type  : bar with 5 bars
Plotly scatter traces  : ['markers', 'lines']
Iris points plotted    : 150
```

Note. The script writes six HTML files; open them in a browser to hover, zoom and pan. City populations are illustrative numbers. `trendline='ols'` in Plotly needs the `statsmodels` package.

Suggested marks split. comparison 3, five working charts 5, choice guide 2.

M2-C6. EDA, K-means, PCA and the modelling libraries [10 marks]

K-means and PCA in brief

1. **K-means**: choose k, place k centroids, assign every point to its nearest centroid, move each centroid to the mean of its points, repeat until assignments stop changing. Choose k with the elbow plot of inertia (within-cluster sum of squares).
2. **PCA**: standardise features, find orthogonal directions of maximum variance (eigenvectors of the covariance matrix, computed by SVD), project the data on the first components. `explained_variance_ratio_` says how much information each component keeps.

Reading the results below

- Cluster sizes 62 / 50 / 38. Cluster 1 holds exactly the 50 setosa flowers. Versicolor and virginica overlap: cluster 0 has 48 versicolor + 14 virginica, cluster 2 has 2 versicolor + 36 virginica. K-means finds structure without labels, but it cannot separate overlapping species perfectly.
- PCA: PC1 explains 73.0% and PC2 22.9% of the variance, together 95.8%, so two components preserve almost all the information. PC1 is a "size" axis: petal length, petal width and sepal length load positively.
- The correlation matrix of random data is near zero everywhere, except `log_F1` with `Feature1` (0.90), because one is a transform of the other.

Code: code/m2_q6_ml.py

```
"""M2-Q6  K-means clustering and PCA on Iris (Lab experiments 4, 11 and 12)."""
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import plotly.express as px
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

iris = load_iris(as_frame=True)
X = iris.data
y = iris.target

# ---- Lab 4: feature engineering / EDA on a small random frame ---------------
rng = np.random.default_rng(3)
df = pd.DataFrame({"Feature1": rng.uniform(1, 100, 200), "Feature2": rng.normal(50, 15, 200),
```

```
                  "Target": rng.integers(0, 2, 200)})
df["log_F1"] = np.log(df["Feature1"])                                  # log feature
df["F2_bin"] = pd.cut(df["Feature2"], bins=3, labels=["low", "medium", "high"])
print("bins:", df["F2_bin"].value_counts().to_dict())
print("correlation matrix (rounded):")
print(df[["Feature1", "Feature2", "Target", "log_F1"]].corr().round(2))

# ---- Lab 11: K-means, k = 3, on the four measurements -----------------------
km = KMeans(n_clusters=3, n_init=10, random_state=42).fit(X)
print("\nKMeans cluster sizes :", np.bincount(km.labels_).tolist())
print("inertia (WCSS)       :", round(km.inertia_, 2))
print("cluster vs true species (rows = cluster):")
print(pd.crosstab(km.labels_, y.map(dict(enumerate(iris.target_names)))))

fig3d = px.scatter_3d(X.assign(cluster=km.labels_.astype(str)),
                      x="sepal length (cm)", y="sepal width (cm)", z="petal length (cm)",
                      color="cluster", title="K-means clusters (Iris)")
fig3d.write_html("kmeans_3d.html", include_plotlyjs="cdn")

# ---- Lab 12: PCA to two components ------------------------------------------
Xs = StandardScaler().fit_transform(X)                                 # scale first!
pca = PCA(n_components=2).fit(Xs)
Z = pca.transform(Xs)
print("\nPCA explained variance ratio:", pca.explained_variance_ratio_.round(3).tolist(),
      "| total", round(pca.explained_variance_ratio_.sum(), 3))
print("PC1 loadings:", dict(zip(X.columns, pca.components_[0].round(2))))

fig, ax = plt.subplots(1, 2, figsize=(9, 3.6))
ax[0].scatter(Z[:, 0], Z[:, 1], c=km.labels_, cmap="viridis", s=18)
ax[0].set_title("PCA projection coloured by K-means cluster")
ax[0].set_xlabel("PC1 (%.1f%%)" % (100 * pca.explained_variance_ratio_[0]))
ax[0].set_ylabel("PC2 (%.1f%%)" % (100 * pca.explained_variance_ratio_[1]))
ks = range(1, 8)
ax[1].plot(list(ks), [KMeans(k, n_init=10, random_state=42).fit(X).inertia_ for k in ks], marker="o")
ax[1].set_title("Elbow method: inertia vs k")
ax[1].set_xlabel("k")
ax[1].set_ylabel("inertia")
fig.tight_layout()
fig.savefig("kmeans_pca.png", dpi=150)
print("\nsaved kmeans_pca.png")
```

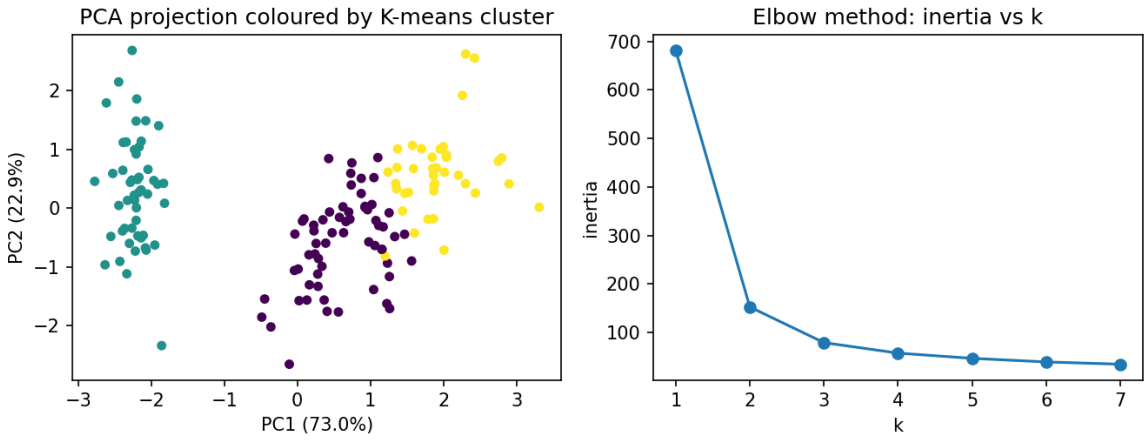
Output

```
bins: {'medium': 140, 'high': 33, 'low': 27}
correlation matrix (rounded):
          Feature1  Feature2  Target  log_F1
Feature1      1.00     -0.04   -0.09    0.90
Feature2     -0.04      1.00    0.04   -0.06
Target       -0.09      0.04    1.00   -0.05
log_F1        0.90     -0.06   -0.05    1.00

KMeans cluster sizes : [62, 50, 38]
inertia (WCSS)       : 78.85
cluster vs true species (rows = cluster):
target  setosa  versicolor  virginica
row_0
0            0          48         14
1           50           0          0
2            0           2         36

PCA explained variance ratio: [0.73, 0.229] | total 0.958
PC1 loadings: {'sepal length (cm)': 0.52, 'sepal width (cm)': -0.27, 'petal length (cm)': 0.58, 'petal width (cm)':
    0.56}

saved kmeans_pca.png
```



Left: Iris projected on the first two principal components, coloured by K-means cluster. Right: elbow plot.

Note. PCA used standardised features, K-means used the raw measurements (all in cm), so the two views are not identical. Always scale before PCA when the units differ.

Suggested marks split. EDA steps 2, K-means 3, PCA 3, interpretation 2.

Module 3: Scala data structures and object-oriented programming

Syllabus (12 hours): expanded function format; variables and strings; user input; numbers; variable types; operators; booleans; arrays, lists, tuples, sets, hash set, maps; functional combinators (map); objects and classes; anonymous object, singleton; companion object, case classes; constructors; method overloading; `this`; inheritance; field overriding; `final`; abstract class; trait and mixins; access modifiers; Scala array and the REPL.

Revision: Scala in ten lines

- Statically typed, runs on the JVM, mixes object-oriented and functional styles; everything is an object, every expression has a value.
- Prefer `val`, immutable collections and expressions over statements.
- Labs 13 to 17 of your list of experiments map to the five long questions below.

Tip. For code questions, write the result of each `println` as a comment next to it. It shows you can predict output without a compiler.

Watch out. Do not call `List` "mutable" because you wrote `var l = List(1)`; the variable changes, the list does not.

Module 3: Test yourself (questions only)

A. Multiple choice (11 questions, 1 mark each)

M3-A1. What happens with `val x = 5; x = 6` in Scala?

(a) x becomes 6
(b) compile-time error: reassignment to val
(c) runtime exception
(d) x stays 5 silently

M3-A2. Which expression creates a new List with 0 added at the front of `List(1, 2)`?

(a) `List(1, 2) :+ 0`
(b) `0 :: List(1, 2)`
(c) `List(1, 2).add(0)`
(d) `List(1, 2)(0) = 0`

M3-A3. Which of these is NOT generated automatically for a case class?

(a) `toString`
(b) `copy`
(c) structural `equals`
(d) a `main` method

M3-A4. `Student("Rohan", 101)` (no `new`) works for a normal class when:

(a) the class is final
(b) its companion object defines `apply`
(c) the class is abstract
(d) the class has a private field

M3-A5. In Scala a class can extend:

(a) any number of classes
(b) one class and any number of traits
(c) only one trait
(d) one abstract class and no traits

M3-A6. A method declared `final def` in a superclass:

(a) cannot be overridden in a subclass
(b) must be overridden
(c) is abstract
(d) is private

M3-A7. What does `List(1, 2, 3, 4).foldLeft(0)((acc, x) => acc + x)` return?

(a) List(1, 3, 6, 10)
(b) 10
(c) 0
(d) 24

M3-A8. A `private` member of a class is accessible:

(a) everywhere
(b) in the class itself (and its companion object)
(c) in subclasses only
(d) in the same package only

M3-A9. `object Config { val name = "app" }` defines:

(a) a class with no constructor
(b) a singleton: exactly one instance created lazily on first use
(c) a trait
(d) a static import

M3-A10. What is `List(1, 2, 3).map(_ * 2).filter(_ > 2)`?

(a) List(2, 4, 6)
(b) List(4, 6)
(c) List(1, 2, 3)
(d) List(6)

M3-A11. To redefine a concrete method inherited from a superclass you must write:

(a) `overload`
(b) `override`
(c) `super`
(d) `final`

B. Short answers

M3-B1. Explain `val`, `var`, `lazy val` and type inference. [3 marks]

M3-B2. Show the different ways to write a function in Scala (expanded format, short format, function literal). [3 marks]

M3-B3. Differentiate Array, List, Set, Map and Tuple in Scala. [5 marks]

M3-B4. Explain `map`, `flatMap`, `filter`, `foldLeft` and `reduce` with one example each. [5 marks]

M3-B5. What are primary and auxiliary constructors? What does `this` do? [3 marks]

M3-B6. Differentiate method overloading and method overriding. [3 marks]

M3-B7. Explain Scala access modifiers. [3 marks]

M3-B8. Differentiate a singleton `object`, a companion object and an anonymous class. [3 marks]

M3-B9. What is the Scala REPL? List two commands. [2 marks]

M3-B10. Explain string interpolation: `s`, `f` and `raw`. [2 marks]

C. Long questions

M3-C1. Scala collections and functional combinators [10 marks]

(Lab experiments 13 to 16) Create an array of integers 2, 5, 9, 14, 20 and a function returning the sum; create an array of five fruit names and a function that joins them with commas; iterate over the array. Create a list of the first five primes and square each element; build a set of unique characters of "hello world" and intersect two sets; create a tuple (Int, String, Boolean) and print each element. Create a HashMap of names and grades with a duplicate key, merge two maps keeping the higher grade, and store several values per key using lists (chaining). Use `filter`, `map`, `flatMap` and `foldLeft` on the list 1 to 10.

M3-C2. Classes, constructors, inheritance and companion objects [10 marks]

(FAT high-frequency question 6) Write a Scala program that demonstrates: classes with primary and auxiliary constructors; method overloading; the `this` keyword; inheritance with field overriding and `final`; an abstract class; access modifiers; a singleton object; a companion object with an `apply` factory; and an anonymous class. Explain how a companion object removes the need for `new`.

M3-C3. Case classes and pattern matching [10 marks]

(Lab experiment 17, FAT high-frequency question 5) Create `case class Person(name, age, city)` and a list of five people. Filter out people older than 30, group by city, convert each person to the string "Name is Age years old and lives in City", and sort by age. List four capabilities that distinguish a case class from a normal class and show pattern matching with a guard.

M3-C4. Traits, mixins and linearization [10 marks]

(FAT high-frequency question 9) Contrast traits with abstract classes. Explain trait mixins and how Scala resolves the diamond problem through linearization, using `class D extends B with C` where both `B` and `C` extend `A` and override `who` calling `super.who`.

M3-C5. Scala fundamentals [10 marks]

Demonstrate Scala fundamentals: the expanded function format and shorter forms, variables and strings, reading user input, numbers and variable types, operators and booleans. Explain what is special about operators and about `==` versus `eq` in Scala.

Module 3: Answers

A. Multiple-choice key

No.	Ans.	Why
M3-A1	b	`val` is an immutable reference. Use `var` if the reference must change.
M3-A2	b	`::` (cons) prepends in O(1). `:+` appends and costs O(n) on a List. Lists are immutable, so a new list is returned.
M3-A3	d	The compiler adds `apply`, `unapply`, `copy`, `equals`, `hashCode` and `toString`. A `main` method you write yourself (or extend `App`).
M3-A4	b	The call is rewritten to `Student.apply("Rohan", 101)`. Case classes get this companion `apply` for free.
M3-A5	b	Single class inheritance plus multiple trait mixins (`class A extends B with T1 with T2`).
M3-A6	a	`final` forbids overriding (on a class it forbids extending).
M3-A7	b	Accumulator steps: 0+1=1, 1+2=3, 3+3=6, 6+4=10.
M3-A8	b	`protected` adds subclasses; default (no modifier) is public in Scala.
M3-A9	b	An `object` is Scala's singleton, used for utilities and as the home of `main` and companion factories.
M3-A10	b	map gives List(2, 4, 6); filter keeps values greater than 2.
M3-A11	b	`override` is mandatory when replacing a concrete member (optional for abstract ones).

B. Short answers

M3-B1. Explain `val`, `var`, `lazy val` and type inference. [3 marks]

- `val` : immutable binding, assigned once. `var` : mutable binding. Prefer `val` (functional style, thread-safe).
- `lazy val` : evaluated on first use, once, then cached.
- Type inference: `val n = 10` is an `Int`, `val s = "hi"` a `String`. Annotate public method return types: `def area: Double`.

M3-B2. Show the different ways to write a function in Scala (expanded format, short format, function literal). [3 marks]

```
def addExpanded(a: Int, b: Int): Int = {   // expanded: braces + explicit result type
    a + b
}
def add(a: Int, b: Int) = a + b             // short: type inferred, no braces
val addLit: (Int, Int) => Int = (a, b) => a + b   // function literal stored in a val
def twice(f: Int => Int, x: Int) = f(f(x))  // higher-order: takes a function
twice(_ * 3, 2)                              // 18, `_` is the placeholder argument
```

The last expression is the result; `return` is legal but not idiomatic.

M3-B3. Differentiate Array, List, Set, Map and Tuple in Scala. [5 marks]

Type	Mutable?	Ordered / unique	Notes
`Array[T]`	elements yes, size fixed	ordered, duplicates ok	index `a(0)`; maps to a JVM array; O(1) access
`List[T]`	no (immutable)	ordered, duplicates ok	linked list; `::` prepend O(1); index access O(n)
`Set[T]`	no by default	unordered, unique	membership test fast; `union`, `intersect`, `diff`
`Map[K,V]`	no by default	unique keys	`get` returns Option; `getOrElse`; `+` adds a pair
`(A, B, C)`	no	fixed size, mixed types	access with `._1`, `._2`; destructure `val (a, b) = t`

Mutable versions live in `scala.collection.mutable` (`ArrayBuffer`, `mutable.Set`, `mutable.HashMap`).

M3-B4. Explain `map`, `flatMap`, `filter`, `foldLeft` and `reduce` with one example each. [5 marks]

Combinator	Idea	Example -> result
`map`	transform each element	`List(1,2,3).map(_ * 2)` -> List(2, 4, 6)
`filter`	keep elements passing a test	`List(1,2,3,4).filter(_ % 2 == 0)` -> List(2, 4)
`flatMap`	map then flatten one level	`List("hi","yo").flatMap(_.toList)` -> List(h, i, y, o)
`foldLeft`	accumulate from the left with a start value	`List(1,2,3,4).foldLeft(0)(_ + _)` -> 10
`reduce`	like fold but starts with the first element (fails on an empty list)	`List(1,2,3).reduce(_ * _)` -> 6

M3-B5. What are primary and auxiliary constructors? What does `this` do? [3 marks]

- The **primary constructor** is the class header and body: `class Account(val owner: String, var balance: Double)`.
- An **auxiliary constructor** is `def this(...)`; its first statement must call another constructor: `def this(owner: String) = this(owner, 0.0)`.
- `this` refers to the current object (`this.balance = balance` when a parameter hides a field) and is also the name used to call another constructor.

M3-B6. Differentiate method overloading and method overriding. [3 marks]

Overloading: same method name, different parameter lists, in the same class; resolved at compile time (`deposit(x)` and `deposit(x, note)`). **Overriding**: a subclass replaces an inherited method or field with the same signature using `override`; resolved at run time through dynamic dispatch. `final` blocks overriding.

M3-B7. Explain Scala access modifiers. [3 marks]

Modifier	Visible in
(none) public	everywhere (default)
`protected`	the class and its subclasses (not the rest of the package)
`private`	the class itself and its companion object
`private[pkg]`	qualified: visible up to the named package or class

M3-B8. Differentiate a singleton `object`, a companion object and an anonymous class. [3 marks]

- **Singleton object**: `object Config { ... }` is a single instance, no `new`, used for utilities and `main`.
- **Companion object**: an `object` with the same name as a class in the same file; it can read the class's private members and defines `apply` factories and static-like helpers.
- **Anonymous class**: a one-off subclass created inline: `new Animal { override val sound = "meow" }`.

M3-B9. What is the Scala REPL? List two commands. [2 marks]

The REPL (Read-Eval-Print Loop) is an interactive shell started with the `scala` command. You type an expression, it compiles and evaluates it, prints the value and gives it a name (`res0`). `:paste` enters multi-line mode (end with Ctrl+D); `:quit` exits; `:help` lists commands. It is ideal for trying array and list operations.

M3-B10. Explain string interpolation: `s`, `f` and `raw`. [2 marks]

```
val name = "Rohan"; val gpa = 8.4567
s"Hello $name, next year ${2026 + 1}"   // s: embed values and expressions
f"GPA = $gpa%.2f"                      // f: printf-style formatting -> 8.46
raw"no escape \n here"                 // raw: backslashes are not interpreted
```

C. Long answers

M3-C1. Scala collections and functional combinators [10 marks]

Key ideas

- Arrays are mutable and fixed size even when the reference is a `val`; Lists, Sets and Maps are immutable by default, so every "change" returns a new collection.
- A duplicate key in a Map (or `put` into a mutable HashMap) **overwrites** the old value. This is a key conflict, resolved by a rule such as `math.max`. A true hash collision (two different keys in one bucket) is resolved internally by the hash table. Storing a `List` of values per key is the user-level "chaining" asked in the lab.
- Combinator pipeline: `filter` -> `map` -> `flatMap` (flatten) -> `foldLeft`/`reduce` (aggregate). `foldLeft(1L)(_ * _)` over 1 to 10 gives 3628800 (use `Long`, because 13! already overflows `Int`).
- `Set` operations: `intersect`, `union`, `diff`; a set built from a String keeps unique characters.

Code: code/m3_q1_collections.scala

```
// M3-Q1  Collections and functional combinators (Lab experiments 13-16)
import scala.collection.mutable

object M3Q1 extends App {
  // ---- Arrays (mutable, fixed size, index with parentheses) ----
  val nums = Array(2, 5, 9, 14, 20)
  def sum(a: Array[Int]): Int = a.sum
  val fruits = Array("apple", "banana", "cherry", "mango", "orange")
  def join(a: Array[String]): String = a.mkString(",")
  nums(0) = 99                                  // arrays are mutable even when declared with val
  println(s"sum = ${sum(nums)}, joined = ${join(fruits)}")
```

```
  // ---- List (immutable linked list), Set, Tuple ----
  val primes = List(2, 3, 5, 7, 11)
  println("squares : " + primes.map(p => p * p))
  println("cons    : " + (0 :: primes))                  // O(1) prepend creates a new list
  val s1 = "hello world".filter(_ != ' ').toSet          // unique characters
  val s2 = "world".toSet
  println("set and : " + (s1 intersect s2).toList.sorted)
  val t = (42, "scala", true)
  println(s"tuple   : ${t._1}, ${t._2}, ${t._3}")

  // ---- Map: duplicate key overwrites; merge with conflict rule; chaining ----
  val grades = mutable.HashMap("Alice" -> 85, "Bob" -> 78)
  grades.put("Alice", 95)                                // same key -> old value replaced
  println("grades  : " + grades.toList.sorted)

  def mergeMax(a: Map[String, Int], b: Map[String, Int]): Map[String, Int] =
    (a.keySet ++ b.keySet).map(k => k -> math.max(a.getOrElse(k, 0), b.getOrElse(k, 0))).toMap
  println("merge   : " + mergeMax(Map("Alice" -> 85, "Bob" -> 90), Map("Alice" -> 92, "Emma" -> 88)).toList.sorted)

  val attempts = Seq("Alice" -> 85, "Bob" -> 78, "Alice" -> 95)
  val chained = attempts.foldLeft(Map.empty[String, List[Int]]) { case (acc, (k, v)) =>
    acc + (k -> (acc.getOrElse(k, Nil) :+ v))            // separate chaining with a List per key
  }
  println("chained : " + chained.toList.sortBy(_._1))

  // ---- Functional combinators ----
  val xs = (1 to 10).toList
  println("filter  : " + xs.filter(_ % 2 == 0))
  println("map     : " + xs.map(_ * 3))
  println("flatMap : " + xs.take(4).flatMap(n => List((n, n * n))))
  println("foldLeft: " + xs.foldLeft(1L)(_ * _))         // 10! = 3628800
  println("reduce  : " + xs.reduce(_ + _))
  println("flatMap on strings: " + List("hi", "yo").flatMap(_.toList))
}
```

Output

```
sum = 147, joined = apple,banana,cherry,mango,orange
squares : List(4, 9, 25, 49, 121)
cons    : List(0, 2, 3, 5, 7, 11)
set and : List(d, l, o, r, w)
tuple   : 42, scala, true
grades  : List((Alice,95), (Bob,78))
merge   : List((Alice,92), (Bob,90), (Emma,88))
chained : List((Alice,List(85, 95)), (Bob,List(78)))
filter  : List(2, 4, 6, 8, 10)
map     : List(3, 6, 9, 12, 15, 18, 21, 24, 27, 30)
flatMap : List((1,1), (2,4), (3,9), (4,16))
foldLeft: 3628800
reduce  : 55
flatMap on strings: List(h, i, y, o)
```

Note. All Scala programs in this bank were compiled and run with Scala 2.13.12 on Java 17 (via scala-cli), the same Scala version as your Colab (Almond) setup.

Suggested marks split. arrays 2, list/set/tuple 2, map conflicts and chaining 3, combinators 3.

M3-C2. Classes, constructors, inheritance and companion objects [10 marks]

Syllabus concept	Where it appears in the code
Primary constructor	`class Account(val owner: String, private var balance: Double)`
Auxiliary constructor, `this`	`def this(owner: String) = this(owner, 0.0)`
Method overloading	`deposit(amount)` and `deposit(amount, note)`
Inheritance, field overriding	`class Dog extends Animal { override val sound = "woof" }`
`final`	`final def kind()` cannot be overridden
Abstract class	`abstract class Shape { def area: Double }` implemented by `Circle`, `Rect`
Access modifiers	`private`, `protected`, public in `Base` / `Child`
Singleton object	`object Config`
Companion object, `apply`	`class Student private (...)` + `object Student { def apply(...) }`
Anonymous class	`new Animal { override val sound = "meow" }`

How the companion object works

The compiler rewrites `Student("Rohan", 101)` to `Student.apply("Rohan", 101)`. The companion shares the class name and file, so it can call the private constructor and read private members. It is the idiomatic factory: it can validate, count instances (as the demo does: `created = 2`), cache objects or return a subtype, none of which `new` allows. Case classes get a generated `apply` automatically.

Watch out. A `def` cannot override a `val`, but a `val` can override a parameterless `def`. When overriding a concrete member you must write `override`.

Code: code/m3_q2_oop.scala

```scala
// M3-Q2  Classes, constructors, overloading, inheritance, access modifiers, singleton, companion
class Account(val owner: String, private var balance: Double) {       // primary constructor
  def this(owner: String) = this(owner, 0.0)                          // auxiliary constructor (uses this(...))

  def deposit(amount: Double): Unit = { balance += amount }
  def deposit(amount: Double, note: String): Unit = {                 // method overloading: different parameters
    balance += amount
    println(s"  note: $note")
  }
  def current: Double = balance                                       // private var exposed read-only
  override def toString: String = f"Account($owner, $balance%.2f)"
}

// ---- inheritance, field overriding, final ----
class Animal {
  val sound: String = "..."
  def describe(): String = s"makes the sound '$sound'"
  final def kind(): String = "animal"                                 // final -> cannot be overridden
}
class Dog extends Animal {
  override val sound: String = "woof"                                 // field overriding needs `override`
  override def describe(): String = "A dog " + super.describe()
}

// ---- abstract class ----
abstract class Shape {
  def area: Double                                                    // abstract member
  def show(): String = f"${getClass.getSimpleName} area = $area%.2f"  // concrete member
}
class Circle(r: Double) extends Shape { def area: Double = math.Pi * r * r }
class Rect(w: Double, h: Double) extends Shape { def area: Double = w * h }

// ---- access modifiers ----
class Base {
  private val secret = "private"            // visible only inside Base
  protected val shared = "protected"        // Base and subclasses
  val open = "public"                       // everywhere (default)
  def reveal: String = secret
}
class Child extends Base { def peek: String = shared }

// ---- singleton object and companion object with apply ----
object Config { val appName = "PDS Demo"; def hello(): String = s"Welcome to $appName" }  // singleton

class Student private (val name: String, val id: Int)                 // private constructor
object Student {                                                      // companion: same name, same file
  private var count = 0
  def apply(name: String, id: Int): Student = { count += 1; new Student(name, id) }  // factory
  def created: Int = count
}

object M3Q2 extends App {
  val a = new Account("Asha")                 // auxiliary constructor
  a.deposit(500)
  a.deposit(250, "salary")                    // overloaded version
  println(a)

  val d = new Dog
  println(d.describe() + " | kind = " + d.kind())

  val shapes: List[Shape] = List(new Circle(1.5), new Rect(2, 3))
  shapes.foreach(s => println(s.show()))

  val c = new Child
  println(s"${c.peek}, ${c.open}, ${c.reveal}")
  // c.secret  // would not compile: value secret in class Base cannot be accessed

  println(Config.hello())                     // singleton: no `new`, one instance
  val s1 = Student("Rohan", 101)              // companion apply: no `new`
  val s2 = Student("Meena", 102)
  println(s"${s1.name}, ${s2.name}, created = ${Student.created}")

  // anonymous class: an instance of a one-off subclass
  val cat = new Animal { override val sound = "meow" }
  println("anonymous: " + cat.describe())
}
```

Output

```
  note: salary
Account(Asha, 750.00)
A dog makes the sound 'woof' | kind = animal
Circle area = 7.07
Rect area = 6.00
protected, public, private
Welcome to PDS Demo
Rohan, Meena, created = 2
anonymous: makes the sound 'meow'
```

Suggested marks split. constructors and overloading 3, inheritance and abstract 3, access modifiers 1, singleton/companion 3.

M3-C3. Case classes and pattern matching [10 marks]

Capability	Normal class	Case class
Constructor parameters	private unless marked `val`	public `val` fields automatically
Instantiation	`new` required (unless a companion `apply` exists)	companion `apply` generated: `Person("A", 22, "X")`
Equality	reference equality (`eq`) by default	structural `equals` and `hashCode` on all fields
`toString`	`Person@1b2c3d`	`Person(Asha,22,Chennai)`
Copy with changes	write your own	`p.copy(city = "Pune")`
Pattern matching	needs a hand-written `unapply`	generated `unapply`: `case Person(n, a, _) =>`

Case classes are meant for immutable data records. Combined with `sealed` traits they give exhaustive pattern matching (the compiler warns if a case is missing).

Code: code/m3_q3_case_traits.scala

```scala
// M3-Q3  Case classes, pattern matching, traits and mixins (Lab 17, FAT top-10 questions 5, 6, 9)
case class Person(name: String, age: Int, city: String)

case class Transaction(id: String, amount: Double, status: String)

trait Logger { def log(msg: String): Unit = println(s"  [LOG] $msg") }
trait Auditor { def audit(msg: String): Unit = println(s"  [AUDIT] $msg") }
trait Notifier { def notifyUser(msg: String): Unit = println(s"  [NOTIFY] $msg") }

// Linearization demo: each trait calls super, so the call order shows the chain
trait A { def who: String = "A" }
trait B extends A { override def who: String = "B > " + super.who }
trait C extends A { override def who: String = "C > " + super.who }
class D extends B with C { override def who: String = "D > " + super.who }

class Worker(val name: String) extends Logger with Auditor {          // mixin composition
  def work(): Unit = { log(s"$name started"); audit(s"$name finished") }
}

object M3Q3 extends App {
  // ---- case class: what the compiler generates ----
  val p1 = Person("Asha", 22, "Chennai")
  val p2 = Person("Asha", 22, "Chennai")
  println(s"toString : $p1")
  println(s"equals   : ${p1 == p2}  (structural)   eq: ${p1 eq p2}  (reference)")
  println(s"hashCode : ${p1.hashCode == p2.hashCode}")
  println(s"copy     : ${p1.copy(city = "Pune")}")
  val Person(n, a, _) = p1                                              // unapply -> deconstruction
  println(s"unapply  : name=$n age=$a")

  // ---- Lab 17 style operations ----
  val people = List(Person("Asha", 22, "Chennai"), Person("Ravi", 34, "Delhi"),
                    Person("Meena", 41, "Chennai"), Person("Kabir", 29, "Pune"), Person("Sneha", 33, "Delhi"))
  println("older than 30  : " + people.filter(_.age > 30).map(_.name))
  println("grouped by city: " + people.groupBy(_.city).map { case (c, ps) => c -> ps.map(_.name)
      }.toList.sortBy(_._1))
  println("as strings     : " + people.take(2).map(p => s"${p.name} is ${p.age} years old and lives in ${p.city}"))
  println("sorted by age  : " + people.sortBy(_.age).map(_.name))

  // ---- pattern matching on a case class ----
  def process(tx: Transaction): String = tx match {
    case Transaction(id, amt, "SUCCESS") if amt > 10000 => s"Large transaction $id logged"
    case Transaction(id, _, "SUCCESS")                  => s"Standard transaction $id approved"
    case Transaction(id, _, "FAILED")                   => s"Alert: Transaction $id failed"
    case _                                              => "Unknown transaction state"
  }
  List(Transaction("T1", 25000, "SUCCESS"), Transaction("T2", 900, "SUCCESS"),
       Transaction("T3", 50, "FAILED"), Transaction("T4", 10, "PENDING")).foreach(t => println("  " + process(t)))
```

```
  // ---- traits, mixins and linearization ----
  new Worker("Asha").work()
  println("linearization of D extends B with C: " + (new D).who)
  val adHoc = new Worker("Kabir") with Notifier                    // mix a trait in at instantiation time
  adHoc.work()
  adHoc.notifyUser("shift over")
}
```

Output

```
toString : Person(Asha,22,Chennai)
equals   : true  (structural)   eq: false  (reference)
hashCode : true
copy     : Person(Asha,22,Pune)
unapply  : name=Asha age=22
older than 30  : List(Ravi, Meena, Sneha)
grouped by city: List((Chennai,List(Asha, Meena)), (Delhi,List(Ravi, Sneha)), (Pune,List(Kabir)))
as strings     : List(Asha is 22 years old and lives in Chennai, Ravi is 34 years old and lives in Delhi)
sorted by age  : List(Asha, Kabir, Sneha, Ravi, Meena)
  Large transaction T1 logged
  Standard transaction T2 approved
  Alert: Transaction T3 failed
  Unknown transaction state
  [LOG] Asha started
  [AUDIT] Asha finished
linearization of D extends B with C: D > C > B > A
  [LOG] Kabir started
  [AUDIT] Kabir finished
  [NOTIFY] shift over
```

Suggested marks split. four capabilities 4, list operations 3, pattern matching with guard 3.

M3-C4. Traits, mixins and linearization [10 marks]

Point	Abstract class	Trait
Inheritance	a class extends exactly one	a class mixes in many: `extends Base with T1 with T2`
Constructor parameters	yes	not in Scala 2 (Scala 3 allows them)
State and behaviour	fields, concrete and abstract methods	also; commonly small reusable behaviours
Java interop	behaves like an abstract Java class	compiles to an interface (plus helper code)
Use when	a strong "is-a" base type with constructor arguments	adding orthogonal capabilities (logging, auditing)

Mixins

`class Worker(name) extends Logger with Auditor` inherits the concrete methods of both traits. A trait can also be mixed in for a single object: `new Worker("Kabir") with Notifier`.

Linearization of `D extends B with C`

Scala orders the whole parent chain into one line, so `super` always has exactly one meaning. The rule: **L(D) = D, followed by L(C) then L(B), where a type already seen later in the line replaces its earlier occurrence**.

1. L(A) = A
2. L(B) = B, A
3. L(C) = C, A
4. L(D) = D, L(C) +> L(B) = D, {C, A} +> {B, A} = **D, C, B, A**
5. `A` appears once, at the end, so the diamond is resolved. `super.who` in D calls C, which calls B, which calls A.

The program printed `D > C > B > A`, confirming the order. The class mixed in *last* (C) is consulted first.

Excerpt from m3_q3_case_traits.scala

```
trait A { def who: String = "A" }
trait B extends A { override def who = "B > " + super.who }
trait C extends A { override def who = "C > " + super.who }
class D extends B with C { override def who = "D > " + super.who }
println((new D).who)
```

Output of the whole program (the last lines show the trait mixins and `D > C > B > A`)

```
  [LOG] Asha started
  [AUDIT] Asha finished
linearization of D extends B with C: D > C > B > A
  [LOG] Kabir started
  [AUDIT] Kabir finished
  [NOTIFY] shift over
```

Suggested marks split. trait vs abstract class 3, mixins 2, linearization steps 4, code 1.

M3-C5. Scala fundamentals [10 marks]

- **Types**: `Byte` (8-bit), `Short` (16), `Int` (32), `Long` (64), `Float` (32), `Double` (64), `Char` (16), `Boolean`, `String`. `Int` arithmetic overflows silently: `Int.MaxValue + 1` is `-2147483648`.
- **Operators are methods**: `1 + 2` is `1.+(2)`; `a max b` is `a.max(b)`. Precedence follows the first character of the operator.
- `==` calls `equals` (value equality); `eq` compares references. `&&` and `||` short-circuit, so the right side may never run.
- **Input**: `scala.io.StdIn.readLine()` and `readInt()` read from the keyboard. The demo feeds them from a `StringReader` so the program is repeatable.

Code: code/m3_q4_basics.scala

```
// M3-Q4  Scala fundamentals: function formats, variables, strings, input, numbers, operators, booleans
import java.io.StringReader

object M3Q4 extends App {
  // ---- function formats: full (expanded) form, short form, function literal, higher-order ----
  def addExpanded(a: Int, b: Int): Int = {          // expanded: braces, explicit return type
    return a + b                                    // `return` is legal but unidiomatic
  }
  def add(a: Int, b: Int) = a + b                   // last expression is the result, type inferred
  val addLit: (Int, Int) => Int = (a, b) => a + b   // anonymous function stored in a val
  def twice(f: Int => Int, x: Int): Int = f(f(x))   // higher-order function
  println(s"functions : ${addExpanded(2, 3)} ${add(2, 3)} ${addLit(2, 3)} ${twice(_ * 3, 2)}")

  // ---- val / var, type inference, lazy val ----
  val fixed = 10                                    // immutable reference
  var counter = 0                                   // mutable reference
  counter += 1
  lazy val expensive = { println("  (computing lazy val once)"); 42 }
  println(s"val/var    : $fixed $counter, types ${fixed.getClass.getSimpleName}")
  println("lazy       : " + expensive + " " + expensive)

  // ---- strings and interpolation ----
  val name = "Rohan"; val gpa = 8.4567
  println(s"s-string   : Hello $name, next year ${2026 + 1}")
  println(f"f-string   : GPA = $gpa%.2f, padded = ${42}%05d")
  println(raw"raw-string : no escape \n here")
  println("methods    : " + "scala".toUpperCase + " " + "scala".reverse + " " + "a,b,c".split(",").toList)

  // ---- getting user input (simulated here with a StringReader instead of the keyboard) ----
  Console.withIn(new StringReader("25\nScala\n")) {
    val age = scala.io.StdIn.readInt()
    val lang = scala.io.StdIn.readLine()
    println(s"input      : age=$age, language=$lang")
  }

  // ---- numbers and operators (operators are just methods) ----
  println(s"numbers    : ${7 / 2} ${7 % 2} ${7 / 2.0} ${1 + 2 * 3} ${(1).+(2)}")
  println(s"types      : ${Int.MaxValue} ${Long.MaxValue} ${5.toDouble} ${3.9.toInt} ${'a'.toInt}")
  println(s"overflow   : ${Int.MaxValue + 1}")
  println(s"bitwise    : ${6 & 3} ${6 | 3} ${6 ^ 3} ${1 << 3}")

  // ---- booleans, short-circuit evaluation, == versus eq ----
  def noisy(b: Boolean): Boolean = { println("  evaluated"); b }
  println("short-circuit: " + (false && noisy(true)))      // right side never evaluated
  println("or           : " + (true || noisy(false)))
  println(s"equality   : ${"ab" == "a" + "b"} ${List(1, 2) == List(1, 2)} ${new String("x") eq new String("x")}")
}
```

Output

```
functions : 5 5 5 18
val/var    : 10 1, types int
  (computing lazy val once)
lazy       : 42 42
s-string   : Hello Rohan, next year 2027
f-string   : GPA = 8.46, padded = 00042
raw-string : no escape \n here
methods    : SCALA alacs List(a, b, c)
input      : age=25, language=Scala
numbers    : 3 1 3.5 7 3
types      : 2147483647 9223372036854775807 5.0 3 97
overflow   : -2147483648
bitwise    : 2 7 5 8
short-circuit: false
or           : true
equality   : true true false
```

Suggested marks split. functions 2, variables and strings 2, input 1, numbers and operators 3, booleans and equality 2.

Module 4: Scala libraries and Spark basics

Syllabus (8 hours): Scala libraries: Breeze and Saddle; exception handling; Apache Spark architecture; Spark and big data; Apache Spark components.

Labs 18 to 20 (Saddle, Vegas, Breeze, Plotly.Scala, Spire) belong here. Spark is not in the lab list, so expect theory and short code in the FAT.

Tip. Draw the Spark architecture diagram in every Spark answer: driver, cluster manager, workers with executors and tasks. It is cheap marks.

Watch out. Do not say Spark "stores" data. It computes on data held in external storage; executors only cache partitions temporarily.

Module 4: Test yourself (questions only)

A. Multiple choice (11 questions, 1 mark each)

M4-A1. Which Spark process builds the DAG of stages and schedules tasks on executors?

(a) Executor
(b) Driver
(c) Worker disk
(d) Cluster manager only

M4-A2. Which of these is an action?

(a) `map`
(b) `filter`
(c) `reduceByKey`
(d) `count`

M4-A3. RDD stands for:

(a) Random Data Directory
(b) Resilient Distributed Dataset
(c) Relational Data Document
(d) Remote Data Driver

M4-A4. After an executor fails, a lost RDD partition is recovered by:

(a) restoring a replica from HDFS always
(b) recomputing it from its lineage (the DAG of transformations)
(c) restarting the whole application
(d) asking the user

M4-A5. What does `Try("4x2".toInt)` evaluate to?

(a) `Success(4)`
(b) `Failure(NumberFormatException)`
(c) `None`
(d) an uncaught exception

M4-A6. Which Spark component provides DataFrames, SQL and the Catalyst optimiser?

(a) Spark Core
(b) Spark SQL
(c) MLlib
(d) GraphX

M4-A7. "Lazy evaluation" in Spark means:

(a) data is loaded slowly
(b) transformations are only recorded; computation starts when an action is called
(c) actions are queued for the next day
(d) executors start late

M4-A8. In Breeze, `a *:* b` for two `DenseVector[Double]` computes:

(a) the dot product
(b) the element-wise product
(c) the cross product
(d) the outer product

M4-A9. `Option[T]` is mainly used to:

(a) loop over a collection
(b) represent a value that may be absent as `Some(x)` or `None`, avoiding null
(c) declare a mutable value
(d) catch exceptions with a message

M4-A10. Which operation normally causes a shuffle (wide dependency)?

(a) `map`
(b) `filter`
(c) `reduceByKey`
(d) `flatMap`

M4-A11. Spark can run under which cluster managers?

(a) only its own standalone manager
(b) Standalone, YARN and Kubernetes
(c) only YARN
(d) only Kubernetes

B. Short answers

M4-B1. How does exception handling in Scala differ from Java? Show `try / catch / finally`. [5 marks]
M4-B2. Differentiate `Option`, `Try` and `Either`. [5 marks]
M4-B3. Compare Breeze, Saddle and Spire. [3 marks]
M4-B4. Name the main parts of the Spark architecture and the role of each. [5 marks]
M4-B5. State the four properties of an RDD and explain lineage. [3 marks]
M4-B6. Give three transformations and three actions of Spark, and explain why laziness helps. [3 marks]
M4-B7. Explain narrow and wide transformations, and what a shuffle is. [3 marks]
M4-B8. List the Spark ecosystem components and one use of each. [5 marks]
M4-B9. Why is Spark often preferred over Hadoop MapReduce? [3 marks]
M4-B10. RDD versus DataFrame. [3 marks]
M4-B11. Briefly describe the characteristics of big data (the V's). [2 marks]

C. Long questions

M4-C1. Exception handling in Scala [10 marks]

Explain exception handling in Scala. Write a program that shows `try / catch / finally`, a custom exception, `Option`, `Try` (with `recover`) and `Either`. Explain why functional error handling is preferred to throwing exceptions.

M4-C2. Scala libraries: Breeze, Saddle and Spire [10 marks]

(Lab experiments 18 to 20) Use Breeze to compute total revenue per fruit (price times quantity) and solve a linear system; use Saddle to load a small dataset, filter apples and compute their average price; use Spire for exact arithmetic. Say what role each library plays, and how charts are added (Vegas, Plotly.Scala).

M4-C3. Apache Spark architecture [10 marks]

Draw and explain the Apache Spark architecture. Describe how a Spark application is turned into jobs, stages and tasks, using a word-count program as the example.

M4-C4. RDDs, lineage, transformations and actions [10 marks]

(FAT high-frequency questions 3 and 4) What is an RDD? Define its four properties and explain how DAG lineage gives fault tolerance without replication. List three transformations and three actions, and show with a program that transformations are lazy.

M4-C5. Spark components and the Catalyst optimiser [10 marks]

(FAT high-frequency question 10) Describe the components of the Spark ecosystem. Explain how Spark SQL's Catalyst optimiser improves a query through logical optimisation, physical planning and code generation, and show an example.

Module 4: Answers

A. Multiple-choice key

No.	Ans.	Why
M4-A1	b	The driver runs your `main()`, holds the SparkSession, builds the DAG and sends tasks to executors. The cluster manager only allocates resources.
M4-A2	d	`count`, `collect`, `take`, `reduce` and `saveAsTextFile` trigger execution; the others only extend the lineage.
M4-A3	b	Resilient (recoverable), Distributed (partitioned over nodes), Dataset (collection of records); also immutable.
M4-A4	b	Spark remembers how each RDD was derived, so only the lost partitions are recomputed.
M4-A5	b	`Try` captures the exception inside a `Failure` value instead of throwing.
M4-A6	b	Spark SQL (with the DataFrame/Dataset API).
M4-A7	b	Laziness lets Spark see the whole plan, pipeline narrow steps and skip unused work.
M4-A8	b	`*:*` is element-wise; `dot` gives the inner product and `*` on matrices is matrix multiplication.
M4-A9	b	It makes absence explicit in the type, so the compiler makes you handle it.
M4-A10	c	`reduceByKey` must bring equal keys together from different partitions. `map`, `filter`, `flatMap` work inside each partition (narrow).
M4-A11	b	The application code is the same; the cluster manager only changes how resources are obtained.

B. Short answers

M4-B1. How does exception handling in Scala differ from Java? Show `try / catch / finally`. [5 marks]

- `try` is an *expression*: it has a value (the last expression of the try or the matching catch).
- `catch` takes **pattern-matching** cases: `case e: ArithmeticException => ...`.
- There are no *checked* exceptions: the compiler never forces you to declare or catch them.
- `finally` always runs. `throw` is an expression of type `Nothing`.

```
val r = try { 10 / d } catch {
  case _: ArithmeticException => 0
  case e: Exception           => -1
} finally println("done")
```

M4-B2. Differentiate `Option`, `Try` and `Either`. [5 marks]

Type	Cases	Use when	Example
`Option[A]`	`Some(a)` / `None`	a value may simply be absent	`map.get("k")`
`Try[A]`	`Success(a)` / `Failure(ex)`	wrapping code that may throw	`Try(s.toInt)`
`Either[E, A]`	`Left(error)` / `Right(a)`	you want your own error value	`safeSqrt(-4)` -> `Left("...")`

All three work with `map`, `flatMap`, `for` comprehensions and pattern matching, so failures propagate without nested `if` or `try`.

M4-B3. Compare Breeze, Saddle and Spire. [3 marks]

Library	Purpose	Main types
Breeze	numerical computing and linear algebra, statistics, optimisation	`DenseVector`, `DenseMatrix`, `*:*`, `\`, `.t`
Saddle	data manipulation with labelled, indexed data and NA handling (like a small pandas)	`Vec`, `Series`, `Frame`, `Index`
Spire	exact and generic numerics	`Rational`, `Algebraic`, `Complex`, typeclasses `Field`, `Ring`

M4-B4. Name the main parts of the Spark architecture and the role of each. [5 marks]

Part	Role
Driver	runs `main()`, creates the SparkSession, turns actions into a DAG of stages and tasks, schedules them, collects results
Cluster manager	allocates resources: Standalone, YARN or Kubernetes
Worker node / Executor	executor is a JVM process that runs tasks and caches partitions in memory or on disk
Task	the smallest unit of work: one stage applied to one partition

Part	Role
Partition	a slice of an RDD or DataFrame; the unit of parallelism

M4-B5. State the four properties of an RDD and explain lineage. [3 marks]

Resilient (recoverable through lineage), **Distributed** (split in partitions across nodes), **Dataset** (typed records), **Immutable** (transformations create new RDDs). **Lineage** is the recorded chain of transformations that produced an RDD, shown by `rdd.toDebugString`. A lost partition is rebuilt by replaying only the needed part of that chain, so no data replication is required.

M4-B6. Give three transformations and three actions of Spark, and explain why laziness helps. [3 marks]

Transformations: `map`, `filter`, `reduceByKey` (also `flatMap`, `groupByKey`, `join`). Actions: `count`, `collect`, `saveAsTextFile` (also `take`, `reduce`, `first`).

Because nothing runs until an action, Spark sees the whole plan first. It can pipeline several narrow steps into one pass over the data, avoid materialising intermediate results, and (for DataFrames) let Catalyst reorder and prune work.

M4-B7. Explain narrow and wide transformations, and what a shuffle is. [3 marks]

Narrow: each output partition depends on one input partition (`map`, `filter`, `flatMap`); pipelined in one stage. **Wide**: an output partition needs data from many input partitions (`reduceByKey`, `groupByKey`, `join`); Spark must **shuffle** (redistribute data by key across the network and disk), which ends a stage. Shuffles are the main cost in Spark jobs, so reduce them (prefer `reduceByKey` over `groupByKey`).

M4-B8. List the Spark ecosystem components and one use of each. [5 marks]

Component	Use
Spark Core	RDD API, task scheduling, memory management, fault recovery
Spark SQL	structured data with DataFrames, Datasets and SQL; Catalyst optimiser
Structured Streaming	process live data (Kafka, sockets, files) with the same DataFrame API
MLlib	scalable machine learning: pipelines, regression, classification, clustering
GraphX	graph computation (PageRank, connected components)

M4-B9. Why is Spark often preferred over Hadoop MapReduce? [3 marks]

- MapReduce writes intermediate results to disk between every map and reduce step. Spark plans a DAG and keeps intermediate data in memory where possible.
- One engine for batch, SQL, streaming, ML and graphs; rich APIs in Scala, Python, Java, R and SQL.
- Iterative algorithms (ML) reuse cached datasets instead of re-reading from disk each iteration.

M4-B10. RDD versus DataFrame. [3 marks]

An RDD is a low-level distributed collection of objects with no schema, so Spark cannot optimise it. A DataFrame is a distributed table with named, typed columns; Catalyst optimises its plan and Tungsten stores it compactly. The Spark quick-start recommends switching to Datasets/DataFrames for better performance, and keeping RDDs for low-level control.

M4-B11. Briefly describe the characteristics of big data (the V's). [2 marks]

Volume (size), Velocity (speed of arrival), Variety (structured, semi-structured, unstructured), commonly extended with Veracity (quality) and Value. Spark is built to process data that is too large or too fast for a single machine.

C. Long answers

M4-C1. Exception handling in Scala [10 marks]

- **Throwing and catching**: `throw new X(msg)`; `catch { case e: X => ... }` uses pattern matching; `finally` always runs. `try` returns a value, so `val r = try ... catch ...` is idiomatic.
- **Custom exception**: `class InsufficientFundsException(msg: String) extends Exception(msg)`.
- **Functional alternatives** keep errors in the type so callers cannot ignore them: `Option` (absence), `Try` (captured exception, with `recover`, `map`, `getOrElse`, `toOption`), `Either` (a typed error in `Left`).
- In a `for` comprehension the first `Left` or `Failure` stops the chain, which replaces nested `try` blocks.

Code: code/m4_q1_exceptions.scala

```scala
// M4-Q1  Exception handling in Scala: try/catch/finally, custom exceptions, Option, Try, Either
import scala.util.{Try, Success, Failure}

class InsufficientFundsException(msg: String) extends Exception(msg)    // custom (checked-style) exception

object M4Q1 extends App {
  // ---- 1. try / catch / finally: catch is a pattern match, try is an expression ----
  def divide(a: Int, b: Int): Int =
    try {
      a / b
    } catch {
      case _: ArithmeticException => println("  caught ArithmeticException"); 0
      case e: Exception           => println("  other: " + e.getMessage); -1
    } finally {
      println(s"  finally runs for divide($a, $b)")
    }
  println("divide(10, 2) = " + divide(10, 2))
  println("divide(10, 0) = " + divide(10, 0))

  // ---- 2. throwing a custom exception ----
  def withdraw(balance: Double, amount: Double): Double = {
    if (amount > balance) throw new InsufficientFundsException(s"balance $balance < $amount")
    balance - amount
  }
  try withdraw(100, 250) catch { case e: InsufficientFundsException => println("custom: " + e.getMessage) }

  // ---- 3. Option: absence of a value without null ----
  val phone = Map("asha" -> "98400", "ravi" -> "98410")
  println("Option  : " + phone.get("asha") + " | " + phone.get("zed") + " | " + phone.get("zed").getOrElse("n/a"))

  // ---- 4. Try: wrap code that may throw, then map / recover ----
  def parse(s: String): Try[Int] = Try(s.trim.toInt)
  for (s <- List("42", "4x2")) parse(s) match {
    case Success(v) => println(s"Try     : '$s' -> Success($v)")
    case Failure(e) => println(s"Try     : '$s' -> Failure(${e.getClass.getSimpleName})")
  }
  println("recover : " + parse("oops").recover { case _: NumberFormatException => -1 }.get)
  println("chain   : " + parse("20").map(_ * 2).filter(_ > 30).toOption)

  // ---- 5. Either: an error value that carries information (Left = error, Right = success) ----
  def safeSqrt(x: Double): Either[String, Double] =
    if (x < 0) Left(s"cannot take sqrt of $x") else Right(math.sqrt(x))
  println("Either  : " + safeSqrt(16) + " | " + safeSqrt(-4))
  println("compose : " + (for { a <- safeSqrt(16); b <- safeSqrt(a) } yield b))
}
```

Output

```
  finally runs for divide(10, 2)
divide(10, 2) = 5
  caught ArithmeticException
  finally runs for divide(10, 0)
divide(10, 0) = 0
custom: balance 100.0 < 250.0
Option  : Some(98400) | None | n/a
Try     : '42' -> Success(42)
Try     : '4x2' -> Failure(NumberFormatException)
recover : -1
chain   : Some(40)
Either  : Right(4.0) | Left(cannot take sqrt of -4.0)
compose : Right(2.0)
```

Tip. In the output the line printed by `finally` appears before `divide(10, 2) = 5`: the `finally` block runs before the function hands its value back to the caller's `println`.

Suggested marks split. try/catch/finally 3, custom exception 1, Option/Try/Either 4, explanation of preference 2.

M4-C2. Scala libraries: Breeze, Saddle and Spire [10 marks]

Library	Lab	What the code shows
Breeze	19	`DenseVector`, element-wise `*:*`, `sum`, `mean`, `variance` (sample variance, divides by n-1), `DenseMatrix`, `\` solve, `dot`
Saddle	18, 20	`Vec` with a boolean mask, `Series` with an `Index`, `Frame` of aligned columns, NA handling in `mean`
Spire	20	`Rational(1,10) + Rational(2,10)` equals exactly 3/10, while `0.1 + 0.2` in `Double` is `0.30000000000000004`
Vegas / Plotly.Scala	18, 19	declarative charts (grammar of graphics) built from data; need a notebook or a browser to view

Code: code/m4_q2_breeze.scala

```
//> using scala 2.13.12
//> using dep org.scalanlp::breeze:2.1.0
//> using dep org.typelevel::spire:0.18.0
// M4-Q2  Breeze (numerical computing, Lab 19) and Spire (exact numerics, Lab 20)
import breeze.linalg._
import breeze.stats.{mean, variance}

object M4Q2 extends App {
  val fruits = Seq("Apple", "Banana", "Cherry", "Mango", "Orange")
  val price = DenseVector(2.50, 1.20, 4.00, 3.50, 2.00)
  val qty = DenseVector(150.0, 300.0, 80.0, 110.0, 200.0)

  val revenue = price *:* qty                                  // element-wise (Hadamard) product
  fruits.indices.foreach(i => println(f"${fruits(i)}%-7s ${price(i)}%.2f x ${qty(i)}%.0f = ${revenue(i)}%.2f"))
  println(f"total revenue = ${sum(revenue)}%.2f, mean price = ${mean(price)}%.3f, variance =
      ${variance(price)}%.4f")

  val A = DenseMatrix((2.0, 1.0), (1.0, 3.0))
  val b = DenseVector(3.0, 5.0)
  val x = A \ b                                                // solve A x = b
  println("A*x = b  ->  x = " + x.map(v => math.round(v * 1000) / 1000.0))
  println("A * A^T  = " + (A * A.t))
  println("dot      = " + (price dot qty))

  // ---- Spire: exact rational arithmetic, no floating-point error ----
  import spire.math.Rational
  val d = 0.1 + 0.2
  val r = Rational(1, 10) + Rational(2, 10)
  println(s"Double 0.1 + 0.2 = $d   | Rational 1/10 + 2/10 = $r  (exactly 3/10: ${r == Rational(3, 10)})")
}
```

Output

```
Apple   2.50 x 150 = 375.00
Banana  1.20 x 300 = 360.00
Cherry  4.00 x 80 = 320.00
Mango   3.50 x 110 = 385.00
Orange  2.00 x 200 = 400.00
total revenue = 1840.00, mean price = 2.640, variance = 1.2730
A*x = b  ->  x = DenseVector(0.8, 1.4)
A * A^T  = 5.0  5.0
5.0  10.0
dot      = 1840.0
Double 0.1 + 0.2 = 0.30000000000000004   | Rational 1/10 + 2/10 = 3/10  (exactly 3/10: true)
```

Code: code/m4_q3_saddle.scala

```
//> using scala 2.13.12
//> using dep io.github.pityka::saddle-core:4.0.0-M14
// M4-Q3  Saddle (maintained Scala 2.13 fork; the original org.scala-saddle artifacts on Maven Central stop at Scala
     2.11): indexed Vec / Series / Frame (Lab experiments 18 and 20, data part only)
// (the Vegas chart in Lab 18 needs a notebook / browser and is not run here)
import org.saddle._
import org.saddle.ops.BinOps._                          // operator instances (Vec * Vec, ...) in Saddle 4.x

object M4Q3 extends App {
  val fruit = Vec("Apple", "Apple", "Apple", "Banana", "Orange", "Apple")
  val price = Vec(2.40, 2.65, 2.50, 1.20, 1.80, 3.10)
  val qty = Vec(120.0, 110.0, 140.0, 250.0, 180.0, 130.0)
  val date = Index("2023-10-01", "2023-10-02", "2023-10-03", "2023-10-01", "2023-10-02", "2023-10-04")

  val isApple = fruit.map(_ == "Apple")                 // boolean mask
  val appleLocs = isApple.find(identity).toArray        // positions where the mask is true
  val applePrice = price.take(appleLocs)
  println("apple prices     : " + applePrice.toSeq.mkString(", "))
  println(f"average apple    : ${applePrice.mean}%.4f")

  // Series = values + an index (labels); here the index is the date
  val s = Series(price, date)
  println("Series at 2023-10-02:\n" + s.filterAt(i => i == 1 || i == 4))
```

```
  // Frame = table of aligned columns
  val frame = Frame("Price" -> price, "Qty" -> qty)
  val revenue = frame.firstCol("Price").toVec * frame.firstCol("Qty").toVec   // vectorised column arithmetic
  println("frame shape      : " + frame.numRows + " rows x " + frame.numCols + " cols")
  println("revenue column   : " + revenue.toSeq.map(v => f"$v%.1f").mkString(", "))

  // missing values are first-class (NA), and statistics skip them
  val withNA = Vec(2.0, Double.NaN, 4.0)                // NaN is Saddle's NA for Double
  println(s"mean with NA     : ${withNA.mean}  (NA skipped), count = ${withNA.count}")
}
```

Output

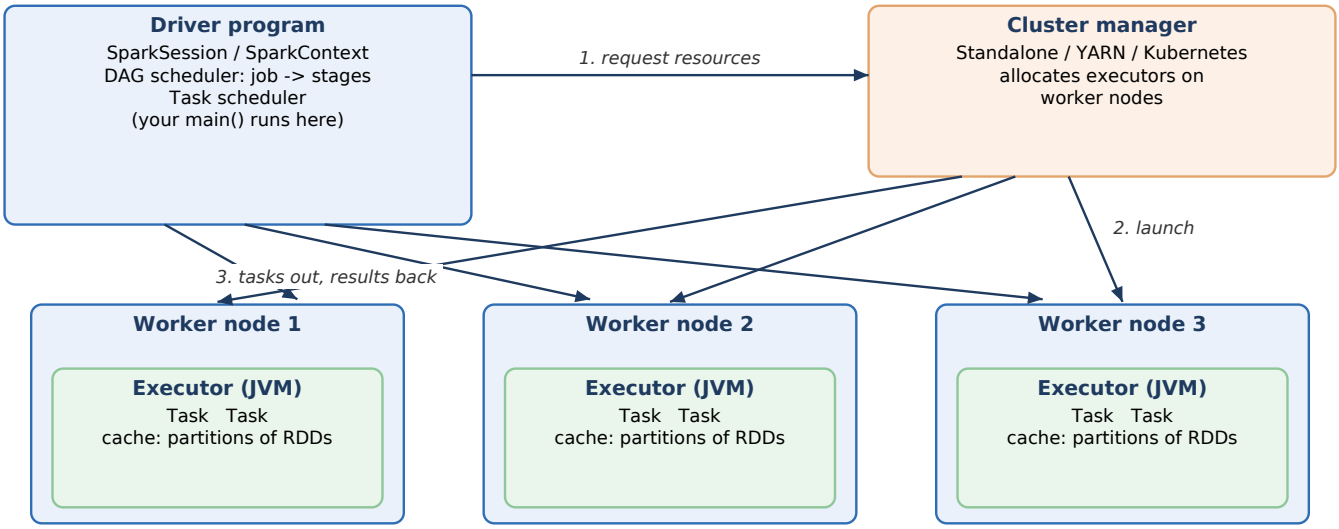
```
apple prices     : 2.4, 2.65, 2.5, 3.1
average apple    : 2.6625
Series at 2023-10-02:
[2 x 1]
2023-10-02 -> 2.6500
2023-10-02 -> 1.8000

frame shape      : 6 rows x 2 cols
revenue column   : 288.0, 291.5, 350.0, 300.0, 324.0, 403.0
mean with NA     : 3.0  (NA skipped), count = 2
```

Note. Breeze printed two start-up messages about the Java version of its native-library loader; they are omitted above. Saddle: the original `org.scala-saddle` artifacts on Maven Central stop at Scala 2.11, so this program uses the maintained fork `io.github.pityka::saddle-core:4.0.0-M14` (a milestone release) on Scala 2.13. In that version `Frame.firstCol(name)` returns a Series and operators need `import org.saddle.ops.BinOps._`; your lab file uses the older `frame.col(...)` style, so check the API of the version you load.

Suggested marks split. Breeze 3, Saddle 3, Spire 2, charts and role comparison 2.

M4-C3. Apache Spark architecture [10 marks]

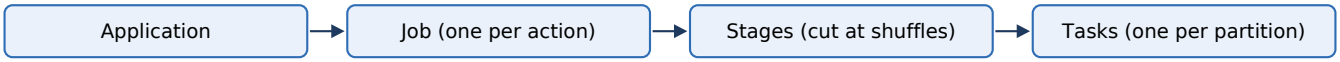


Spark cluster architecture: the driver talks to the cluster manager, executors run tasks and cache data.

Component roles

- **Driver**: your `main()`, the SparkSession, DAG scheduler and task scheduler. It must stay alive for the whole application.
- **Cluster manager**: grants executors (Standalone, YARN, Kubernetes).
- **Executors**: JVMs on worker nodes; run tasks in threads, keep cached partitions, report results back to the driver.

From code to tasks



Execution hierarchy.

- **Job**: created each time an action (`collect`, `count`, `save`) is called.

- **Stage**: a group of narrow transformations that can run without moving data. A wide transformation (`reduceByKey`) starts a new stage.
- **Task**: one stage on one partition. With 2 partitions, each stage has 2 tasks.

In the word count below, `flatMap` and `map` form stage 0; `reduceByKey` needs a shuffle, so the reduce side is stage 1. The `(2)` in `toDebugString` is the number of partitions, and the `+-` marks the shuffle boundary.

Code: m4_q4_spark.scala (first part: session and word count)

```
//> using scala 2.13.12
//> using dep org.apache.spark::spark-sql:3.5.1
//> using javaOpt --add-opens=java.base/java.lang=ALL-UNNAMED ... (13 flags that let Spark 3.5 run on Java 17; see
    the file)
// M4-Q4  Spark in local mode: RDD, lazy transformations vs actions, lineage, DataFrame + Catalyst plan
import org.apache.spark.sql.SparkSession

case class Emp(name: String, dept: String, salary: Double)

object M4Q4 {
  def main(args: Array[String]): Unit = {
    val spark = SparkSession.builder().appName("pds-demo").master("local[2]")
      .config("spark.ui.enabled", "false").config("spark.driver.bindAddress", "127.0.0.1")
      .config("spark.driver.host", "127.0.0.1").getOrCreate()
    val sc = spark.sparkContext
    sc.setLogLevel("ERROR")
    import spark.implicits._

    // ---- RDD word count: transformations build a DAG, the action runs it ----
    val lines = sc.parallelize(Seq("spark is fast", "spark is lazy", "scala runs spark"), 2)
    val counts = lines.flatMap(_.split(" ")).map(w => (w, 1)).reduceByKey(_ + _)
    println("partitions       : " + lines.getNumPartitions)
... (30 more lines omitted)
```

Output (first part: partitions and lineage)

```
partitions       : 2
lineage (DAG)    :
(2) ShuffledRDD[3] at reduceByKey at m4_q4_spark.scala:20 []
 +-(2) MapPartitionsRDD[2] at map at m4_q4_spark.scala:20 []
    |  MapPartitionsRDD[1] at flatMap at m4_q4_spark.scala:20 []
    |  ParallelCollectionRDD[0] at parallelize at m4_q4_spark.scala:19 []
word counts      : List((spark,3), (is,2), (fast,1), (lazy,1), (runs,1), (scala,1))
after map()      : map ran 0 times   (nothing computed yet)
after reduce()   : map ran 10 times, result = 110
```

Note. Run in local mode (`local[2]`, two worker threads in one JVM) with Spark 3.5.1 on Java 17, so the "cluster" is simulated. The architecture and the stage/task logic are the same on a real cluster.

Suggested marks split. diagram 3, component roles 3, job-stage-task 3, example 1.

M4-C4. RDDs, lineage, transformations and actions [10 marks]

Property	Meaning
Resilient	lost partitions are recomputed from lineage
Distributed	data is split into partitions spread over executors
Dataset	collection of typed records
Immutable	never modified; every transformation produces a new RDD

Lineage and fault tolerance

Every RDD stores a reference to its parents and the function applied. If an executor dies, the driver reschedules only the lost partitions on other executors, which re-read the source data and replay the transformations. Nothing was copied in advance, so there is no replication overhead during normal running; the price is recomputation time after a failure. `persist`/`cache` avoid recomputing an RDD that is used again, and checkpointing writes it to reliable storage so a very long lineage can be cut.

Transformations (lazy) versus actions (eager)

	Transformations	Actions
Returns	a new RDD	a value to the driver, or writes to storage
Executes	no, only extends the lineage	yes, triggers a job
Examples	`map`, `filter`, `flatMap`, `reduceByKey`	`count`, `collect`, `take`, `reduce`, `saveAsTextFile`

Proof of laziness in the output below: after calling `map`, the accumulator that counts executions of the function still reads 0; after the action `reduce`, it reads 10.

Excerpt from m4_q4_spark.scala (laziness demo)

```
    // ---- Laziness: an accumulator counts how often the map function really runs ----
    val calls = sc.longAccumulator("map calls")
    val doubled = sc.parallelize(1 to 10).map { x => calls.add(1); x * 2 }     // transformation only
    println("after map()      : map ran " + calls.value + " times   (nothing computed yet)")
    val total = doubled.reduce(_ + _)                                          // action
    println("after reduce()   : map ran " + calls.value + " times, result = " + total)
```

Relevant lines of the output of m4_q4_spark.scala (listed in the previous question)

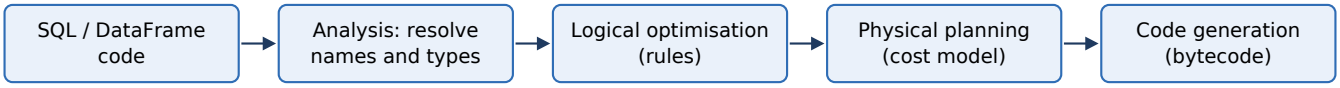
```
word counts      : List((spark,3), (is,2), (fast,1), (lazy,1), (runs,1), (scala,1))
after map()      : map ran 0 times   (nothing computed yet)
after reduce()   : map ran 10 times, result = 110
```

Suggested marks split. four properties 2, lineage 3, transformations vs actions with examples 3, laziness demo 2.

M4-C5. Spark components and the Catalyst optimiser [10 marks]

Component	Purpose
Spark Core	RDDs, scheduling, memory, fault tolerance
Spark SQL	DataFrames/Datasets, SQL, Catalyst
Structured Streaming	incremental processing of live data with the DataFrame API
MLlib	machine learning algorithms and pipelines
GraphX	graph processing

Catalyst



Phases of Catalyst.

1. **Analysis**: unresolved column and table names are resolved against the catalog; types are checked.
2. **Logical optimisation**: rule-based rewrites such as predicate pushdown, column pruning, constant folding and combining filters.
3. **Physical planning**: one or more physical plans are generated (for example broadcast hash join versus sort-merge join) and the cheapest is chosen.
4. **Code generation**: whole-stage code generation compiles the chosen plan into Java bytecode that avoids virtual calls and per-row overhead.

In the second query the code has two separate `filter` calls and a multiplication by a literal. After Catalyst, the two filters are merged into one `(id > 10) AND (id < 50)` and `cast(5 as bigint)` has been folded into the literal `5`. The first query, on a small in-memory table, shows no `Filter` node in its optimised plan although the code filters `salary > 68000`; the result still excludes Kabir (65000), so Catalyst applied the predicate to the local data while optimising.

Excerpt from m4_q4_spark.scala (DataFrame and Catalyst demo)

```
    // ---- DataFrame + Catalyst: look at the plans ----
    val df = Seq(Emp("Asha", "AI", 90000), Emp("Ravi", "DB", 70000),
                 Emp("Meena", "AI", 110000), Emp("Kabir", "DB", 65000)).toDF()
    val q = df.filter($"salary" > 68000).groupBy("dept").avg("salary").orderBy("dept")
    println("\nresult:")
    q.show()
    println("optimised logical plan and physical plan (excerpt):")
    val plan = q.queryExecution
    println(plan.optimizedPlan.toString.linesIterator.map("  " + _).mkString("\n"))
    println(plan.executedPlan.toString.linesIterator.map("  " + _).mkString("\n"))

    // ---- Catalyst rules visible: two Filters merge into one; cast(5 as bigint) folds to 5 ----
    val r = spark.range(100).filter($"id" > 10).filter($"id" < 50).select(($"id" * (2 + 3)).as("x"))
    val show = (s: String) => s.linesIterator.map("  " + _).mkString("\n")
    println("\nanalysed plan (as written):\n" + show(r.queryExecution.analyzed.toString))
    println("optimised plan (after Catalyst):\n" + show(r.queryExecution.optimizedPlan.toString))
    println("rows kept: " + r.count())
```

Plans printed by m4_q4_spark.scala

```
result:
+----+-----------+
|dept|avg(salary)|
+----+-----------+
|  AI|   100000.0|
|  DB|    70000.0|
+----+-----------+

optimised logical plan and physical plan (excerpt):
  Sort [dept#4 ASC NULLS FIRST], true
  +- Aggregate [dept#4], [dept#4, avg(salary#5) AS avg(salary)#17]
     +- LocalRelation [dept#4, salary#5]
  AdaptiveSparkPlan isFinalPlan=false
  +- Sort [dept#4 ASC NULLS FIRST], true, 0
     +- Exchange rangepartitioning(dept#4 ASC NULLS FIRST, 200), ENSURE_REQUIREMENTS, [plan_id=107]
        +- HashAggregate(keys=[dept#4], functions=[avg(salary#5)], output=[dept#4, avg(salary)#17])
           +- Exchange hashpartitioning(dept#4, 200), ENSURE_REQUIREMENTS, [plan_id=104]
              +- HashAggregate(keys=[dept#4], functions=[partial_avg(salary#5)], output=[dept#4, sum#28, count#29L])
                 +- LocalTableScan [dept#4, salary#5]

analysed plan (as written):
  Project [(id#39L * cast(5 as bigint)) AS x#45L]
  +- Filter (id#39L < cast(50 as bigint))
     +- Filter (id#39L > cast(10 as bigint))
        +- Range (0, 100, step=1, splits=Some(2))
optimised plan (after Catalyst):
  Project [(id#39L * 5) AS x#45L]
  +- Filter ((id#39L > 10) AND (id#39L < 50))
     +- Range (0, 100, step=1, splits=Some(2))
rows kept: 39
```

Tip. Catalyst is written in Scala and relies on pattern matching over tree nodes, which is one reason Scala features in this course.

Suggested marks split. components 3, Catalyst phases 4, example 3.

Module 5: Programming language selection criteria

Syllabus (6 hours): size of the deployment (data, resource and load); security; skill set; targeted platform; elasticity of a language; time to production; performance; support and community; purpose; programmer experience; ease of development and maintenance; efficiency; availability of an IDE; error checking and diagnosis.

Your FAT notes group the same ideas into **13 criteria** (for example IDE and diagnostics together); the syllabus names about fourteen. Use the syllabus wording in answers and mention the grouping if the question says "13".

Textbooks for this module are not specific; the reference listed in the syllabus is an article on factors to consider when choosing a programming language (see the resources section).

> **Tip.** Never answer "which language is best". Answer "best for this project, because criteria X and Y outweigh Z". Quote facts from the scenario in each justification.

Module 5: Test yourself (questions only)

A. Multiple choice (8 questions, 1 mark each)

M5-A1. A team of data scientists who already know Python must deliver an ML prototype in four weeks. Which criteria weigh most?

(a) performance and security only
(b) time to production and skill set
(c) IDE availability only
(d) elasticity only

M5-A2. Static typing mainly helps a large codebase because:

(a) it makes programs shorter
(b) many type errors are caught at compile time and refactoring is safer
(c) it removes the need for tests
(d) it makes the program run on any platform

M5-A3. Apache Spark itself is mainly written in:

(a) Python
(b) R
(c) Scala
(d) C#

M5-A4. "Elasticity of a language" refers to:

(a) how quickly it compiles
(b) how well it covers small scripts up to large systems and adapts to changing needs
(c) the price of the licence
(d) the number of keywords

M5-A5. For a pure CPU-bound loop written in the language itself, which is typically fastest?

(a) CPython
(b) R
(c) Scala/Java on the JVM (JIT compiled)
(d) a shell script

M5-A6. A weighted decision matrix is used to:

(a) measure code coverage
(b) score each option against weighted criteria and compare totals
(c) choose a database index
(d) estimate cloud cost exactly

M5-A7. Which tool is the usual IDE choice for Scala development?

(a) RStudio
(b) IntelliJ IDEA with the Scala plugin
(c) Xcode only
(d) Notepad only

M5-A8. Python numerical code is fast mostly because:

(a) the interpreter is the fastest
(b) heavy work runs in compiled C/C++/Fortran libraries (NumPy, SciPy)
(c) it has no garbage collection
(d) it is statically typed

B. Short answers

M5-B1. Explain "size of the deployment": data, resource and load. [3 marks]

M5-B2. Why is security a language-selection criterion? [3 marks]

M5-B3. How do skill set and community support influence the choice? [3 marks]

M5-B4. Discuss the trade-off between time to production and performance. [3 marks]

M5-B5. What is meant by the elasticity of a language? [2 marks]

M5-B6. Explain the importance of IDE availability and error checking or diagnosis. [3 marks]

M5-B7. Compare static and dynamic typing for maintenance. [3 marks]

M5-B8. List five criteria that favour Python and five that favour Scala. [5 marks]

C. Long questions

M5-C1. Python versus Scala versus Java versus R [10 marks]

(FAT high-frequency question 7) Compare Python, Scala, Java and R across the language-selection criteria in the syllabus and give an architectural justification for when to use each.

M5-C2. Language selection case studies with a decision matrix [10 marks]

Case A. A bank needs a real-time fraud-detection pipeline that processes about 5 TB of transactions per day on a Spark cluster. The team has eight experienced JVM engineers. **Case B.** A start-up with three data scientists must deliver an ML prototype in six weeks using small CSV and Parquet files. For each case choose a language and justify it with a weighted decision matrix.

Module 5: Answers

A. Multiple-choice key

No.	Ans.	Why
M5-A1	b	Short deadline plus existing skills favour the language the team already knows, with a rich ML ecosystem (Python).
M5-A2	b	Compile-time checking supports maintenance and error diagnosis. It does not replace tests.
M5-A3	c	Scala (running on the JVM). Python, R, Java and SQL are language APIs on top.
M5-A4	b	Can the same language serve a quick script, a service and a distributed pipeline?
M5-A5	c	JIT-compiled JVM code beats the CPython interpreter. Python code is fast only when it calls compiled libraries such as NumPy.
M5-A6	b	Weights reflect the project's priorities; the highest weighted total wins, and the result should be sanity-checked.
M5-A7	b	VS Code with Metals is the other common option.
M5-A8	b	Vectorised calls move the loop out of the interpreter.

B. Short answers

M5-B1. Explain "size of the deployment": data, resource and load. [3 marks]

- **Data**: gigabytes on a laptop (pandas) versus terabytes across a cluster (Spark).
- **Resource**: memory, CPU and budget available; a JVM service needs heap tuning, a script may need none.
- **Load**: number of concurrent users or events per second; high concurrency favours the JVM (Scala, Java) and its mature concurrency libraries.

M5-B2. Why is security a language-selection criterion? [3 marks]

It decides how easily you can meet requirements such as authentication, encryption, input validation and audit. Look at the maturity of security libraries, memory safety, how dependencies are vetted (supply-chain risk), patch frequency and available expertise. Static typing and sandboxing help, but the application design matters more than the language alone.

M5-B3. How do skill set and community support influence the choice? [3 marks]

A language the team already knows shortens delivery and reduces defects. A large community means more libraries, documentation, answered questions, and easier hiring (Python has the largest data-science pool; Scala has fewer but specialised data engineers). Check also how actively the key libraries are maintained.

M5-B4. Discuss the trade-off between time to production and performance. [3 marks]

Python gets a working version out quickly, which matters while requirements are uncertain. Scala/Java need more up-front effort but run faster and scale better on a JVM cluster. A common strategy: prototype in Python, then move the performance-critical path to Scala/Java (or use PySpark DataFrames, which execute in the JVM engine, and avoid slow Python UDFs).

M5-B5. What is meant by the elasticity of a language? [2 marks]

Its ability to stretch from a short exploratory script to a large, long-lived production system, and to adapt as requirements change. Scala (concise scripts to large typed systems) and Python (with typing and packaging tools) score well; a language only good for one-off analysis scores lower.

M5-B6. Explain the importance of IDE availability and error checking or diagnosis. [3 marks]

- IDE: code completion, refactoring, debugging, profiling and test integration raise productivity (IntelliJ IDEA, PyCharm, VS Code, RStudio).
- Error checking: compile-time checks (Scala, Java) catch type errors before running; dynamic languages find them at run time, so they need tests, linters and type hints.
- Diagnosis: quality of stack traces, debuggers and logging decides how fast production incidents are fixed.

M5-B7. Compare static and dynamic typing for maintenance. [3 marks]

	Static (Scala, Java)	Dynamic (Python, R)
Type errors found	at compile time	at run time
Refactoring large code	safer, compiler guides you	needs good tests

	Static (Scala, Java)	Dynamic (Python, R)
Speed of writing	more annotations (less in Scala via inference)	faster to write
Tooling	precise completion and navigation	good with type hints, weaker without

M5-B8. List five criteria that favour Python and five that favour Scala. [5 marks]

Favours Python	Favours Scala
Time to production	Performance on CPU-bound work
Large talent pool	Big-data deployment size and load
ML and data-science ecosystem	Native fit with Spark and the JVM
Gentle learning curve and REPL/notebooks	Static typing for maintenance of large code
Huge community (PyPI)	Concurrency and streaming pipelines

C. Long answers

M5-C1. Python versus Scala versus Java versus R [10 marks]

Judgements below are typical, not absolute; always tie your answer to a stated project.

Criterion	What you ask	Python	Scala	Java	R
Size of deployment (data, resource, load)	how much data, how many users, what hardware	small to medium; scales via Spark/Dask	very large, high concurrency	very large, high concurrency	memory-bound, small to medium
Security	libraries, memory safety, vetting	good; vet dependencies	JVM security stack	mature enterprise stack	limited for services
Skill set	what the team knows	largest pool	smaller, specialised	large pool	statisticians
Targeted platform	where it runs	anywhere, notebooks, cloud	JVM, Spark clusters	JVM everywhere	analysis workstation, Shiny
Elasticity	script to large system	good	very good	good, verbose	mostly analysis
Time to production	delivery speed	fastest	slower	slower	fast for analysis
Performance	CPU-bound speed	slow alone, fast via C libraries	fast (JIT)	fast (JIT)	slow alone
Support and community	libraries, answers	PyPI, huge	smaller, strong in data engineering	Maven, huge	CRAN, strong in statistics
Purpose	ML, ETL, web, stats	ML, analysis, glue	Spark jobs, streaming	enterprise services	statistical modelling, reports
Programmer experience	readability, REPL	very good	expressive, steeper	verbose	good for statisticians
Ease of maintenance	refactoring, typing	needs tests/type hints	strong typing	strong typing	weaker
Efficiency	memory, CPU	overhead per object	JVM tuned	JVM tuned	copies data often
IDE availability	tooling	PyCharm, VS Code, notebooks	IntelliJ, Metals	IntelliJ, Eclipse	RStudio
Error checking and diagnosis	compile vs run time	run time, linters	compile time	compile time	run time

When to use each

- **Python**: exploratory analysis, machine learning, quick prototypes, glue code, teams with mixed backgrounds.
- **Scala**: Spark jobs, streaming and batch pipelines, high-throughput data services, large codebases that need strong typing and functional style.
- **Java**: enterprise back-ends and integration with existing JVM systems; broad hiring pool.
- **R**: statistical modelling, publication-quality graphics, reports in domains with strong R packages.

Suggested marks split. criteria covered (at least 8) 5, comparison across languages 3, when to use each 2.

M5-C2. Language selection case studies with a decision matrix [10 marks]

Method: (1) list the criteria that matter, (2) give each a weight so that weights add up to 1, (3) score each candidate from 1 to 5, (4) multiply and add, (5) compare totals and sanity-check the winner against the case facts.

Criterion	Python	Scala	Java	R	Weight A	Weight B
Deployment size / load	3	5	5	2	0.25	0.05
Performance	2	5	5	2	0.20	0.05
Time to production	5	3	3	4	0.05	0.30
Skill set available	5	2	4	2	0.10	0.25
Ecosystem / community	5	4	5	3	0.05	0.20
Maintainability	3	5	4	2	0.15	0.05
Platform fit (Spark/JVM)	3	5	4	1	0.20	0.10

Example calculation, Case A, Scala: 0.25x5 + 0.20x5 + 0.05x3 + 0.10x2 + 0.05x4 + 0.15x5 + 0.20x5 = 1.25 + 1.00 + 0.15 + 0.20 + 0.20 + 0.75 + 1.00 = **4.55**.

Code: code/m5_decision_matrix.py

```
"""M5  Weighted decision matrix for language selection (two case studies).
Scores are 1-5 judgement values chosen by the analyst - the point is the method.
"""
CRITERIA = ["Deployment size / load", "Performance", "Time to production", "Skill set available",
            "Ecosystem / community", "Maintainability", "Platform fit (Spark/JVM)"]
SCORES = {                      # criterion order as above
    "Python": [3, 2, 5, 5, 5, 3, 3],
    "Scala":  [5, 5, 3, 2, 4, 5, 5],
    "Java":   [5, 5, 3, 4, 5, 4, 4],
    "R":      [2, 2, 4, 2, 3, 2, 1],
}
CASES = {
    "Case A: real-time fraud detection, 5 TB/day, Spark cluster, 8 experienced JVM engineers":
        [0.25, 0.20, 0.05, 0.10, 0.05, 0.15, 0.20],
    "Case B: 6-week ML prototype for a startup, 3 data scientists, small CSV/Parquet data":
        [0.05, 0.05, 0.30, 0.25, 0.20, 0.05, 0.10],
}
for case, weights in CASES.items():
    assert abs(sum(weights) - 1.0) < 1e-9
    print(case)
    print("  weights:", dict(zip(CRITERIA, weights)))
    ranking = sorted(((sum(w * s for w, s in zip(weights, sc)), lang) for lang, sc in SCORES.items()),
                     reverse=True)
    for total, lang in ranking:
        print(f"    {lang:<7} weighted score = {total:.2f}")
    print("  -> choose", ranking[0][1], "\n")
```

Output

```
Case A: real-time fraud detection, 5 TB/day, Spark cluster, 8 experienced JVM engineers
  weights: {'Deployment size / load': 0.25, 'Performance': 0.2, 'Time to production': 0.05, 'Skill set available':
      0.1, 'Ecosystem / community': 0.05, 'Maintainability': 0.15, 'Platform fit (Spark/JVM)': 0.2}
    Scala   weighted score = 4.55
    Java    weighted score = 4.45
    Python  weighted score = 3.20
    R       weighted score = 1.95
  -> choose Scala

Case B: 6-week ML prototype for a startup, 3 data scientists, small CSV/Parquet data
  weights: {'Deployment size / load': 0.05, 'Performance': 0.05, 'Time to production': 0.3, 'Skill set available':
      0.25, 'Ecosystem / community': 0.2, 'Maintainability': 0.05, 'Platform fit (Spark/JVM)': 0.1}
    Python  weighted score = 4.45
    Java    weighted score = 4.00
    Scala   weighted score = 3.45
    R       weighted score = 2.70
  -> choose Python
```

Conclusions

- **Case A -> Scala** (4.55; Java is close at 4.45). Throughput, concurrency, native Spark integration and strong typing dominate, and the team already knows the JVM, so Scala or Java is justified; Python scores 3.20 mainly because of performance and platform fit.
- **Case B -> Python** (4.45). Delivery speed, the available skills and the ML ecosystem dominate. Scala scores 3.45 because its slower start and smaller talent pool hurt in six weeks.
- The scores are the analyst's judgement; the value of the matrix is that it makes the reasoning explicit and lets stakeholders change a weight and see the effect. If Case B later grows to terabytes, revisit the weights: the

decision can change.

Suggested marks split. method and weights 3, scoring and arithmetic 3, justification of each choice 3, caveats 1.

Module 6: Contemporary issues

Syllabus (2 hours): no topics are listed in the syllabus.

> **Watch out.** The syllabus gives Module 6 no topic list, so the questions here cover themes that normally appear under "contemporary issues" (privacy and security, bias, generative AI, drift). Ask your faculty which topics were taught and drop or replace these accordingly.

Module 6: Test yourself (questions only)

A. Multiple choice (5 questions, 1 mark each)

M6-A1. The GDPR principle of "data minimisation" means:

(a) store everything in case it is useful
(b) collect and keep only the data needed for the stated purpose
(c) encrypt all data twice
(d) delete data after one day

M6-A2. A model trained on historically biased hiring data is most likely to:

(a) remove the bias automatically
(b) reproduce or amplify the bias in its predictions
(c) refuse to predict
(d) become more accurate for every group

M6-A3. An LLM "hallucination" is:

(a) a slow response
(b) a fluent but incorrect or invented answer
(c) a syntax error
(d) a network timeout

M6-A4. Pseudonymised data differs from anonymised data because:

(a) it is public
(b) it can be linked back to a person using extra information that is kept separately
(c) it is encrypted
(d) it has fewer rows

M6-A5. "Data drift" after deployment means:

(a) the file moves to another disk
(b) the statistics of incoming data change, so the model's accuracy may fall
(c) the schema is deleted
(d) Spark loses an executor

B. Short answers

M6-B1. List practices for responsible handling of personal data in a data-science pipeline. [5 marks]

M6-B2. Where does bias in a machine-learning system come from, and how can it be reduced? [5 marks]

M6-B3. What are the benefits and risks of using an AI coding assistant for data-science programming? [3 marks]

M6-B4. What is model or data drift and how do you handle it? [3 marks]

C. Long questions

M6-C1. Responsible data science: a hospital pipeline [10 marks]

A hospital wants to predict patient re-admission using ten years of records (about 4 TB) on a Spark cluster. Discuss the privacy, security and fairness issues, and justify the language(s) you would use for the pipeline and the modelling.

M6-C2. AI coding assistants in data-science work [10 marks]

Discuss how AI coding assistants change the way data-science programs are written, and give guidelines for using them responsibly in a university course.

Module 6: Answers

A. Multiple-choice key

No.	Ans.	Why
M6-A1	b	Less data collected means less risk and easier compliance.
M6-A2	b	Models learn patterns in the data, including unfair ones.
M6-A3	b	Generated code and facts must be verified, never trusted blindly.
M6-A4	b	Truly anonymised data cannot be re-identified; pseudonymised data usually still counts as personal data under privacy laws.
M6-A5	b	Monitor input distributions and model metrics, and retrain when needed.

B. Short answers

M6-B1. List practices for responsible handling of personal data in a data-science pipeline. [5 marks]

- Lawful basis and consent; collect the minimum data for the purpose.
- Access control (least privilege), encryption at rest and in transit, audit logs.
- Pseudonymise or anonymise before analysis; keep keys separate.
- Retention limits and deletion on request.
- Document data lineage so you can show where each field came from and who used it.

Laws to know by name: the EU General Data Protection Regulation (GDPR) and India's Digital Personal Data Protection Act, 2023.

M6-B2. Where does bias in a machine-learning system come from, and how can it be reduced? [5 marks]

Source	Mitigation
Unrepresentative or historically biased training data	audit and rebalance data; collect missing groups
Proxy features that encode sensitive attributes (such as a postcode)	feature review; remove or constrain proxies
Labels created by biased people	clear labelling guidelines; multiple annotators
Evaluation only on the overall average	report metrics per group; set fairness checks before release

M6-B3. What are the benefits and risks of using an AI coding assistant for data-science programming? [3 marks]

Benefits: faster boilerplate, explanations, test generation. Risks: plausible but wrong code (hallucination), insecure or outdated API usage, licence and data-leakage concerns when pasting private data. Good practice: read and run what you accept, write tests, never paste secrets or personal data, and be able to explain every line (the viva will ask).

M6-B4. What is model or data drift and how do you handle it? [3 marks]

Drift is a change after deployment in the input data (data drift) or in the relationship between inputs and target (concept drift). Handle it by monitoring input statistics and model metrics, alerting on thresholds, and retraining with recent data on a schedule or trigger.

C. Long answers

M6-C1. Responsible data science: a hospital pipeline [10 marks]

Concern	What to do
Privacy	obtain the lawful basis and ethics approval; remove direct identifiers; pseudonymise with keys kept apart; minimise the fields used
Security	role-based access, encryption at rest and in transit, audit logging, isolated cluster network, no personal data in notebooks or chat tools
Fairness	check the data covers age, gender and region groups; evaluate error rates per group; review proxy features
Quality and drift	validate schemas on ingestion; monitor drift after deployment
Language choice	Scala (or PySpark DataFrames) for the 4 TB ETL on Spark; Python with scikit-learn or MLlib for modelling because of the ML ecosystem and the data-science team's skills

A short, structured answer that names the risk, the mitigation and links the language choice to the Module 5 criteria earns the marks.

Suggested marks split. privacy 3, security 2, fairness 2, language justification 3.

M6-C2. AI coding assistants in data-science work [10 marks]

- **Changes**: faster drafts of data loading, cleaning and plotting code; quick explanation of unfamiliar APIs; more time for design and interpretation.
- **Risks**: wrong or hallucinated code, subtle statistical mistakes, insecure patterns, over-reliance that weakens understanding, and privacy issues.
- **Guidelines**: treat output as a draft; run it on small data and check results by hand; add tests; do not share personal or confidential data; cite assistance where the course requires it; follow the academic-integrity policy of your institute.

Link to the course: the lab experiments are designed so you understand the steps (for example the Scala combinators or the pandas merge). If you cannot explain a line the assistant produced, you do not yet own it.

Suggested marks split. changes 3, risks 3, guidelines 3, link to course 1.

Self-scorecard

Write the marks you scored after checking your answers. Aim for at least 60 percent in every module before the exam; redo the questions you missed after one day and one week.

Module	MCQ	Short	Long	All	My score	%
1: Python data structures, OOP, APIs and NLP	/ 10	/ 34	/ 60	/ 104		
2: Python libraries for data science	/ 10	/ 33	/ 60	/ 103		
3: Scala data structures and object-oriented programming	/ 11	/ 32	/ 50	/ 93		
4: Scala libraries and Spark basics	/ 11	/ 40	/ 50	/ 101		
5: Programming language selection criteria	/ 8	/ 25	/ 20	/ 53		
6: Contemporary issues	/ 5	/ 16	/ 20	/ 41		
Total				/ 495		

Mock paper (100 marks, 3 hours)

This paper samples every module using questions from the book. It is my own format, not the official pattern; adjust it once you have seen a past paper. Attempt it closed-book, then check against the module answers.

Part A: multiple choice (20 questions, 1 mark each)

1. [M1-A1] Which built-in structure is the best choice to test whether a roll number is among 10 million stored roll numbers?
(a) list (b) tuple (c) set (d) str

2. [M1-A2] `heapq.heappop(h)` on a list `h` managed by heapq returns:
(a) the largest item (b) the smallest item (c) the most recently pushed item (d) the oldest item

3. [M1-A3] `d = {'a': 1}; d.get('z', 0)` evaluates to:
(a) KeyError (b) None (c) 0 (d) 'z'

4. [M1-A4] Which statement about `finally` is correct?
(a) It runs only if an exception occurs (b) It runs only if no exception occurs (c) It runs whether or not an exception occurs (d) It replaces `except`

5. [M2-A1] For a DataFrame with the default integer index, `df.loc[1:3]` and `df.iloc[1:3]` return respectively:
(a) 3 rows and 3 rows (b) 3 rows and 2 rows (c) 2 rows and 3 rows (d) 2 rows and 2 rows

6. [M2-A2] Which array can be broadcast against an array of shape (3, 4) in an addition?
(a) shape (3,) (b) shape (4,) (c) shape (2, 4) (d) shape (3, 2)

7. [M2-A3] `pd.merge(a, b, on='k', how='left')` returns:
(a) only keys present in both (b) all rows of `a`, with NaN where `b` has no match (c) all rows of `b`, with NaN where `a` has no match (d) the Cartesian product

8. [M2-A4] `df.groupby('city')['sales'].sum()` is an example of:
(a) map-reduce on disk (b) split-apply-combine (c) broadcasting (d) one-hot encoding

9. [M3-A1] What happens with `val x = 5; x = 6` in Scala?
(a) x becomes 6 (b) compile-time error: reassignment to val (c) runtime exception (d) x stays 5 silently

10. [M3-A2] Which expression creates a new List with 0 added at the front of `List(1, 2)`?
(a) `List(1, 2) :+ 0` (b) `0 :: List(1, 2)` (c) `List(1, 2).add(0)` (d) `List(1, 2)(0) = 0`

11. [M3-A3] Which of these is NOT generated automatically for a case class?
(a) `toString` (b) `copy` (c) structural `equals` (d) a `main` method

12. [M3-A4] `Student("Rohan", 101)` (no `new`) works for a normal class when:
(a) the class is final (b) its companion object defines `apply` (c) the class is abstract (d) the class has a private field

13. [M4-A1] Which Spark process builds the DAG of stages and schedules tasks on executors?
(a) Executor (b) Driver (c) Worker disk (d) Cluster manager only

14. [M4-A2] Which of these is an action?
(a) `map` (b) `filter` (c) `reduceByKey` (d) `count`

15. [M4-A3] RDD stands for:
(a) Random Data Directory (b) Resilient Distributed Dataset (c) Relational Data Document (d) Remote Data

Driver

16. [M4-A4] After an executor fails, a lost RDD partition is recovered by:
(a) restoring a replica from HDFS always (b) recomputing it from its lineage (the DAG of transformations) (c) restarting the whole application (d) asking the user

17. [M5-A1] A team of data scientists who already know Python must deliver an ML prototype in four weeks. Which criteria weigh most?
(a) performance and security only (b) time to production and skill set (c) IDE availability only (d) elasticity only

18. [M5-A2] Static typing mainly helps a large codebase because:
(a) it makes programs shorter (b) many type errors are caught at compile time and refactoring is safer (c) it removes the need for tests (d) it makes the program run on any platform

19. [M6-A1] The GDPR principle of "data minimisation" means:
(a) store everything in case it is useful (b) collect and keep only the data needed for the stated purpose (c) encrypt all data twice (d) delete data after one day

20. [M6-A2] A model trained on historically biased hiring data is most likely to:
(a) remove the bias automatically (b) reproduce or amplify the bias in its predictions (c) refuse to predict (d) become more accurate for every group

Part B: short answers (10 questions, 3 marks each)

21. [M1-B1] Differentiate between a list and a tuple. When would you prefer a tuple? (3 marks)

22. [M1-B5] Compare HTTP GET and POST. (3 marks)

23. [M2-B3] Distinguish `.loc`, `.iloc` and boolean filtering with examples. (3 marks)

24. [M2-B6] How do you work with text data in a DataFrame? Give four `.str` operations. (3 marks)

25. [M3-B1] Explain `val`, `var`, `lazy val` and type inference. (3 marks)

26. [M3-B6] Differentiate method overloading and method overriding. (3 marks)

27. [M4-B6] Give three transformations and three actions of Spark, and explain why laziness helps. (3 marks)

28. [M4-B7] Explain narrow and wide transformations, and what a shuffle is. (3 marks)

29. [M5-B1] Explain "size of the deployment": data, resource and load. (3 marks)

30. [M6-B3] What are the benefits and risks of using an AI coding assistant for data-science programming? (3 marks)

Part C: long questions (5 questions, 10 marks each; full statements are in Part 1 of the module)

31. [M1-C2] Text editor with undo and redo (10 marks)

32. [M2-C2] Split-apply-combine and merging DataFrames (10 marks)

33. [M3-C4] Traits, mixins and linearization (10 marks)

34. [M4-C4] RDDs, lineage, transformations and actions (10 marks)

35. [M5-C2] Language selection case studies with a decision matrix (10 marks)

Study material

Each link below returned HTTP 200 when checked on 7 November 2026. Official documentation comes first because it is the most reliable and its terms match what examiners use.

1. Books already in your folder, mapped to modules

Open the PDFs in *Prog For Data Science/books*. Chapter and page numbers below were read from each file's contents list.

Module	Book (file in your folder)	Read
1	*Python for Data Analysis* (Python-for-Data-Analysis.pdf)	Ch. 3 Built-in data structures, functions and files (p. 65); Ch. 2 language basics (p. 35)
2	*Python for Data Analysis*	Ch. 4 NumPy (p. 101); Ch. 5 pandas (p. 141); Ch. 7 cleaning (p. 221); Ch. 8 join, combine, reshape (p. 265); Ch. 9 plotting (p. 299); Ch. 10 groupby (p. 337); Ch. 12 modelling libraries (p. 423)
2	*Introduction to Machine Learning with Python*	Ch. 3 unsupervised learning and preprocessing: PCA, k-means (p. 145); Ch. 7 text data (p. 337)
2	*Become a Python Data Analyst* (books/Become-a-Python-Data-AnalystV2B-master.zip)	the syllabus textbook 1; unzip and read the chapters on pandas and visualisation
3, 4	*Data Engineering with Scala and Spark* (Packt, 2024)	Ch. 1 Scala essentials for data engineers (p. 20); Ch. 3 Spark and its APIs (p. 80)
4	*Learning Spark* (Lightning-Fast Big Data Analysis)	Ch. 3 programming with RDDs (p. 41); Ch. 7 running on a cluster (p. 135); Ch. 9 Spark SQL (p. 179)
4	*Spark in Action*	Ch. 1 introduction (p. 31); Ch. 2 fundamentals (p. 46); Ch. 4 the Spark API in depth (p. 94); Ch. 5 Spark SQL (p. 133)
2	preprints202312.0957.v1.pdf	a 2023 preprint on supervised machine-learning workflows with numerical and categorical data: background for the modelling part

Note. The Learning Spark and Spark in Action files in your folder look like earlier editions than the ones named in the syllabus (Learning Spark 2nd edition; Spark in Action 2nd edition): their chapters on DStreams, Mesos and H2O are from the first editions. Concepts are the same, page numbers will not match, and the newer editions put DataFrames first.

2. Official documentation by topic

Topic	Link
Python data structures, heapq, deque	https://docs.python.org/3/library/heapq.html https://docs.python.org/3/library/collections.html
Python exceptions	https://docs.python.org/3/tutorial/errors.html
json and csv modules	https://docs.python.org/3/library/json.html https://docs.python.org/3/library/csv.html
Requests (HTTP)	https://requests.readthedocs.io/en/latest/
NLTK book (tokenizing, stopwords, text)	https://www.nltk.org/book/ch03.html
ChatterBot	https://docs.chatterbot.us/ https://github.com/gunthercox/ChatterBot
NumPy broadcasting	https://numpy.org/doc/stable/user/basics.broadcasting.html
pandas: merge/join/concat, groupby, missing data, text	https://pandas.pydata.org/docs/user_guide/merging.html https://pandas.pydata.org/docs/user_guide/groupby.html https://pandas.pydata.org/docs/user_guide/missing_data.html https://pandas.pydata.org/docs/user_guide/text.html
Beautiful Soup, Scrapy	https://www.crummy.com/software/BeautifulSoup/bs4/doc/ https://docs.scrapy.org/en/latest/intro/overview.html
Plotly, Bokeh, Folium	https://plotly.com/python/ https://docs.bokeh.org/en/latest/ https://python-visualization.github.io/folium/latest/
scikit-learn: clustering, PCA	https://scikit-learn.org/stable/modules/clustering.html https://scikit-learn.org/stable/modules/decomposition.html
Scala 3 book, Scala tour, case classes	https://docs.scala-lang.org/scala3/book/introduction.html https://docs.scala-lang.org/tour/tour-of-scala.html https://docs.scala-lang.org/overviews/scala-book/case-classes.html

Topic	Link
Breeze, Saddle, Spire	https://github.com/scalanlp/breeze https://saddle.github.io/ https://github.com/typelevel/spire
Spark RDD guide, SQL guide, cluster overview	https://spark.apache.org/docs/latest/rdd-programming-guide.html https://spark.apache.org/docs/latest/sql-programming-guide.html https://spark.apache.org/docs/latest/cluster-overview.html
Catalyst optimiser (original article)	https://www.databricks.com/blog/2015/04/13/deep-dive-into-spark-sqls-catalyst-optimizer.html
Class vs case class explained	https://www.baeldung.com/scala/case-class
Choosing a language (syllabus reference 7)	https://www.aalpha.net/blog/factors-to-consider-when-choosing-a-programming-language/
Running Scala without installing it	https://github.com/VirtusLab/scala-cli

3. Past papers and how your professors set questions

I did not find an official VIT marking scheme or a published paper for MACSE502 specifically. These community collections hold VIT CAT and FAT papers for other courses; check whether this course appears, and if it does, study the question verbs and mark splits.

- https://papers.codechefvit.com/
- https://github.com/CodeChefVIT/papers-codechef
- https://github.com/Nirant07/VIT-Papers

The best predictors you already own are the 10 high-frequency FAT questions in your FAT notes (section 6.1), the CAT 2 predicted syllabus file, and the 20 lab experiments in the syllabus; this book covers all three. Ask seniors or your faculty for the exact FAT pattern and the weight of lab viva.

Appendix: how the code was verified

File (in Question_Bank/code)	Runtime	Notes
m1_q1_placement.py, m1_q2_editor.py, m1_q6_students.py	Python 3.10	ran as is; output matches the expected sets, trace and report
m1_q3_headlines.py	Python 3.10, NLTK 3.10.4	API simulated with a mock session returning real `requests.Response` objects; needs `punkt_tab`, `stopwords`, `wordnet`
m1_q4_chatbot.py	Python 3.10, NLTK, ChatterBot 1.2.15	ChatterBot ran with `ListTrainer`; needs `python -m spacy download en_core_web_sm`; FAQ data is made up
m1_q5_price_alerts.py	Python 3.10	mock session (timeout, 429 with Retry-After, 404, invalid JSON); `time.sleep` omitted
m2_q1 to m2_q3, m2_q6	pandas 2.3.3, NumPy 1.26.4, scikit-learn 1.7.2	ran as is; random values depend on version and seed
m2_q4_scrape_bs4.py, m2_q4_scrape_scrapy.py	Beautiful Soup 4.15, Scrapy 2.19	page is a local file (`file://` for Scrapy)
m2_q5_viz.py	Plotly 7.1, Bokeh 3.9.2, Folium 0.20	HTML files were generated and their contents checked for size; not viewed in a browser
m3_q1 to m3_q4 (.scala)	Scala 2.13.12, Java 17, scala-cli	ran as is
m4_q1_exceptions.scala	Scala 2.13.12	ran as is
m4_q2_breeze.scala	Breeze 2.1.0, Spire 0.18.0	ran; two start-up messages from Breeze's native loader omitted in the book
m4_q3_saddle.scala	Saddle 4.0.0-M14 (io.github.pityka fork)	milestone release; API differs from older Saddle used in your lab file
m4_q4_spark.scala	Spark 3.5.1, `local[2]`	local mode on one machine; plans and stages are as on a cluster
m5_decision_matrix.py	Python 3.10	scores are judgements, arithmetic is exact

Not verified

- Interactive charts (Plotly, Bokeh, Folium) were built but not opened in a browser.
- Vegas and Plotly.Scala charts of Labs 18 and 19 were not run; only the Breeze and Saddle parts were.
- No live REST service was called; the mock responses reproduce the status codes and bodies described.
- Module 6 content is general; the syllabus gives no topics.
- Timings (such as NumPy being 38 times faster) are from one machine.

Run it yourself

```
# Python
pip install pandas numpy scikit-learn plotly bokeh folium beautifulsoup4 scrapy nltk requests chatterbot
python -m spacy download en_core_web_sm        # needed by ChatterBot 1.2
python run_and_capture.py m1_q1_placement.py   # output goes to outputs/

# Scala (download scala-cli from https://github.com/VirtusLab/scala-cli)
scala-cli run code/m3_q2_oop.scala --scala 2.13.12
scala-cli run code/m4_q4_spark.scala   # downloads Spark on first run
```